Security and compliance in Amazon DynamoDB

Cloud security at AWS is the highest priority. As an AWS customer, you benefit from a data center and network architecture that is built to meet the requirements of the most security-sensitive organizations.

Security is a shared responsibility between AWS and you. The shared responsibility model describes this as security *of* the cloud and security *in* the cloud:

- **Security of the cloud** – AWS is responsible for protecting the infrastructure that runs AWS services in the AWS Cloud. AWS also provides you with services that you can use securely. The effectiveness of our security is regularly tested and verified by third-party auditors as part of the AWS compliance programs. To learn about the compliance programs that apply to DynamoDB, see AWS Services in Scope by Compliance Program.
- **Security in the cloud** – Your responsibility is determined by the AWS service that you use. You are also responsible for other factors including the sensitivity of your data, your organization's requirements, and applicable laws and regulations.

This documentation will help you understand how to apply the shared responsibility model when using DynamoDB. The following topics show you how to configure DynamoDB to meet your security and compliance objectives. You'll also learn how to use other AWS services that can help you to monitor and secure your DynamoDB resources.

Topics

- AWS managed policies for Amazon DynamoDB
- Using resource-based policies for DynamoDB
- Using attribute-based access control with DynamoDB
- Data protection in DynamoDB
- AWS Identity and Access Management (IAM) and DynamoDB
- Compliance validation by industry for DynamoDB
- Resilience and disaster recovery in Amazon DynamoDB
- Infrastructure security in Amazon DynamoDB
- AWS PrivateLink for DynamoDB
- Configuration and vulnerability analysis in Amazon DynamoDB

- Security best practices for Amazon DynamoDB

AWS managed policies for Amazon DynamoDB

DynamoDB uses AWS managed policies to define a set of permissions the service needs to perform specific actions. DynamoDB maintains and updates its AWS managed policies. You can't change the permissions in AWS managed policies. For more information about AWS managed policies, see AWS managed policies in the IAM User Guide.

DynamoDB may occasionally add additional permissions to an AWS managed policy to support new features. This type of update affects all identities (users, groups, and roles) where the policy is attached. An AWS managed policy is most likely to be updated when a new feature is launched or when new operations become available. DynamoDB will not remove permissions from an AWS managed policy, so policy updates won't break your existing permissions. For a full list of AWS managed policies, see AWS managed policies.

AWS managed policy: DynamoDBReplicationServiceRolePolicy

You can’t attach the `DynamoDBReplicationServiceRolePolicy` policy to your IAM entities. This policy is attached to a service-linked role that allows DynamoDB to perform actions on your behalf. For more information, see Using IAM with global tables.

This policy grants permissions that allow the service-linked role to perform data replication between global table replicas. It also grants administrative permissions to manage global table replicas on your behalf.

Permissions details

This policy grants permissions to do the following:

- dynamodb – Perform data replication and manage table replicas.
- `application-autoscaling` – Retrieve and manage table Auto Scaling settings
- `account` – Retrieve region status for evaluating replica accessibility.
- `iam` – To create the service-linked role for application Auto Scaling in the event that the service-linked role does not already exist.

The definition of this managed policy can be found here.

AWS managed policy: AmazonDynamoDBFullAccess_v2

The scoped-down `AmazonDynamoDBFullAccess_v2` policy grants specific access privileges to users. You can attach the `AmazonDynamoDBFullAccess_v2` policy to your IAM identities. This policy grants administrative access to Amazon DynamoDB resources and grants an IAM identity (such as a user, group, or role) access to the AWS services that DynamoDB is integrated with to use all of DynamoDB features. Using this policy allows access to all of DynamoDB features that are available in the AWS Management Console.

Permissions details

This policy includes the following permissions:

- `Amazon DynamoDB`
- `DynamoDB Accelerator`
- `AWS KMS`
- `AWS Resource Groups Tagging`
- `Lambda`
- `Application Auto Scaling`
- `CloudWatch`
- `Amazon Kinesis`
- `Amazon EC2`
- `IAM`

To review the policy in `JSON` format, see AmazonDynamoDBFullAccess_v2.

AWS managed policy: AmazonDynamoDBReadOnlyAccess

You can attach the `AmazonDynamoDBReadOnlyAccess` policy to your IAM identities.

This policy grants read-only access to Amazon DynamoDB.

Permissions details

This policy includes the following permissions:

- `Amazon DynamoDB` – Provides read-only access to Amazon DynamoDB.
- `Amazon DynamoDB Accelerator (DAX)` – Provides read-only access to Amazon DynamoDB Accelerator (DAX).
- `Application Auto Scaling` – Allows principals to view configurations from Application Auto Scaling. This is required so that users can view automatic scaling policies that are attached to a table.
- `CloudWatch` – Allows principals to view metric data and alarms configured in CloudWatch. This is required so users can view the billable table size and CloudWatch alarms that have been configured for a table.
- `AWS Data Pipeline` – Allows principals to view AWS Data Pipeline and associated objects.
- `Amazon EC2` – Allows principals to view Amazon EC2 VPCs, subnets, and security groups.
- `IAM` – Allows principals to view IAM roles.
- `AWS KMS` – Allows principals to view keys configured in AWS KMS. This is required so users can view AWS KMS keys that they create and manage in their account.
- `Amazon SNS` – Allows principals to list Amazon SNS topics and subscriptions by topic.
- `AWS Resource Groups` – Allows principals to view resource groups and their queries.
- `AWS Resource Groups Tagging` – Allows principals to list all the tagged or previously tagged resources in a Region.
- `Kinesis` – Allows principals to view Kinesis data streams descriptions.
- `Amazon CloudWatch Contributor Insights` – Allow principals to view time series data collected by Contributor Insights rules.

To review the policy in `JSON` format, see AmazonDynamoDBReadOnlyAccess.

DynamoDB updates to AWS managed policies

This table shows updates to the AWS access management policies for DynamoDB.

Change	Description	Date Changed
`AmazonDynamoDBFullAccess` – Deprecated	This policy has been replaced by a scoped-down policy named `AmazonDynamoDBFullAccess_v2` .	April 28, 2025

Change	Description	Date Changed
	The `AmazonDynamoDBFullAccess` policy is deprecated and is no longer recommended.	
`AmazonDynamoDBReadOnlyAccess` update to an existing policy	`AmazonDynamoDBReadOnlyAccess` added the permissions: `dynamodb:GetAbacStatus` and `dynamodb:UpdateAbacStatus`. These permissions allow you to view the ABAC status and enable ABAC for your AWS account in the current Region.	November 18, 2024
`AmazonDynamoDBReadOnlyAccess` update to an existing policy	`AmazonDynamoDBReadOnlyAccess` added the permission `dynamodb:GetResourcePolicy`. This permission provides access to read resource-based policies attached to DynamoDB resources.	March 20, 2024
`DynamoDBReplicationServiceRolePolicy` update to an existing policy	`DynamoDBReplicationServiceRolePolicy` added the permission `dynamodb:GetResourcePolicy`. This permission allows the service-linked role to read resource-based policies attached to DynamoDB resources.	December 15, 2023
`DynamoDBReplicationServiceRolePolicy` update to an existing policy	`DynamoDBReplicationServiceRolePolicy` added the permission `account:ListRegions`. This permission allows the service-linked role to evaluate replica accessibility	May 10, 2023

Change	Description	Date Changed
`DynamoDBRepliccationServiceRolePolicy` added to list of managed policies	Added information about the managed policy `DynamoDBReplicationServiceRolePolicy`, which is used by the DynamoDB global tables service-linked role.	May 10, 2023
DynamoDB global tables started tracking changes	DynamoDB global tables started tracking changes for its AWS managed policies.	May 10, 2023

Using resource-based policies for DynamoDB

DynamoDB supports resource-based policies for tables, indexes, and streams. Resource-based policies let you define access permissions by specifying who has access to each resource, and the actions they are allowed to perform on each resource.

You can attach a resource-based policy to DynamoDB resources, such as a table or a stream. In this policy, you specify permissions for Identity and Access Management (IAM) principals that can perform specific actions on these DynamoDB resources. For example, the policy attached to a table will contain permissions for access to the table and its indexes. As a result, resource-based policies can help you simplify access control for your DynamoDB tables, indexes, and streams, by defining permissions at the resource level. The maximum size of a policy you can attach to a DynamoDB resource is 20 KB.

A significant benefit of using resource-based policies is to simplify cross-account access control for providing cross-account access to IAM principals in different AWS accounts. For more information, see Resource-based policy for cross-account access.

Resource-based policies also support integrations with IAM Access Analyzer external access analyzer and Block Public Access (BPA) capabilities. IAM Access Analyzer reports cross-account access to external entities specified in resource-based policies. It also provides visibility to help you refine permissions and conform to the least privilege principle. BPA helps you prevent public access

to your DynamoDB tables, indexes, and streams, and is automatically enabled in the resource-based policies creation and modification workflows.

Topics

- Create a table with a resource-based policy
- Attach a policy to an DynamoDB existing table
- Attach a resource-based policy to a DynamoDB stream
- Remove a resource-based policy from a DynamoDB table
- Cross-account access with resource-based policies in DynamoDB
- Blocking public access with resource-based policies in DynamoDB
- DynamoDB API operations supported by resource-based policies
- Authorization with IAM identity-based policies and DynamoDB resource-based policies
- DynamoDB resource-based policy examples
- DynamoDB resource-based policy considerations
- DynamoDB resource-based policy best practices

Create a table with a resource-based policy

You can add a resource-based policy while you create a table by using the DynamoDB console, CreateTable API, AWS CLI, AWS SDK, or an CloudFormation template.

AWS CLI

The following example creates a table named *`MusicCollection`* using the `create-table` AWS CLI command. This command also includes the `resource-policy` parameter that adds a resource-based policy to the table. This policy allows the user *`John`* to perform the RestoreTableToPointInTime, GetItem, and PutItem API actions on the table.

Remember to replace the *`italicized`* text with your resource-specific information.

```
aws dynamodb create-table \
    --table-name MusicCollection \
    --attribute-definitions AttributeName=Artist,AttributeType=S
 AttributeName=SongTitle,AttributeType=S \
    --key-schema AttributeName=Artist,KeyType=HASH
 AttributeName=SongTitle,KeyType=RANGE \
```

```
    --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5 \
    --resource-policy \
        "{
            \"Version\": \"2012-10-17\",
            \"Statement\": [
              {
                    \"Effect\": \"Allow\",
                    \"Principal\": {
                        \"AWS\": \"arn:aws:iam::123456789012:user/John\"
                    },
                    \"Action\": [
                        \"dynamodb:RestoreTableToPointInTime\",
                        \"dynamodb:GetItem\",
                        \"dynamodb:DescribeTable\"
                    ],
                    \"Resource\": \"arn:aws:dynamodb:us-
west-2:123456789012:table/MusicCollection\"
                }
            ]
        }"
```

AWS Management Console

1. Sign in to the AWS Management Console and open the DynamoDB console at https://console.aws.amazon.com/dynamodb/.
2. On the dashboard, choose **Create table**.
3. In **Table details**, enter the table name, partition key, and sort key details.
4. In **Table settings**, choose **Customize settings**.
5. (Optional) Specify your options for **Table class**, **Capacity calculator**, **Read/write capacity settings**, **Secondary indexes**, **Encryption at rest**, and **Deletion protection**.
6. In **Resource-based policy**, add a policy to define the access permissions for the table and its indexes. In this policy, you specify who has access to these resources, and the actions they are allowed to perform on each resource. To add a policy, do one of the following:

   - Type or paste a JSON policy document. For details about the IAM policy language, see Creating policies using the JSON editor in the *IAM User Guide*.

> **Tip**
>
> To see examples of resource-based policies in the Amazon DynamoDB Developer Guide, choose **Policy examples**.

- Choose **Add new statement** to add a new statement and enter the information in the provided fields. Repeat this step for as many statements as you would like to add.

> **Important**
>
> Make sure that you resolve any security warnings, errors, or suggestions before you save your policy.

The following IAM policy example allows the user *John* to perform the RestoreTableToPointInTime, GetItem, and PutItem API actions on the table *MusicCollection*.

Remember to replace the *italicized* text with your resource-specific information.

JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:user/username"
      },
      "Action": [
        "dynamodb:RestoreTableToPointInTime",
        "dynamodb:GetItem",
        "dynamodb:PutItem"
      ],
      "Resource": "arn:aws:dynamodb:us-
east-1:123456789012:table/MusicCollection"
    }
```

```
  ]
}
```

7. (Optional) Choose **Preview external access** in the lower-right corner to preview how your new policy affects public and cross-account access to your resource. Before you save your policy, you can check whether it introduces new IAM Access Analyzer findings or resolves existing findings. If you don't see an active analyzer, choose **Go to Access Analyzer** to create an account analyzer in IAM Access Analyzer. For more information, see Preview access.
8. Choose **Create table**.

AWS CloudFormation template

Using the AWS::DynamoDB::Table resource

The following CloudFormation template creates a table with a stream using the AWS::DynamoDB::Table resource. This template also includes resource-based policies that are attached to both the table and the stream.

```
{
    "AWSTemplateFormatVersion": "2010-09-09",
    "Resources": {
        "MusicCollectionTable": {
            "Type": "AWS::DynamoDB::Table",
            "Properties": {
                "AttributeDefinitions": [
                    {
                        "AttributeName": "Artist",
                        "AttributeType": "S"
                    }
                ],
                "KeySchema": [
                    {
                        "AttributeName": "Artist",
                        "KeyType": "HASH"
                    }
                ],
                "BillingMode": "PROVISIONED",
                "ProvisionedThroughput": {
                    "ReadCapacityUnits": 5,
                    "WriteCapacityUnits": 5
                },
                "StreamSpecification": {
```

```
                    "StreamViewType": "OLD_IMAGE",
                    "ResourcePolicy": {
                      "PolicyDocument": {
                        "Version": "2012-10-17",
                        "Statement": [
                          {
                              "Principal": {
                                  "AWS": "arn:aws:iam::111122223333:user/John"
                              },
                              "Effect": "Allow",
                              "Action": [
                                  "dynamodb:GetRecords",
                                  "dynamodb:GetShardIterator",
                                  "dynamodb:DescribeStream"
                              ],
                              "Resource": "*"
                          }
                        ]
                      }
                    }
                },
                "TableName": "MusicCollection",
                "ResourcePolicy": {
                    "PolicyDocument": {
                        "Version": "2012-10-17",
                        "Statement": [
                            {
                                "Principal": {
                                    "AWS": [
                                        "arn:aws:iam::111122223333:user/John"
                                    ]
                                },
                                "Effect": "Allow",
                                "Action": "dynamodb:GetItem",
                                "Resource": "*"
                            }
                        ]
                    }
                }
            }

        }
    }
```

```
}
```

Using the AWS::DynamoDB::GlobalTable resource

The following CloudFormation template creates a table with the AWS::DynamoDB::GlobalTable resource and attaches a resource-based policy to the table and its stream.

```
{
    "AWSTemplateFormatVersion": "2010-09-09",
    "Resources": {
        "GlobalMusicCollection": {
            "Type": "AWS::DynamoDB::GlobalTable",
            "Properties": {
                "TableName": "MusicCollection",
                "AttributeDefinitions": [{
                    "AttributeName": "Artist",
                    "AttributeType": "S"
                }],
                "KeySchema": [{
                    "AttributeName": "Artist",
                    "KeyType": "HASH"
                }],
                "BillingMode": "PAY_PER_REQUEST",
                "StreamSpecification": {
                    "StreamViewType": "NEW_AND_OLD_IMAGES"
                },
                "Replicas": [
                    {
                        "Region": "us-east-1",
                        "ResourcePolicy": {
                            "PolicyDocument": {
                                "Version": "2012-10-17",
                                "Statement": [{
                                    "Principal": {
                                        "AWS": [
                                            "arn:aws:iam::111122223333:user/John"
                                        ]
                                    },
                                    "Effect": "Allow",
                                    "Action": "dynamodb:GetItem",
                                    "Resource": "*"
                                }]
                            }
```

```
                        },
                        "ReplicaStreamSpecification": {
                            "ResourcePolicy": {
                                "PolicyDocument": {
                                    "Version": "2012-10-17",
                                    "Statement": [{
                                        "Principal": {
                                            "AWS":
 "arn:aws:iam::111122223333:user/John"
                                        },
                                        "Effect": "Allow",
                                        "Action": [
                                            "dynamodb:GetRecords",
                                            "dynamodb:GetShardIterator",
                                            "dynamodb:DescribeStream"
                                        ],
                                        "Resource": "*"
                                    }]
                                }
                            }
                        }
                    }
                ]
            }
        }
    }
}
```

Attach a policy to an DynamoDB existing table

You can attach a resource-based policy to an existing table or modify an existing policy by using the DynamoDB console, PutResourcePolicy API, the AWS CLI, AWS SDK, or an CloudFormation template.

AWS CLI example to attach a new policy

The following IAM policy example uses the `put-resource-policy` AWS CLI command to attach a resource-based policy to an existing table. This example allows the user *John* to perform the GetItem, PutItem, UpdateItem, and UpdateTable API actions on an existing table named *MusicCollection*.

Remember to replace the *italicized* text with your resource-specific information.

```
aws dynamodb put-resource-policy \
    --resource-arn arn:aws:dynamodb:us-west-2:123456789012:table/MusicCollection \
    --policy \
        "{
            \"Version\": \"2012-10-17\",
            \"Statement\": [
              {
                    \"Effect\": \"Allow\",
                    \"Principal\": {
                        \"AWS\": \"arn:aws:iam::111122223333:user/John\"
                    },
                    \"Action\": [
                        \"dynamodb:GetItem\",
                        \"dynamodb:PutItem\",
                        \"dynamodb:UpdateItem\",
                        \"dynamodb:UpdateTable\"
                    ],
                    \"Resource\": \"arn:aws:dynamodb:us-
west-2:123456789012:table/MusicCollection\"
                }
            ]
        }"
```

AWS CLI example to conditionally update an existing policy

To conditionally update an existing resource-based policy of a table, you can use the optional `expected-revision-id` parameter. The following example will only update the policy if it exists in DynamoDB and its current revision ID matches the provided `expected-revision-id` parameter.

```
aws dynamodb put-resource-policy \
    --resource-arn arn:aws:dynamodb:us-west-2:123456789012:table/MusicCollection \
    --expected-revision-id 1709841168699 \
    --policy \
        "{
            \"Version\": \"2012-10-17\",
            \"Statement\": [
              {
                    \"Effect\": \"Allow\",
                    \"Principal\": {
                        \"AWS\": \"arn:aws:iam::111122223333:user/John\"
                    },
```

```
                    \"Action\": [
                        \"dynamodb:GetItem\",
                        \"dynamodb:UpdateItem\",
                        \"dynamodb:UpdateTable\"
                    ],
                    \"Resource\": \"arn:aws:dynamodb:us-
west-2:123456789012:table/MusicCollection\"
                }
            ]
        }"
```

AWS Management Console

1. Sign in to the AWS Management Console and open the DynamoDB console at https://console.aws.amazon.com/dynamodb/.
2. From the dashboard, choose an existing table.
3. Navigate to the **Permissions** tab, and choose **Create table policy**.
4. In the resource-based policy editor, add the policy you would like to attach and choose **Create policy**.

   The following IAM policy example allows the user *John* to perform the GetItem, PutItem, UpdateItem, and UpdateTable API actions on an existing table named *MusicCollection*.

   Remember to replace the *italicized* text with your resource-specific information.

   JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::111122223333:user/username"
      },
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:UpdateItem",
        "dynamodb:UpdateTable"
```

```
      ],
      "Resource": "arn:aws:dynamodb:us-
east-1:123456789012:table/MusicCollection"
    }
  ]
}
```

AWS SDK for Java 2.x

The following IAM policy example uses the `putResourcePolicy` method to attach a resource-based policy to an existing table. This policy allows a user to perform the GetItem API action on an existing table.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.PutResourcePolicyRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * Get started with the AWS SDK for Java 2.x
 */
public class PutResourcePolicy {

    public static void main(String[] args) {
        final String usage = """

                Usage:
                    <tableArn> <allowedAWSPrincipal>

                Where:
                    tableArn - The Amazon DynamoDB table ARN to attach the policy to.
 For example, arn:aws:dynamodb:us-west-2:123456789012:table/MusicCollection.
                    allowedAWSPrincipal - Allowed AWS principal ARN that the example
 policy will give access to. For example, arn:aws:iam::123456789012:user/John.
                """;

        if (args.length != 2) {
```

```
            System.out.println(usage);
            System.exit(1);
        }

        String tableArn = args[0];
        String allowedAWSPrincipal = args[1];
        System.out.println("Attaching a resource-based policy to the Amazon DynamoDB
table with ARN " +
                tableArn);
        Region region = Region.US_WEST_2;
        DynamoDbClient ddb = DynamoDbClient.builder()
                .region(region)
                .build();

        String result = putResourcePolicy(ddb, tableArn, allowedAWSPrincipal);
        System.out.println("Revision ID for the attached policy is " + result);
        ddb.close();
    }

    public static String putResourcePolicy(DynamoDbClient ddb, String tableArn, String
allowedAWSPrincipal) {
        String policy = generatePolicy(tableArn, allowedAWSPrincipal);
        PutResourcePolicyRequest request = PutResourcePolicyRequest.builder()
                .policy(policy)
                .resourceArn(tableArn)
                .build();

        try {
            return ddb.putResourcePolicy(request).revisionId();
        } catch (DynamoDbException e) {
            System.err.println(e.getMessage());
            System.exit(1);
        }

        return "";
    }

    private static String generatePolicy(String tableArn, String allowedAWSPrincipal) {
        return "{\n" +
                "    \"Version\": \"2012-10-17\",\n" +,
                "    \"Statement\": [\n" +
                "        {\n" +
                "            \"Effect\": \"Allow\",\n" +
```

```
                "            \"Principal\": {\"AWS\":\"" + allowedAWSPrincipal + "\"},
\n" +
                "            \"Action\": [\n" +
                "                \"dynamodb:GetItem\"\n" +
                "            ],\n" +
                "            \"Resource\": \"" + tableArn + "\"\n" +
                "        }\n" +
                "    ]\n" +
                "}";
    }
}
```

Attach a resource-based policy to a DynamoDB stream

You can attach a resource-based policy to an existing table's stream or modify an existing policy by using the DynamoDB console, PutResourcePolicy API, the AWS CLI, AWS SDK, or an CloudFormation template.

> **Note**
>
> You can't attach a policy to a stream while creating it using the CreateTable or UpdateTable APIs. However, you can modify or delete a policy after a table is deleted. You can also modify or delete the policy of a disabled stream.

AWS CLI

The following IAM policy example uses the `put-resource-policy` AWS CLI command to attach a resource-based policy to the stream of a table named *`MusicCollection`*. This example allows the user *`John`* to perform the GetRecords, GetShardIterator, and DescribeStream API actions on the stream.

Remember to replace the *`italicized`* text with your resource-specific information.

```
aws dynamodb put-resource-policy \
    --resource-arn arn:aws:dynamodb:us-west-2:123456789012:table/MusicCollection/
stream/2024-02-12T18:57:26.492 \
    --policy \
        "{
            \"Version\": \"2012-10-17\",
            \"Statement\": [
```

```
                {
                      \"Effect\": \"Allow\",
                      \"Principal\": {
                          \"AWS\": \"arn:aws:iam::111122223333:user/John\"
                      },
                      \"Action\": [
                          \"dynamodb:GetRecords\",
                          \"dynamodb:GetShardIterator\",
                          \"dynamodb:DescribeStream\"
                      ],
                      \"Resource\": \"arn:aws:dynamodb:us-
west-2:123456789012:table/MusicCollection/stream/2024-02-12T18:57:26.492\"
                }
            ]
        }"
```

AWS Management Console

1. Sign in to the AWS Management Console and open the DynamoDB console at https://console.aws.amazon.com/dynamodb/.
2. On the DynamoDB console dashboard, choose **Tables** and then select an existing table.

   Make sure the table you select has streams turned on. For information about turning on streams for a table, see Enabling a stream.
3. Choose the **Permissions** tab.
4. In **Resource-based policy for active stream**, choose **Create stream policy**.
5. In the **Resource-based policy** editor, add a policy to define the access permissions for the stream. In this policy, you specify who has access to the stream and the actions they are allowed to perform on the stream. To add a policy, do one of the following:

   - Type or paste a JSON policy document. For details about the IAM policy language, see Creating policies using the JSON editor in the *IAM User Guide*.

     > **Tip**
     >
     > To see examples of resource-based policies in the Amazon DynamoDB Developer Guide, choose **Policy examples**.

   - Choose **Add new statement** to add a new statement and enter the information in the provided fields. Repeat this step for as many statements as you would like to add.

> **Important**
>
> Make sure that you resolve any security warnings, errors, or suggestions before you save your policy.

6. (Optional) Choose **Preview external access** in the lower-right corner to preview how your new policy affects public and cross-account access to your resource. Before you save your policy, you can check whether it introduces new IAM Access Analyzer findings or resolves existing findings. If you don't see an active analyzer, choose **Go to Access Analyzer** to create an account analyzer in IAM Access Analyzer. For more information, see Preview access.
7. Choose **Create policy**.

The following IAM policy example attaches a resource-based policy to the stream of a table named *MusicCollection*. This example allows the user *John* to perform the GetRecords, GetShardIterator, and DescribeStream API actions on the stream.

Remember to replace the *italicized* text with your resource-specific information.

JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::111122223333:user/username"
      },
      "Action": [
        "dynamodb:GetRecords",
        "dynamodb:GetShardIterator",
        "dynamodb:DescribeStream"
      ],
      "Resource": [
        "arn:aws:dynamodb:us-east-1:123456789012:table/MusicCollection/
stream/2024-02-12T18:57:26.492"
      ]
    }
```

```
  ]
}
```

Remove a resource-based policy from a DynamoDB table

You can delete a resource-based policy from an existing table by using the DynamoDB console, DeleteResourcePolicy API, the AWS CLI, AWS SDK, or an CloudFormation template.

AWS CLI

The following example uses the `delete-resource-policy` AWS CLI command to remove a resource-based policy from a table named *`MusicCollection`*.

Remember to replace the *`italicized`* text with your resource-specific information.

```
aws dynamodb delete-resource-policy \
    --resource-arn arn:aws:dynamodb:us-west-2:123456789012:table/MusicCollection
```

AWS Management Console

1. Sign in to the AWS Management Console and open the DynamoDB console at https://console.aws.amazon.com/dynamodb/.
2. On the DynamoDB console dashboard, choose **Tables** and then select an existing table.
3. Choose **Permissions**.
4. From the **Manage policy** dropdown, choose **Delete policy**.
5. In the **Delete resource-based policy for table** dialog box, type **`confirm`** to confirm the delete action.
6. Choose **Delete**.

Cross-account access with resource-based policies in DynamoDB

Using a resource-based policy, you can provide cross-account access to resources available in different AWS accounts. All cross-account access allowed by the resource-based policies will be reported through IAM Access Analyzer external access findings if you have an analyzer in the same AWS Region as the resource. IAM Access Analyzer runs policy checks to validate your policy against IAM policy grammar and best practices. These checks generate findings and provide actionable recommendations to help you author policies that are functional and conform to security best

practices. You can view the active findings from IAM Access Analyzer in the **Permissions** tab of the DynamoDB console.

For information about validating policies by using IAM Access Analyzer, see IAM Access Analyzer policy validation in the *IAM User Guide*. To view a list of the warnings, errors, and suggestions that are returned by IAM Access Analyzer, see IAM Access Analyzer policy check reference.

To grant GetItem permission to a user A in account A for accessing a table B in account B, perform the following steps:

1. Attach a resource-based policy to table B that grants permission to user A for performing the `GetItem` action.
2. Attach an identity-based policy to user A that grants it permission to perform the `GetItem` action on table B.

Using the **Preview external access** option available in DynamoDB console, you can preview how your new policy affects public and cross-account access to your resource. Before you save your policy, you can check whether it introduces new IAM Access Analyzer findings or resolves existing findings. If you don't see an active analyzer, choose **Go to Access Analyzer** to create an account analyzer in IAM Access Analyzer. For more information, see Preview access.

The table name parameter in the DynamoDB data plane and control plane APIs accept complete Amazon Resource Name (ARN) of the table to support cross-account operations. If you only provide the table name parameter instead of a complete ARN, the API operation will be performed on the table in the account to which the requestor belongs. For an example of a policy that uses cross-account access, see Resource-based policy for cross-account access.

The resource owner's account will be charged even when a principal from another account is reading from or writing to the DynamoDB table in the owner's account. If the table has provisioned throughput, the sum of all the requests from the owner accounts and the requestors in other accounts will determine if the request will be throttled (if autoscaling is disabled) or scaled up/down if autoscaling is enabled.

The requests will be logged in the CloudTrail logs of both the owner and the requestor accounts so that each of the two accounts can track which account accessed what data.

Share access with cross-account AWS Lambda functions

Lambda functions in account A

1. Go to the IAM console to create an IAM role that will be used as the Lambda execution role for your AWS Lambda function in account A. Add the managed IAM policy `AWSLambdaDynamoDBExecutionRole` which has the required DynamoDB Streams and Lambda invocation permissions. This policy also grants access to all potential DynamoDB Streams resources you may have access to in account A.
2. In the Lambda console, create an AWS Lambda function to process records in a DynamoDB stream and during the setup for the execution role, choose the role you created in the previous step.
3. Provide the Lambda function execution role to the DynamoDB Streams' owner of account B to configure the resource-based policy for cross-account read access.
4. Finish setting up the Lambda function.

DynamoDB Stream in Account B

1. Get the cross-account Lambda execution role from account A that will invoke the Lambda function.
2. On the Amazon DynamoDB console in account B, choose the table for Lambda cross-account trigger. Under the **Exports and streams** tab, locate your DynamoDB stream ARN. Ensure that DynamoDB Stream status is On and note the full stream ARN as you will need it for the resource policy.
3. Under the **Permissions** tab, click the **create stream policy** button to start the visual policy editor. Click the **Add new statement** button or edit the policy if one already exists.
4. Create a policy that specifies the Lambda execution role in account A as the principal and grant the required DynamoDB Stream actions. Make sure to include the actions `dynamodb:DescribeStream`, `dynamodb:GetRecords`, `dynamodb:GetShardIterator`, and `dynamodb:ListShards`. For more information on example resource policies for DynamoDB Streams, see DynamoDB resource-based policy examples.

> **Note**
>
> The cross-account access of control plane APIs has a lower transactions per second (TPS) limit of 500 requests.

Blocking public access with resource-based policies in DynamoDB

Block Public Access (BPA) is a feature that identifies and prevents the attaching of resource-based policies that grant public access to your DynamoDB tables, indexes, or streams across your Amazon Web Services (AWS) accounts. With BPA, you can prevent public access to your DynamoDB resources. BPA performs checks during the creation or modification of a resource-based policy and helps improve your security posture with DynamoDB.

BPA uses automated reasoning to analyze the access granted by your resource-based policy and alerts you if such permissions are found at the time of administering a resource-based policy. The analysis verifies access across all resource-based policy statements, actions, and the set of condition keys used in your policies.

> **Important**
>
> BPA helps protect your resources by preventing public access from being granted through the resource-based policies that are directly attached to your DynamoDB resources, such as tables, indexes, and streams. In addition to using BPA, carefully inspect the following policies to confirm that they do not grant public access:
>
> - Identity-based policies attached to associated AWS principals (for example, IAM roles)
> - Resource-based policies attached to associated AWS resources (for example, AWS Key Management Service (KMS) keys)

You must ensure that the principal doesn't include a `*` entry or that one of the specified condition keys restrict access from principals to the resource. If the resource-based policy grants public access to your table, indexes, or stream across AWS accounts, DynamoDB will block you from creating or modifying the policy until the specification within the policy is corrected and deemed non-public.

You can make a policy non-public by specifying one or more principals inside the `Principal` block. The following resource-based policy example blocks public access by specifying two principals.

```
{
  "Effect": "Allow",
  "Principal": {
    "AWS": [
      "123456789012",
```

```
      "111122223333"
    ]
  },
  "Action": "dynamodb:*",
  "Resource": "*"
}
```

Policies that restrict access by specifying certain condition keys are also not considered public. Along with evaluation of the principal specified in the resource-based policy, the following trusted condition keys are used to complete the evaluation of a resource-based policy for non-public access:

- `aws:PrincipalAccount`
- `aws:PrincipalArn`
- `aws:PrincipalOrgID`
- `aws:PrincipalOrgPaths`
- `aws:SourceAccount`
- `aws:SourceArn`
- `aws:SourceVpc`
- `aws:SourceVpce`
- `aws:UserId`
- `aws:PrincipalServiceName`
- `aws:PrincipalServiceNamesList`
- `aws:PrincipalIsAWSService`
- `aws:Ec2InstanceSourceVpc`
- `aws:SourceOrgID`
- `aws:SourceOrgPaths`

Additionally, for a resource-based policy to be non-public, the values for Amazon Resource Name (ARN) and string keys must not contain wildcards or variables. If your resource-based policy uses the `aws:PrincipalIsAWSService` key, you must make sure that you've set the key value to true.

The following policy limits access to the user `John` in the specified account. The condition makes the `Principal` constrained and not be considered as public.

```
{
  "Effect": "Allow",
  "Principal": {
    "AWS": "*"
  },
  "Action": "dynamodb:*",
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "aws:PrincipalArn": "arn:aws:iam::123456789012:user/John"
    }
  }
}
```

The following example of a non-public resource-based policy constrains `sourceVPC` using the `StringEquals` operator.

JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "*"
      },
      "Action": "dynamodb:*",
      "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/
MusicCollection",
      "Condition": {
        "StringEquals": {
          "aws:SourceVpc": [
            "vpc-91237329"
          ]
        }
      }
    }
  ]
}
```

DynamoDB API operations supported by resource-based policies

This topic lists the API operations that are supported by resource-based policies. However, for cross-account access, you can only use a certain set of DynamoDB APIs through resource-based policies. You can't attach resource-based policies to resource types, such as backups and imports. The IAM actions, which correspond with the APIs operating on these resource types, are excluded from the supported IAM actions in resource-based policies. Because table administrators configure internal table settings within the same account, APIs, such as UpdateTimeToLive and DisableKinesisStreamingDestination, don't support cross-account access through resource-based policies.

The DynamoDB data plane and control plane APIs that support cross-account access also support table name overloading, which lets you specify the table ARN instead of the table name. You can specify table ARN in the `TableName` parameter of these APIs. However, not all of these APIs support cross-account access.

Topics

- Data plane API operations
- PartiQL API operations
- Control plane API operations
- Version 2019.11.21 (Current) global tables API operations
- Version 2017.11.29 (Legacy) global tables API operations
- Tags API operations
- Backup and Restore API operations
- Continuous Backup/Restore (PITR) API operations
- Contributor Insights API operations
- Export API operations
- Import API operations
- Amazon Kinesis Data Streams API operations
- Resource-based policy API operations
- Time-to-Live API operations
- Other API operations
- DynamoDB Streams API operations

Data plane API operations

The following table lists the API-level support provided by data plane API operations for resource-based policies and cross-account access.

Data Plane - Tables/indexes APIs	Resource-based policy support	Cross-account support
DeleteItem	Yes	Yes
GetItem	Yes	Yes
PutItem	Yes	Yes
Query	Yes	Yes
Scan	Yes	Yes
UpdateItem	Yes	Yes
TransactGetItems	Yes	Yes
TransactWriteItems	Yes	Yes
BatchGetItem	Yes	Yes
BatchWriteItem	Yes	Yes

PartiQL API operations

The following table lists the API-level support provided by PartiQL API operations for resource-based policies and cross-account access.

PartiQL APIs	Resource-based policy support	Cross-account support
BatchExecuteStatement	Yes	No
ExecuteStatement	Yes	No

PartiQL APIs	Resource-based policy support	Cross-account support
ExecuteTransaction	Yes	No

Control plane API operations

The following table lists the API-level support provided by control plane API operations for resource-based policies and cross-account access.

Control Plane - Tables APIs	Resource-based policy support	Cross-account support
CreateTable	No	No
DeleteTable	Yes	Yes
DescribeTable	Yes	Yes
UpdateTable	Yes	Yes

Version 2019.11.21 (Current) global tables API operations

The following table lists the API-level support provided by Version 2019.11.21 (Current) global tables API operations for resource-based policies and cross-account access.

Version 2019.11.21 (Current) global tables APIs	Resource-based policy support	Cross-account support
DescribeTableReplicaAutoScaling	Yes	No
UpdateTableReplicaAutoScaling	Yes	No

Version 2017.11.29 (Legacy) global tables API operations

The following table lists the API-level support provided by Version 2017.11.29 (Legacy) global tables API operations for resource-based policies and cross-account access.

Version 2017.11.29 (Legacy) global tables APIs	Resource-based policy support	Cross-account support
CreateGlobalTable	No	No
DescribeGlobalTable	No	No
DescribeGlobalTableSettings	No	No
ListGlobalTables	No	No
UpdateGlobalTable	No	No
UpdateGlobalTableSettings	No	No

Tags API operations

The following table lists the API-level support provided by API operations related to tags for resource-based policies and cross-account access.

Tags APIs	Resource-based policy support	Cross-account support
ListTagsOfResource	Yes	Yes
TagResource	Yes	Yes
UntagResource	Yes	Yes

Backup and Restore API operations

The following table lists the API-level support provided by API operations related to backup and restore for resource-based policies and cross-account access.

Backup and Restore APIs	Resource-based policy support	Cross-account support
CreateBackup	Yes	No
DescribeBackup	No	No
DeleteBackup	No	No
RestoreTableFromBackup	No	No

Continuous Backup/Restore (PITR) API operations

The following table lists the API-level support provided by API operations related to Continuous Backup/Restore (PITR) for resource-based policies and cross-account access.

Continuous Backup/Restore (PITR) APIs	Resource-based policy support	Cross-account support
DescribeContinuousBackups	Yes	No
RestoreTableToPointInTime	Yes	No
UpdateContinuousBackups	Yes	No

Contributor Insights API operations

The following table lists the API-level support provided by API operations related to Continuous Backup/Restore (PITR) for resource-based policies and cross-account access.

Contributor Insights APIs	Resource-based policy support	Cross-account support
DescribeContributorInsights	Yes	No
ListContributorInsights	No	No
UpdateContributorInsights	Yes	No

Export API operations

The following table lists the API-level support provided by Export API operations for resource-based policies and cross-account access.

Export APIs	Resource-based policy support	Cross-account support
DescribeExport	No	No
ExportTableToPointInTime	Yes	No
ListExports	No	No

Import API operations

The following table lists the API-level support provided by Import API operations for resource-based policies and cross-account access.

Import APIs	Resource-based policy support	Cross-account support
DescribeImport	No	No
ImportTable	No	No
ListImports	No	No

Amazon Kinesis Data Streams API operations

The following table lists the API-level support provided by Kinesis Data Streams API operations for resource-based policies and cross-account access.

Kinesis APIs	Resource-based policy support	Cross-account support
DescribeKinesisStreamingDestination	Yes	No
DisableKinesisStreamingDestination	Yes	No
EnableKinesisStreamingDestination	Yes	No
UpdateKinesisStreamingDestination	Yes	No

Resource-based policy API operations

The following table lists the API-level support provided by resource-based policy API operations for resource-based policies and cross-account access.

Resource-based policy APIs	Resource-based policy support	Cross-account support
GetResourcePolicy	Yes	No
PutResourcePolicy	Yes	No
DeleteResourcePolicy	Yes	No

Time-to-Live API operations

The following table lists the API-level support provided by time to live (TTL) API operations for resource-based policies and cross-account access.

TTL APIs	Resource-based policy support	Cross-account support
DescribeTimeToLive	Yes	No
UpdateTimeToLive	Yes	No

Other API operations

The following table lists the API-level support provided by other miscellaneous API operations for resource-based policies and cross-account access.

Other APIs	Resource-based policy support	Cross-account support
DescribeLimits	No	No
DescribeEndpoints	No	No
ListBackups	No	No
ListTables	No	No

DynamoDB Streams API operations

The following table lists the API-level support of DynamoDB Streams APIs for resource-based policies and cross-account access.

DynamoDB Streams APIs	Resource-based policy support	Cross-account support
DescribeStream	Yes	Yes
GetRecords	Yes	Yes
GetShardIterator	Yes	Yes
ListStreams	No	No

Authorization with IAM identity-based policies and DynamoDB resource-based policies

Identity-based policies are attached to an identity, such as IAM users, groups of users, and roles. These are IAM policy documents that control what actions an identity can perform, on which resources, and under what conditions. Identity-based policies can be managed or inline policies.

Resource-based policies are IAM policy documents that you attach to a resource, such as a DynamoDB table. These policies grant the specified principal permission to perform specific actions on that resource and defines under what conditions this applies. For example, the resource-based policy for a DynamoDB table also includes the index associated with the table. Resource-based policies are inline policies. There are no managed resource-based policies.

For more information about these policies, see Identity-based policies and resource-based policies in the *IAM User Guide*.

If the IAM principal is from the same account as the resource owner, a resource-based policy is sufficient to specify access permissions to the resource. You can still choose to have an IAM identity-based policy along with a resource-based policy. For cross-account access, you must explicitly allow access in both the identity and resource policies as specified in Cross-account access with resource-based policies in DynamoDB. When you use both types of policies, a policy is evaluated as described in Determining whether a request is allowed or denied within an account.

> **Important**
>
> If an identity-based policy grants unconditional access to a DynamoDB table (for example, `dynamodb:GetItem` with no conditions), a resource-based policy that allows access with conditions on `dynamodb:Attributes` won't restrict that access. The identity-based policy's unconditional allow takes precedence, and the resource-based policy's conditions are not applied as restrictions. To restrict access to specific attributes, use an explicit `Deny` statement instead of relying solely on conditional `Allow` statements in the resource-based policy.

DynamoDB resource-based policy examples

When you specify an ARN in the `Resource` field of a resource-based policy, the policy takes effect only if the specified ARN matches the ARN of the DynamoDB resource to which it is attached.

> **Note**
>
> Remember to replace the *italicized* text with your resource-specific information.

Resource-based policy for a table

The following resource-based policy attached to a DynamoDB table named *MusicCollection*, gives the IAM users *John* and *Jane* permission to perform GetItem and BatchGetItem actions on the *MusicCollection* resource.

JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Sid": "1111",
        "Effect": "Allow",
        "Principal": {
          "AWS": [
            "arn:aws:iam::111122223333:user/username",
            "arn:aws:iam::111122223333:user/Jane"
          ]
        },
        "Action": [
          "dynamodb:GetItem",
          "dynamodb:BatchGetItem"
        ],
        "Resource": [
          "arn:aws:dynamodb:us-east-1:123456789012:table/MusicCollection"
        ]
    }
  ]
}
```

Resource-based policy for a stream

The following resource-based policy attached to a DynamoDB stream named `2024-02-12T18:57:26.492` gives the IAM users *John* and *Jane* permission to perform GetRecords, GetShardIterator, and DescribeStream API actions on the `2024-02-12T18:57:26.492` resource.

JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Sid": "1111",
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::111122223333:user/username",
          "arn:aws:iam::111122223333:user/Jane"
        ]
      },
      "Action": [
        "dynamodb:DescribeStream",
        "dynamodb:GetRecords",
        "dynamodb:GetShardIterator"
      ],
      "Resource": [
        "arn:aws:dynamodb:us-east-1:123456789012:table/MusicCollection/
stream/2024-02-12T18:57:26.492"
      ]
    }
  ]
}
```

Resource-based policy for access to perform all actions on specified resources

To allow a user to perform all actions on a table and all associated indexes with a table, you can use a wildcard (*) to represent the actions and the resources associated with the table. Using a

wild card character for the resources, will allow the user access to the DynamoDB table and all its associated indexes, including the ones that haven't yet been created. For example, the following policy will give the user *John* permission to perform any actions on the *MusicCollection* table and all of its indexes, including any indexes that will be created in the future.

JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Sid": "1111",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::111122223333:user/role-name"
      },
      "Action": "dynamodb:*",
      "Resource": [
        "arn:aws:dynamodb:us-east-1:123456789012:table/MusicCollection",
        "arn:aws:dynamodb:us-east-1:123456789012:table/MusicCollection/
index/index-name"
      ]
    }
  ]
}
```

Resource-based policy for cross-account access

You can specify permissions for a cross-account IAM identity to access DynamoDB resources. For example, you might need a user from a trusted account to get access to read the contents of your table, with the condition that they access only specific items and specific attributes in those items. The following policy allows access to user *John* from a trusted AWS account ID *111111111111* to access data from a table in account *123456789012* by using the GetItem API. The policy ensures that the user can access only items with a primary key *Jane* and that the user can only retrieve the attributes `Artist` and `SongTitle`, but no other attributes.

> **Important**
>
> If you do not specify the `SPECIFIC_ATTRIBUTES` condition, you'll see all attributes for the items returned.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "CrossAccountTablePolicy",
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::111111111111:user/John"
            },
            "Action": "dynamodb:GetItem",
            "Resource": [
                "arn:aws:dynamodb:us-west-2:123456789012:table/MusicCollection"
            ],
            "Condition": {
                "ForAllValues:StringEquals": {
                    "dynamodb:LeadingKeys": "Jane",
                    "dynamodb:Attributes": [
                        "Artist",
                        "SongTitle"
                    ]
                },
                "StringEquals": {
                    "dynamodb:Select": "SPECIFIC_ATTRIBUTES"
                }
            }
        }
    ]
}
```

In addition to the preceding resource-based policy, the identity-based policy attached to the user *John* also needs to allow the `GetItem` API action for the cross-account access to work. The following is an example of an identity-based policy that you must attach to the user *John*.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "CrossAccountIdentityBasedPolicy",
            "Effect": "Allow",
            "Action": [
                "dynamodb:GetItem"
            ],
            "Resource": [
                "arn:aws:dynamodb:us-east-1:123456789012:table/MusicCollection"
            ],
            "Condition": {
                "ForAllValues:StringEquals": {
                    "dynamodb:LeadingKeys": "Jane",
                    "dynamodb:Attributes": [
                        "Artist",
                        "SongTitle"
                    ]
                },
                "StringEquals": {
                    "dynamodb:Select": "SPECIFIC_ATTRIBUTES"
                }
            }
        }
    ]
}
```

The user John can make a `GetItem` request by specifying the table ARN in the `table-name` parameter for accessing the table *`MusicCollection`* in the account *`123456789012`*.

```
aws dynamodb get-item \
    --table-name arn:aws:dynamodb:us-west-2:123456789012:table/MusicCollection \
    --key '{"Artist": {"S": "Jane"}}' \
    --projection-expression 'Artist, SongTitle' \
    --return-consumed-capacity TOTAL
```

Resource-based policy with IP address conditions

You can apply a condition to restrict source IP addresses, virtual private clouds (VPCs), and VPC endpoint (VPCE). You can specify permissions based on the source addresses of the originating request. For example, you might want to allow a user to access DynamoDB resources only if they are being accessed from a specific IP source, such as a corporate VPN endpoint. Specify these IP addresses in the `Condition` statement.

The following example allows the user *John* access to any DynamoDB resource when the source IPs are `54.240.143.0/24` and `2001:DB8:1234:5678::/64`.

JSON

```
{
  "Id":"PolicyId2",
  "Version":"2012-10-17",
  "Statement":[
    {
      "Sid":"AllowIPmix",
      "Effect":"Allow",
      "Principal": {
        "AWS": "arn:aws:iam::111111111111:user/username"
      },
      "Action":"dynamodb:*",
      "Resource":"*",
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": [
            "54.240.143.0/24",
            "2001:DB8:1234:5678::/64"
          ]
        }
      }
    }
  ]
}
```

You can also deny all access to DynamoDB resources except when the source is a specific VPC endpoint, for example *`vpce-1a2b3c4d`*.

> **Important**
>
> When you use DAX with DynamoDB tables that have IP-based resource policies in IPv6-only environments, you must configure additional access rules. If your resource policy restricts access to the IPv4 address space `0.0.0.0/0` on tables, you must allow access for the IAM role associated with your DAX cluster. Add an `ArnNotEquals` condition to your policy to ensure DAX maintains access to your DynamoDB tables. For more information see, DAX and IPv6.

JSON

```
{
  "Id":"PolicyId",
  "Version":"2012-10-17",
  "Statement": [
    {
      "Sid": "AccessToSpecificVPCEOnly",
      "Principal": "*",
      "Action": "dynamodb:*",
      "Effect": "Deny",
      "Resource": "*",
      "Condition": {
        "StringNotEquals":{
          "aws:sourceVpce":"vpce-1a2b3c4d"
        }
      }
    }
  ]
}
```

Resource-based policy using an IAM role

You can also specify an IAM service role in the resource-based policy. IAM entities that assume this role are bounded by the permissible actions specified for the role and to the specific set of resources within the resource-based policy.

The following example allows an IAM entity to perform all DynamoDB actions on the *`MusicCollection`* and *`MusicCollection`* DynamoDB resources.

JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Sid": "1111",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::111122223333:role/role-name" },
      "Action": "dynamodb:*",
      "Resource": [
        "arn:aws:dynamodb:us-east-1:123456789012:table/MusicCollection",
        "arn:aws:dynamodb:us-east-1:123456789012:table/MusicCollection/*"
      ]
    }
  ]
}
```

DynamoDB resource-based policy considerations

When you define resource-based policies for your DynamoDB resources, the following considerations apply:

General considerations

- The maximum size supported for a resource-based policy document is 20 KB. DynamoDB counts whitespaces when calculating the size of a policy against this limit.
- Subsequent updates to a policy for a given resource are blocked for 15 seconds after a successful update of the policy for the same resource.
- Currently, you can only attach a resource-based policy to existing streams. You can't attach a policy to a stream while creating it.

Global table considerations

- Resource-based policies aren't supported for Global table version 2017.11.29 (Legacy) replicas.
- Within a resource-based policy, if the action for a DynamoDB service-linked role (SLR) to replicate data for a global table is denied, adding or deleting a replica will fail with an error.

- The AWS::DynamoDB::GlobalTable resource doesn’t support creating a replica and adding a resource-based policy to that replica in the same stack update in Regions other than the Region where you deploy the stack update.

Cross-account considerations

- Cross-account access using resource-based policies doesn't support encrypted tables with AWS managed keys because you can't grant cross-account access to the AWS managed KMS policy.

CloudFormation considerations

- Resource-based policies don't support drift detection. If you update a resource-based policy outside of the AWS CloudFormation stack template, you'll need to update the CloudFormation stack with the changes.
- Resource-based policies don't support out of band changes. If you add, update, or delete a policy outside of the CloudFormation template, the change won't be overwritten if there are no changes to the policy within the template.

  For example, say that your template contains a resource-based policy which you later update outside of the template. If you don't make any changes to the policy in the template, the updated policy in DynamoDB won’t be synced with the policy in the template.

  Conversely, say that your template doesn’t contain a resource-based policy, but you add a policy outside of the template. This policy won’t be removed from DynamoDB as long as you don’t add it to the template. When you add a policy to the template and update the stack, the existing policy in DynamoDB will be updated to match the one defined in the template.

DynamoDB resource-based policy best practices

This topic describes the best practices for defining access permissions for your DynamoDB resources and the actions allowed on these resources.

Simplify access control to DynamoDB resources

If the AWS Identity and Access Management principals that need access to a DynamoDB resource are part of the same AWS account as the resource owner, an IAM identity-based policy is not required for each principal. A resource-based policy that is attached to the given resources will suffice. This type of configuration simplifies access control.

Protect your DynamoDB resources with resource-based policies

For all DynamoDB tables and streams, create resource-based policies to enforce access control for these resources. Resource-based policies enable you to centralize permissions at the resource level, simplify access control to DynamoDB tables, indexes, and streams, and reduce administration overhead. If no resource-based policy is specified for a table or a stream, access to the table or stream will be implicitly denied, unless identity-based policies associated with the IAM principals allow access.

Apply least-privilege permissions

When you set permissions with resource-based policies for DynamoDB resources, grant only the permissions required to perform an action. You do this by defining the actions that can be taken on specific resources under specific conditions, also known as least-privilege permissions. You might start with broad permissions while you explore the permissions that are required for your workload or use case. As your use case matures, you can work to reduce the permissions that you grant to work toward least privilege.

Analyze cross-account access activity for generating least-privilege policies

IAM Access Analyzer reports cross-account access to external entities specified in resource-based policies, and provides visibility to help you refine permissions and conform to least privilege. For more information about policy generation, see IAM Access Analyzer policy generation.

Use IAM Access Analyzer to generate least-privilege policies

To grant only the permissions required to perform a task, you can generate policies based on your access activity that is logged in AWS CloudTrail. IAM Access Analyzer analyzes the services and actions that your policies use.

Using attribute-based access control with DynamoDB

Attribute-based access control (ABAC) is an authorization strategy that defines access permissions based on tag conditions in your identity-based policies or other AWS policies, such as resource-based policies and organization IAM policies. You can attach tags to DynamoDB tables, which are then evaluated against the tag-based conditions. The indexes associated with a table inherit the tags you add to the table. You can add up to 50 tags for each DynamoDB table. The maximum size supported for all the tags in a table is 10 KB. For more information about tagging DynamoDB

resources and tagging restrictions, see Tagging resources in DynamoDB and the section called "Tagging restrictions in DynamoDB".

For more information about using tags to control access to AWS resources, see the following topics in the IAM User Guide:

- What is ABAC for AWS
- Controlling access to AWS resources using tags

Using ABAC, you can enforce different access levels for your teams and applications to perform actions on DynamoDB tables using fewer policies. You can specify a tag in the condition element of an IAM policy to control access to your DynamoDB tables or indexes. These conditions determine the level of access an IAM principal, a user, or role, has to DynamoDB tables and indexes. When an IAM principal makes an access request to DynamoDB, the resource and identity's tags are evaluated against the tag conditions in the IAM policy. Thereafter, the policy becomes effective only if tag conditions are met. This enables you to create an IAM policy that effectively says one of the following:

- *Allow the user to manage only those resources that have a tag with a key X and a value Y.*
- *Deny access to all users to resources tagged with a key X.*

For example, you can create a policy that allows users to update a table only if it has the tag key-value pair: `"environment": "staging"`. You can use the aws:ResourceTag condition key to allow or deny access to a table based on the tags that are attached to that table.

You can include attribute-based conditions while creating the policy or later using the AWS Management Console, AWS API, AWS Command Line Interface (AWS CLI), AWS SDK, or AWS CloudFormation.

The following example allows the UpdateItem action on a table named `MusicTable` if it includes a tag key with the name `environment` and value `production`.

JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
```

```
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:UpdateItem"
      ],
      "Resource": "arn:aws:dynamodb:*:*:table/MusicTable",
      "Condition": {
        "StringEquals": {
          "aws:ResourceTag/environment": "production"
        }
      }
    }
  ]
}
```

Topics

- Why should I use ABAC?
- Condition keys to implement ABAC with DynamoDB
- Considerations for using ABAC with DynamoDB
- Enabling ABAC in DynamoDB
- Using ABAC with DynamoDB tables and indexes
- Examples for using ABAC with DynamoDB tables and indexes
- Troubleshooting common ABAC errors for DynamoDB tables and indexes

Why should I use ABAC?

- **Simpler policy management:** You use fewer policies because you don't have to create different policies to define the level of access for each IAM principal.
- **Scalable access control:** Scaling access control is easier with ABAC because you don't have to update your policies when you create new DynamoDB resources. You can use tags to authorize access to IAM principals that contain tags matching the resource's tags. You can onboard new IAM principals or DynamoDB resources and apply appropriate tags to automatically grant the necessary permissions without having to make any policy changes.
- **Fine-grained permission management:** It's a best practice to grant least privilege when you create policies. Using ABAC, you can create tags for the IAM principal, and use them to grant access to specific actions and resources that match the tags on the IAM principal.

- **Alignment with corporate directory:** You can map tags with existing employee attributes from your corporate directory to align your access control policies with your organizational structure.

Condition keys to implement ABAC with DynamoDB

You can use the following condition keys in your AWS policies to control the level of access to your DynamoDB tables and indexes:

- aws:ResourceTag/tag-key: Controls access based on whether or not the tag key-value pair on a DynamoDB table or index matches the tag key and value in a policy. This condition key is relevant to all APIs that operate on an existing table or index.

  The `dynamodb:ResourceTag` conditions are evaluated as if you didn't attach any tags to a resource.
- aws:RequestTag/tag-key: Allows comparing the tag key-value pair that was passed in the request with the tag pair that you specify in the policy. This condition key is relevant to APIs that contain tags as part of the request payload. These APIs include CreateTable and TagResource.
- aws:TagKeys: Compares the tag keys in a request with the keys that you specify in the policy. This condition key is relevant to APIs that contain tags as part of the request payload. These APIs include `CreateTable`, `TagResource`, and `UntagResource`.

Considerations for using ABAC with DynamoDB

When you use ABAC with DynamoDB tables or indexes, the following considerations apply:

- Tagging and ABAC aren't supported for DynamoDB Streams.
- Tagging and ABAC aren't supported for DynamoDB backups. To use ABAC with backups, we recommend that you use AWS Backup.
- Tags aren't preserved in restored tables. You need to add tags to restored tables before you can use tag-based conditions in your policies.

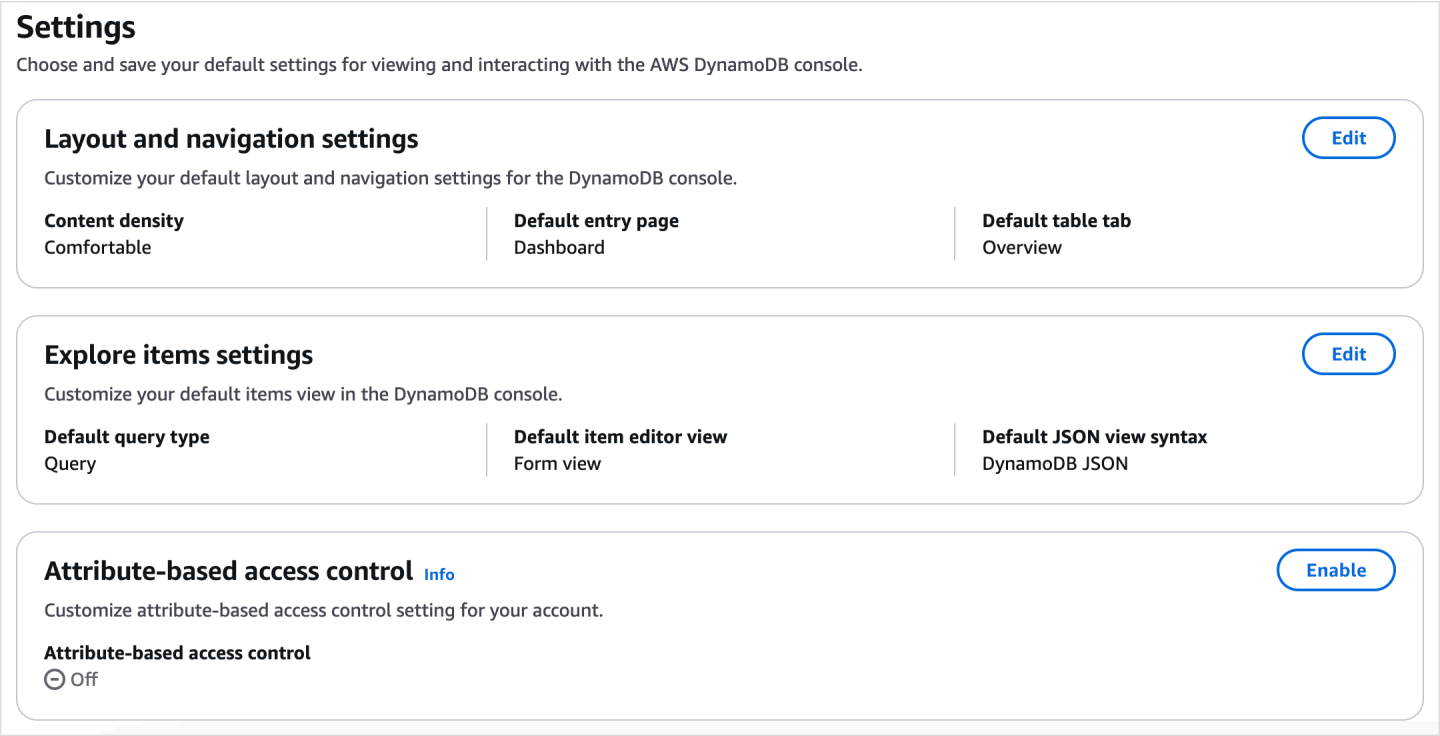
Enabling ABAC in DynamoDB

For most of the AWS accounts, ABAC is enabled by default. Using the DynamoDB console, you can confirm if ABAC is enabled for your account. To do this, make sure that you open the DynamoDB

console with a role that has the dynamodb:GetAbacStatus permission. Then, open the **Settings** page of the DynamoDB console.

If you don't see the **Attribute-based access control** card or if the card displays a status of **On**, it means ABAC is enabled for your account. However, if you see the **Attribute-based access control** card with a status of **Off**, as shown in the following image, ABAC isn't enabled for your account.

Attribute-based access control – not enabled



ABAC isn't enabled for AWS accounts for which tag-based conditions specified in their identity-based policies or other policies still need to be audited. If ABAC isn't enabled for your account, the tag-based conditions in your policies that are intended to act on DynamoDB tables or indexes are evaluated as if no tags are present for your resources or API requests. When ABAC is enabled for your account, the tag-based conditions in the policies of your account are evaluated considering the tags attached to your tables or API requests.

To enable ABAC for your account, we recommend that you first audit your policies as described in the Policy audit section. Then, include the required permissions for ABAC in your IAM policy. Finally, perform the steps described in Enabling ABAC in console to enable ABAC for your account in the current Region. After you enable ABAC, you can opt out within the next seven calendar days of opting in.

Topics

- Auditing your policies before enabling ABAC
- IAM permissions required to enable ABAC
- Enabling ABAC in console

Auditing your policies before enabling ABAC

Before you enable ABAC for your account, audit your policies to confirm that the tag-based conditions which might exist in the policies within your account are set up as intended. Auditing your policies will help avoid surprises from authorization changes with your DynamoDB workflows after ABAC is enabled. To view examples of using attribute-based conditions with tags, and the before and after behavior of ABAC implementation, see Examples for using ABAC with DynamoDB tables and indexes.

IAM permissions required to enable ABAC

You need the `dynamodb:UpdateAbacStatus` permission to enable ABAC for your account in the current Region. To confirm if ABAC is enabled for your account, you must also have the `dynamodb:GetAbacStatus` permission. With this permission, you can view the ABAC status for an account in any Region. You need these permissions in addition to the permission needed for accessing the DynamoDB console.

The following IAM policy grants the permission to enable ABAC and view its status for an account in the current Region.

```
{
"version": "2012-10-17",        &TCX5-2025-waiver;
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "dynamodb:UpdateAbacStatus",
                "dynamodb:GetAbacStatus"
             ],
            "Resource": "*"
        }
    ]
}
```

Enabling ABAC in console

1. Sign in to the AWS Management Console and open the DynamoDB console at https://console.aws.amazon.com/dynamodb/.
2. From the top navigation pane, choose the Region for which you want to enable ABAC.
3. On the left navigation pane, choose **Settings**.
4. On the **Settings** page, do the following:

   a. In the **Attribute-based access control** card, choose **Enable**.

   b. In the **Confirm attribute-based access control setting** box, choose **Enable** to confirm your choice.

   This enables ABAC for the current Region and the **Attribute-based access control** card shows the status of **On**.

   If you want to opt out after enabling ABAC on the console, you can do so within the next seven calendar days of opting in. To opt out, choose **Disable** in the **Attribute-based access control** card on the **Settings** page.

   > **Note**
   >
   > Updating the status of ABAC is an asynchronous operation. If the tags in your policies aren't evaluated right away, you might need to wait for some time because the application of the changes is eventually consistent.

Using ABAC with DynamoDB tables and indexes

The following steps show how to set up permissions using ABAC. In this example scenario, you'll add tags to a DynamoDB table and create an IAM role with a policy that includes tag-based conditions. Then, you'll test the allowed permissions on the DynamoDB table by matching the tag conditions.

Topics

- Step 1: Add tags to a DynamoDB table
- Step 2: Create an IAM role with a policy including tag-based conditions
- Step 3: Test allowed permissions

Step 1: Add tags to a DynamoDB table

You can add tags to new or existing DynamoDB tables using the AWS Management Console, AWS API, AWS Command Line Interface (AWS CLI), AWS SDK, or AWS CloudFormation. For example, the following tag-resource CLI command adds a tag to a table named `MusicTable`.

```
aws dynamodb tag-resource –resource-arn arn:aws:dynamodb:us-east-1:123456789012:table/
MusicTable –tags Key=environment,Value=staging
```

Step 2: Create an IAM role with a policy including tag-based conditions

Create an IAM policy using the aws:ResourceTag/tag-key condition key to compare the tag key-value pair that's specified in the IAM policy with the key-value pair that's attached to the table. The following example policy allows users to put or update items in tables if these tables contain the tag key-value pair: `"environment": "staging"`. If a table doesn't have the specified tag key-value pair, these actions are denied.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "dynamodb:PutItem",
                "dynamodb:UpdateItem"
            ],
            "Resource": "arn:aws:dynamodb:*:*:table/*",
            "Condition": {
                "StringEquals": {
                    "aws:ResourceTag/environment": "staging"
                }
            }
        }
    ]
}
```

Step 3: Test allowed permissions

1. Attach the IAM policy to a test user or role in your AWS account. Make sure that the IAM principal you use doesn't already have access to the DynamoDB table through a different policy.
2. Make sure that your DynamoDB table contains the `"environment"` tag key with a value of `"staging"`.
3. Perform the `dynamodb:PutItem` and `dynamodb:UpdateItem` actions on the tagged table. These actions should succeed if the `"environment": "staging"` tag key-value pair is present.

   If you perform these actions on a table that doesn't have the `"environment": "staging"` tag key-value pair, your request will fail with an `AccessDeniedException`.

You can also review the other sample use cases described in the following section to implement ABAC and perform more tests.

Examples for using ABAC with DynamoDB tables and indexes

The following examples depict some use cases to implement attribute-based conditions using tags.

Topics

- Example 1: Allow an action using aws:ResourceTag
- Example 2: Allow an action using aws:RequestTag
- Example 3: Deny an action using aws:TagKeys

Example 1: Allow an action using aws:ResourceTag

Using the `aws:ResourceTag/tag-key` condition key, you can compare the tag key-value pair that's specified in an IAM policy with the key-value pair that's attached in a DynamoDB table. For example, you can allow a specific action, such as PutItem, if the tag conditions match in an IAM policy and a table. To do this, perform the following steps:

Using the AWS CLI

1. Create a table. The following example uses the create-table AWS CLI command to create a table named `myMusicTable`.

```
aws dynamodb create-table \
  --table-name myMusicTable \
  --attribute-definitions AttributeName=id,AttributeType=S \
  --key-schema AttributeName=id,KeyType=HASH \
  --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5 \
  --region us-east-1
```

2. Add a tag to this table. The following tag-resource AWS CLI command example adds the tag key-value pair `Title: ProductManager` to the `myMusicTable`.

```
aws dynamodb tag-resource --region us-east-1 --resource-arn arn:aws:dynamodb:us-
east-1:123456789012:table/myMusicTable --tags Key=Title,Value=ProductManager
```

3. Create an inline policy and add it to a role which has the AmazonDynamoDBReadOnlyAccess AWS managed policy attached to it, as shown in the following example.

   JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "dynamodb:PutItem",
      "Resource": "arn:aws:dynamodb:*:*:table/*",
      "Condition": {
        "StringEquals": {
          "aws:ResourceTag/Title": "ProductManager"
        }
      }
    }
  ]
}
```

   This policy allows the `PutItem` action on the table when the tag key and value attached to the table matches with the tags specified in the policy.

4. Assume the role with the policies described in Step 3.

5. Use the put-item AWS CLI command to put an item to the `myMusicTable`.

```
aws dynamodb put-item \
    --table-name myMusicTable --region us-east-1 \
    --item '{
        "id": {"S": "2023"},
        "title": {"S": "Happy Day"},
        "info": {"M": {
            "rating": {"N": "9"},
            "Artists": {"L": [{"S": "Acme Band"}, {"S": "No One You Know"}]},
            "release_date": {"S": "2023-07-21"}
        }}
    }'
```

6. Scan the table to verify if the item was added to the table.

```
aws dynamodb scan --table-name myMusicTable  --region us-east-1
```

Using the AWS SDK for Java 2.x

1. Create a table. The following example uses the CreateTable API to create a table named `myMusicTable`.

```
DynamoDbClient dynamoDB = DynamoDbClient.builder().region(region).build();
CreateTableRequest createTableRequest = CreateTableRequest.builder()
    .attributeDefinitions(
        Arrays.asList(
            AttributeDefinition.builder()
            .attributeName("id")
            .attributeType(ScalarAttributeType.S)
            .build()
        )
    )
    .keySchema(
        Arrays.asList(
            KeySchemaElement.builder()
            .attributeName("id")
            .keyType(KeyType.HASH)
            .build()
        )
    )
```

```
    .provisionedThroughput(ProvisionedThroughput.builder()
        .readCapacityUnits(5L)
        .writeCapacityUnits(5L)
        .build()
    )
    .tableName("myMusicTable")
    .build();

CreateTableResponse createTableResponse =
 dynamoDB.createTable(createTableRequest);
String tableArn = createTableResponse.tableDescription().tableArn();
String tableName = createTableResponse.tableDescription().tableName();
```

2. Add a tag to this table. The TagResource API in the following example adds the tag key-value pair `Title: ProductManager` to the `myMusicTable`.

```
TagResourceRequest tagResourceRequest = TagResourceRequest.builder()
    .resourceArn(tableArn)
    .tags(
        Arrays.asList(
            Tag.builder()
            .key("Title")
            .value("ProductManager")
            .build()
        )
    )
    .build();
dynamoDB.tagResource(tagResourceRequest);
```

3. Create an inline policy and add it to a role which has the AmazonDynamoDBReadOnlyAccess AWS managed policy attached to it, as shown in the following example.

   JSON

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "dynamodb:PutItem",
      "Resource": "arn:aws:dynamodb:*:*:table/*",
```

```
        "Condition": {
          "StringEquals": {
            "aws:ResourceTag/Title": "ProductManager"
          }
        }
      }
    ]
  }
```

This policy allows the `PutItem` action on the table when the tag key and value attached to the table matches with the tags specified in the policy.

4. Assume the role with the policies described in Step 3.
5. Use the PutItem API to put an item to the `myMusicTable`.

```
HashMap<String, AttributeValue> info = new HashMap<>();
info.put("rating", AttributeValue.builder().s("9").build());
info.put("artists", AttributeValue.builder().ss(List.of("Acme Band","No One You
 Know").build());
info.put("release_date", AttributeValue.builder().s("2023-07-21").build());

HashMap<String, AttributeValue> itemValues = new HashMap<>();
itemValues.put("id", AttributeValue.builder().s("2023").build());
itemValues.put("title", AttributeValue.builder().s("Happy Day").build());
itemValues.put("info", AttributeValue.builder().m(info).build());


PutItemRequest putItemRequest = PutItemRequest.builder()
                .tableName(tableName)
                .item(itemValues)
                .build();
dynamoDB.putItem(putItemRequest);
```

6. Scan the table to verify if the item was added to the table.

```
ScanRequest scanRequest = ScanRequest.builder()
                .tableName(tableName)
                .build();

ScanResponse scanResponse = dynamoDB.scan(scanRequest);
```

Using the AWS SDK for Python (Boto3)

1. Create a table. The following example uses the CreateTable API to create a table named `myMusicTable`.

```
create_table_response = ddb_client.create_table(
    AttributeDefinitions=[
        {
            'AttributeName': 'id',
            'AttributeType': 'S'
        },
    ],
    TableName='myMusicTable',
    KeySchema=[
        {
            'AttributeName': 'id',
            'KeyType': 'HASH'
        },
    ],
        ProvisionedThroughput={
        'ReadCapacityUnits': 5,
        'WriteCapacityUnits': 5
    },
)

table_arn = create_table_response['TableDescription']['TableArn']
```

2. Add a tag to this table. The TagResource API in the following example adds the tag key-value pair `Title: ProductManager` to the `myMusicTable`.

```
tag_resouce_response = ddb_client.tag_resource(
    ResourceArn=table_arn,
    Tags=[
        {
            'Key': 'Title',
            'Value': 'ProductManager'
        },
    ]
)
```

3. Create an inline policy and add it to a role which has the AmazonDynamoDBReadOnlyAccess AWS managed policy attached to it, as shown in the following example.

   JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
        "Effect": "Allow",
        "Action": "dynamodb:PutItem",
        "Resource": "arn:aws:dynamodb:*:*:table/*",
        "Condition": {
            "StringEquals": {
            "aws:ResourceTag/Title": "ProductManager"
            }
        }
        }
    ]
    }
```

   This policy allows the `PutItem` action on the table when the tag key and value attached to the table matches with the tags specified in the policy.

4. Assume the role with the policies described in Step 3.

5. Use the PutItem API to put an item to the `myMusicTable`.

```
put_item_response = client.put_item(
    TableName = 'myMusicTable'
    Item = {
        'id': '2023',
        'title': 'Happy Day',
        'info': {
            'rating': '9',
            'artists': ['Acme Band','No One You Know'],
            'release_date': '2023-07-21'
        }
    }
)
```

6. Scan the table to verify if the item was added to the table.

```
scan_response = client.scan(
    TableName='myMusicTable'
)
```

Without ABAC

If ABAC isn't enabled for your AWS account, the tag conditions in the IAM policy and the DynamoDB table aren't matched. Consequently, the `PutItem` action returns an `AccessDeniedException` because of the effect of the `AmazonDynamoDBReadOnlyAccess` policy.

```
An error occurred (AccessDeniedException) when calling the PutItem operation:
 User: arn:aws:sts::123456789012:assumed-role/DynamoDBReadOnlyAccess/Alice is
 not authorized to perform: dynamodb:PutItem on resource: arn:aws:dynamodb:us-
east-1:123456789012:table/myMusicTable because no identity-based policy allows the
 dynamodb:PutItem action.
```

With ABAC

If ABAC is enabled for your AWS account, the `put-item` action completes successfully and adds a new item to your table. This is because the inline policy on the table allows the `PutItem` action if the tag conditions in the IAM policy and the table match.

Example 2: Allow an action using aws:RequestTag

Using the aws:RequestTag/tag-key condition key, you can compare the tag key-value pair that's passed in your request with the tag pair that's specified in the IAM policy. For example, you can allow a specific action, such as `CreateTable`, using the `aws:RequestTag` if the tag conditions don't match. To do this, perform the following steps:

Using the AWS CLI

1. Create an inline policy and add it to a role which has the AmazonDynamoDBReadOnlyAccess AWS managed policy attached to it, as shown in the following example.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "dynamodb:CreateTable",
                "dynamodb:TagResource"
            ],
            "Resource": "arn:aws:dynamodb:*:*:table/*",
            "Condition": {
                "StringEquals": {
                    "aws:RequestTag/Owner": "John"
                }
            }
        }
    ]
}
```

2. Create a table that contains the tag key-value pair of `"Owner": "John"`.

```
aws dynamodb create-table \
--attribute-definitions AttributeName=ID,AttributeType=S \
--key-schema AttributeName=ID,KeyType=HASH  \
--provisioned-throughput ReadCapacityUnits=1000,WriteCapacityUnits=500 \
--region us-east-1 \
--tags Key=Owner,Value=John \
--table-name myMusicTable
```

Using the AWS SDK for Python (Boto3)

1. Create an inline policy and add it to a role which has the AmazonDynamoDBReadOnlyAccess AWS managed policy attached to it, as shown in the following example.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "dynamodb:CreateTable",
                "dynamodb:TagResource"
            ],
            "Resource": "arn:aws:dynamodb:*:*:table/*",
            "Condition": {
                "StringEquals": {
                    "aws:RequestTag/Owner": "John"
                }
            }
        }
    ]
}
```

2. Create a table that contains the tag key-value pair of `"Owner": "John"`.

```
ddb_client = boto3.client('dynamodb')

create_table_response = ddb_client.create_table(
    AttributeDefinitions=[
        {
            'AttributeName': 'id',
            'AttributeType': 'S'
        },
    ],
    TableName='myMusicTable',
    KeySchema=[
        {
            'AttributeName': 'id',
            'KeyType': 'HASH'
        },
    ],
        ProvisionedThroughput={
        'ReadCapacityUnits': 1000,
        'WriteCapacityUnits': 500
```

```
    },
    Tags=[
        {
            'Key': 'Owner',
            'Value': 'John'
        },
    ],
)
```

Without ABAC

If ABAC isn't enabled for your AWS account, the tag conditions in the inline policy and the DynamoDB table aren't matched. Consequently, the `CreateTable` request fails and your table isn't created.

```
An error occurred (AccessDeniedException) when calling the CreateTable operation:
 User: arn:aws:sts::123456789012:assumed-role/Admin/John is not authorized to perform:
 dynamodb:CreateTable on resource: arn:aws:dynamodb:us-east-1:123456789012:table/
myMusicTable because no identity-based policy allows the dynamodb:CreateTable action.
```

With ABAC

If ABAC is enabled for your AWS account, your table creation request completes successfully. Because the tag key-value pair of `"Owner": "John"` is present in the `CreateTable` request, the inline policy allows the user `John` to perform the `CreateTable` action.

Example 3: Deny an action using aws:TagKeys

Using the aws:TagKeys condition key, you can compare the tag keys in a request with the keys that are specified in the IAM policy. For example, you can deny a specific action, such as `CreateTable`, using `aws:TagKeys` if a specific tag key is *not* present in the request. To do this, perform the following steps:

Using the AWS CLI

1. Add a customer managed policy to a role which has the AmazonDynamoDBFullAccess AWS managed policy attached to it, as shown in the following example.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Effect": "Deny",
            "Action": [
                "dynamodb:CreateTable",
                "dynamodb:TagResource"
            ],
            "Resource": "arn:aws:dynamodb:*:*:table/*",
            "Condition": {
                "Null": {
                    "aws:TagKeys": "false"
                },
                "ForAllValues:StringNotEquals": {
                    "aws:TagKeys": "CostCenter"
                }
            }
        }
    ]
}
```

2. Assume the role to which the policy was attached, and create a table with the tag key `Title`.

```
aws dynamodb create-table \
--attribute-definitions AttributeName=ID,AttributeType=S \
--key-schema AttributeName=ID,KeyType=HASH  \
--provisioned-throughput ReadCapacityUnits=1000,WriteCapacityUnits=500 \
--region us-east-1 \
--tags Key=Title,Value=ProductManager \
--table-name myMusicTable
```

Using the AWS SDK for Python (Boto3)

1. Add a customer managed policy to a role which has the AmazonDynamoDBFullAccess AWS managed policy attached to it, as shown in the following example.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Effect": "Deny",
            "Action": [
                "dynamodb:CreateTable",
                "dynamodb:TagResource"
            ],
            "Resource": "arn:aws:dynamodb:*:*:table/*",
            "Condition": {
                "Null": {
                    "aws:TagKeys": "false"
                },
                "ForAllValues:StringNotEquals": {
                    "aws:TagKeys": "CostCenter"
                }
            }
        }
    ]
}
```

2. Assume the role to which the policy was attached, and create a table with the tag key `Title`.

```
ddb_client = boto3.client('dynamodb')

create_table_response = ddb_client.create_table(
    AttributeDefinitions=[
        {
            'AttributeName': 'id',
            'AttributeType': 'S'
        },
    ],
    TableName='myMusicTable',
    KeySchema=[
        {
            'AttributeName': 'id',
            'KeyType': 'HASH'
```

```
        },
    ],
        ProvisionedThroughput={
        'ReadCapacityUnits': 1000,
        'WriteCapacityUnits': 500
    },
    Tags=[
        {
            'Key': 'Title',
            'Value': 'ProductManager'
        },
    ],
)
```

Without ABAC

If ABAC isn't enabled for your AWS account, DynamoDB doesn’t send the tag keys in the `create-table` command to IAM. The `Null` condition ensures that the condition evaluates to `false` if there are no tag keys in the request. Because the `Deny` policy doesn't match, the `create-table` command completes successfully.

With ABAC

If ABAC is enabled for your AWS account, the tag keys passed in the `create-table` command are passed to IAM. The tag key `Title` is evaluated against the condition-based tag key, `CostCenter`, present in the `Deny` policy. The tag key `Title` doesn't match the tag key present in the `Deny` policy because of the `StringNotEquals` operator. Therefore, the `CreateTable` action fails and your table isn’t created. Running the `create-table` command returns an `AccessDeniedException`.

```
An error occurred (AccessDeniedException) when calling the CreateTable operation:
 User: arn:aws:sts::123456789012:assumed-role/DynamoFullAccessRole/ProductManager is
 not authorized to perform: dynamodb:CreateTable on resource: arn:aws:dynamodb:us-
east-1:123456789012:table/myMusicTable with an explicit deny in an identity-based
 policy.
```

Troubleshooting common ABAC errors for DynamoDB tables and indexes

This topic provides troubleshooting advice for common errors and issues that you might encounter while implementing ABAC in DynamoDB tables or indexes.

Service-specific condition keys in policies result in an error

Service-specific condition keys aren't considered as valid condition keys. If you've used such keys in your policies, these will result in an error. To fix this issue, you must replace the service-specific condition keys with an appropriate condition key to implement ABAC in DynamoDB.

For example, say that you've used the `dynamodb:ResourceTag` condition key in an inline policy that performs the PutItem request. Imagine that the request fails with an `AccessDeniedException`. The following example shows the erroneous inline policy with the `dynamodb:ResourceTag` condition key.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "dynamodb:PutItem"
            ],
            "Resource": "arn:aws:dynamodb:*:*:table/*",
            "Condition": {
                "StringEquals": {
                    "dynamodb:ResourceTag/Owner": "John"
                }
            }
        }
    ]
}
```

To fix this issue, replace the `dynamodb:ResourceTag` condition key with `aws:ResourceTag`, as shown in the following example.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "dynamodb:PutItem"
            ],
            "Resource": "arn:aws:dynamodb:*:*:table/*",
            "Condition": {
                "StringEquals": {
                    "aws:ResourceTag/Owner": "John"
                }
            }
        }
    ]
}
```

Unable to opt out of ABAC

If ABAC was enabled for your account through Support, you won't be able to opt out of ABAC through the DynamoDB console. To opt out, contact Support.

You can opt out of ABAC yourself *only if* the following are true:

- You used the self-service way of opting in through the DynamoDB console.
- You're opting out within seven calendar days of opting in.

Data protection in DynamoDB

Amazon DynamoDB provides a highly durable storage infrastructure designed for mission-critical and primary data storage. Data is redundantly stored on multiple devices across multiple facilities in an Amazon DynamoDB Region.

DynamoDB protects user data stored at rest and also data in transit between on-premises clients and DynamoDB, and between DynamoDB and other AWS resources within the same AWS Region.

Topics

- DynamoDB encryption at rest
- Securing DynamoDB connections using VPC endpoints and IAM policies"

DynamoDB encryption at rest

All user data stored in Amazon DynamoDB is fully encrypted at rest. DynamoDB encryption at rest provides enhanced security by encrypting all your data at rest using encryption keys stored in AWS Key Management Service (AWS KMS). This functionality helps reduce the operational burden and complexity involved in protecting sensitive data. With encryption at rest, you can build security-sensitive applications that meet strict encryption compliance and regulatory requirements.

DynamoDB encryption at rest provides an additional layer of data protection by always securing your data in an encrypted table—including its primary key, local and global secondary indexes, streams, global tables, backups, and DynamoDB Accelerator (DAX) clusters whenever the data is stored in durable media. Organizational policies, industry or government regulations, and compliance requirements often require the use of encryption at rest to increase the data security of your applications. For more information about encryption for database applications, see AWS Database Encryption SDK.

Encryption at rest integrates with AWS KMS for managing the encryption keys that are used to encrypt your tables. For more information about key types and states, see AWS Key Management Service concepts in the *AWS Key Management Service Developer Guide*.

When creating a new table, you can choose one of the following AWS KMS key types to encrypt your table. You can switch between these key types at any time.

- **AWS owned key –** Default encryption type. The key is owned by DynamoDB (no additional charge).
- **AWS managed key –** The key is stored in your account and is managed by AWS KMS (AWS KMS charges apply).
- **Customer managed key –** The key is stored in your account and is created, owned, and managed by you. You have full control over the KMS key (AWS KMS charges apply).

For more information about key types, see Customer keys and AWS keys.

> **Note**
>
> - When creating a new DAX cluster with encryption at rest enabled, an AWS managed key will be used to encrypt data at rest in the cluster.
> - If your table has a sort key, some of the sort keys that mark range boundaries are stored in plaintext in the table metadata.

When you access an encrypted table, DynamoDB decrypts the table data transparently. You don't have to change any code or applications to use or manage encrypted tables. DynamoDB continues to deliver the same single-digit millisecond latency that you have come to expect, and all DynamoDB queries work seamlessly on your encrypted data.

You can specify an encryption key when you create a new table or switch the encryption keys on an existing table by using the AWS Management Console, AWS Command Line Interface (AWS CLI), or the Amazon DynamoDB API. To learn how, see Managing encrypted tables in DynamoDB.

Encryption at rest using the AWS owned key is offered at no additional charge. However, AWS KMS charges apply for an AWS managed key and for a customer managed key. For more information about pricing, see AWS KMS pricing.

DynamoDB encryption at rest is available in all AWS Regions, including the AWS China (Beijing) and AWS China (Ningxia) Regions and the AWS GovCloud (US) Regions. For more information, see DynamoDB encryption at rest: How it works and DynamoDB encryption at rest usage notes.

DynamoDB encryption at rest: How it works

Amazon DynamoDB encryption at rest encrypts your data using 256-bit Advanced Encryption Standard (AES-256), which helps secure your data from unauthorized access to the underlying storage.

Encryption at rest integrates with AWS Key Management Service (AWS KMS) for managing the encryption keys that are used to encrypt your tables.

> **Note**
>
> In May 2022, AWS KMS changed the rotation schedule for AWS managed keys from every three years (approximately 1,095 days) to every year (approximately 365 days).

New AWS managed keys are automatically rotated one year after they are created, and approximately every year thereafter.
Existing AWS managed keys are automatically rotated one year after their most recent rotation, and every year thereafter.

AWS owned keys

AWS owned keys are not stored in your AWS account. They are part of a collection of KMS keys that AWS owns and manages for use in multiple AWS accounts. AWS services can use AWS owned keys to protect your data. AWS owned keys used by DynamoDB are rotated every year (approximately 365 days).

You cannot view, manage, or use AWS owned keys, or audit their use. However, you do not need to do any work or change any programs to protect the keys that encrypt your data.

You are not charged a monthly fee or a usage fee for use of AWS owned keys, and they do not count against AWS KMS quotas for your account.

AWS managed keys

AWS managed keys are KMS keys in your account that are created, managed, and used on your behalf by an AWS service that is integrated with AWS KMS. You can view the AWS managed keys in your account, view their key policies, and audit their use in AWS CloudTrail logs. However, you cannot manage these KMS keys or change their permissions.

Encryption at rest automatically integrates with AWS KMS for managing the AWS managed keys for DynamoDB (`aws/dynamodb`) that are used to encrypt your tables. If an AWS managed key doesn't exist when you create your encrypted DynamoDB table, AWS KMS automatically creates a new key for you. This key is used with encrypted tables that are created in the future. AWS KMS combines secure, highly available hardware and software to provide a key management system scaled for the cloud.

For more information about managing permissions of the AWS managed key, see Authorizing use of the AWS managed key in the *AWS Key Management Service Developer Guide*.

Customer managed keys

Customer managed keys are KMS keys in your AWS account that you create, own, and manage. You have full control over these KMS keys, including establishing and maintaining their key policies,

IAM policies, and grants; enabling and disabling them; rotating their cryptographic material; adding tags; creating aliases that refer to them; and scheduling them for deletion. For more information about managing permissions of a customer managed key, see Customer managed keys.

When you specify a customer managed key as the table-level encryption key, the DynamoDB table, local and global secondary indexes, and streams are encrypted with the same customer managed key. On-demand backups are encrypted with the table-level encryption key that is specified at the time the backup is created. Updating the table-level encryption key does not change the encryption key that is associated with existing on-demand backups.

Setting the state of the customer managed key to disabled or scheduling it for deletion prevents all users and the DynamoDB service from being able to encrypt or decrypt data and to perform read and write operations on the table. DynamoDB must have access to your encryption key to ensure that you can continue to access your table and to prevent data loss.

If you disable your customer managed key or schedule it for deletion, your table status becomes **Inaccessible**. To ensure that you can continue working with the table, you must provide DynamoDB access to the specified encryption key within seven days. As soon as the service detects that your encryption key is inaccessible, DynamoDB sends you an email notification to alert you.

Note

- If your customer managed key remains inaccessible to the DynamoDB service for longer than seven days, the table is archived and can no longer be accessed. DynamoDB creates an on-demand backup of your table, and you are billed for it. You can use this on-demand backup to restore your data to a new table. To initiate the restore, the last customer managed key on the table must be enabled, and DynamoDB must have access to it.
- If your customer managed key that was used to encrypt a global table replica is inaccessible DynamoDB will remove this replica from the replication group. The replica will not be deleted and replication will stop from and to this region, 20 hours after detecting the customer managed key as inaccessible.

For more information, see enabling keys and deleting keys.

Notes on using AWS managed keys

Amazon DynamoDB can't read your table data unless it has access to the KMS key stored in your AWS KMS account. DynamoDB uses envelope encryption and key hierarchy to encrypt data. Your AWS KMS encryption key is used to encrypt the root key of this key hierarchy. For more information, see Envelope encryption in the *AWS Key Management Service Developer Guide*.

DynamoDB doesn't call AWS KMS for every DynamoDB operation. The key is refreshed once every 5 minutes per caller with active traffic.

Ensure that you have configured the SDK to reuse connections. Otherwise, you will experience latencies from DynamoDB having to reestablish new AWS KMS cache entries for each DynamoDB operation. In addition, you might potentially have to face higher AWS KMS and CloudTrail costs. For example, to do this using the Node.js SDK, you can create a new HTTPS agent with `keepAlive` turned on. For more information, see Configuring keepAlive in Node.js in the *AWS SDK for JavaScript Developer Guide*.

DynamoDB encryption at rest usage notes

Consider the following when you are using encryption at rest in Amazon DynamoDB.

All table data is encrypted

Server-side encryption at rest is enabled on all DynamoDB table data and cannot be disabled. You cannot encrypt only a subset of items in a table.

Encryption at rest only encrypts data while it is static (at rest) on a persistent storage media. If data security is a concern for data in transit or data in use, you might need to take additional measures:

- Data in transit: All your data in DynamoDB is encrypted in transit. By default, communications to and from DynamoDB use the HTTPS protocol, which protects network traffic by using Secure Sockets Layer (SSL)/Transport Layer Security (TLS) encryption.
- Data in use: Protect your data before sending it to DynamoDB using client-side encryption. For more information, see Client-side and server-side encryption in the *Amazon DynamoDB Encryption Client Developer Guide*.

You can use streams with encrypted tables. DynamoDB streams are always encrypted with a table-level encryption key. For more information, see Change data capture for DynamoDB Streams.

DynamoDB backups are encrypted, and the table that is restored from a backup also has encryption enabled. You can use the AWS owned key, AWS managed key, or customer managed key to encrypt your backup data. For more information, see Backup and restore for DynamoDB.

Local secondary indexes and global secondary indexes are encrypted using the same key as the base table.

Encryption types

> **Note**
>
> Customer managed keys are not supported in Global Table Version 2017. If you want to use a customer managed key in a DynamoDB Global Table, you need to upgrade the table to Global Table Version 2019 and then enable it.

On the AWS Management Console, the encryption type is `KMS` when you use the AWS managed key or customer managed key to encrypt your data. The encryption type is `DEFAULT` when you use the AWS owned key. In the Amazon DynamoDB API, the encryption type is `KMS` when you use the AWS managed key or customer managed key. In the absence of encryption type, your data is encrypted using the AWS owned key. You can switch between the AWS owned key, AWS managed key, and customer managed key at any given time. You can use the console, the AWS Command Line Interface (AWS CLI), or the Amazon DynamoDB API to switch the encryption keys.

Note the following limitations when using customer managed keys:

- You cannot use a customer managed key with DynamoDB Accelerator (DAX) clusters. For more information, see DAX encryption at rest.
- You can use a customer managed key to encrypt tables that use transactions. However, to ensure durability for propagation of transactions, a copy of the transaction request is temporarily stored by the service and encrypted using an AWS owned key. Committed data in your tables and secondary indexes is always encrypted at rest using your customer managed key.
- You can use a customer managed key to encrypt tables that use Contributor Insights. However, data that is transmitted to Amazon CloudWatch is encrypted with an AWS owned key.
- When you transition to a new customer managed key, be sure to keep the original key enabled until the process is complete. AWS will still need the original key to decrypt the data before encrypting it with the new key. The process will be complete when the table's SSEDescription

Status is ENABLED and the KMSMasterKeyArn of the new customer managed key is displayed. At this point the original key can be disabled or scheduled for deletion.

- Once the new customer managed key is displayed, the table and any new on-demand backups are encrypted with the new key.
- Any existing on-demand backups remain encrypted with the customer managed key that was used when those backups were created. You will need that same key to restore those backups. You can identify the key for the period when each backup was created by using the DescribeBackup API to view that backup's SSEDescription.
- If you disable your customer managed key or schedule it for deletion, any data in DynamoDB Streams is still subject to a 24-hour lifetime. Any unretrieved activity data is eligible for trimming when it is older than 24 hours.
- If you disable your customer managed key or schedule it for deletion, Time to Live (TTL) deletes continue for 30 minutes. These TTL deletes continue to be emitted to DynamoDB Streams and are subject to the standard trimming/retention interval.

For more information, see enabling keys and deleting keys.

Using KMS keys and data keys

The DynamoDB encryption at rest feature uses an AWS KMS key and a hierarchy of data keys to protect your table data. DynamoDB uses the same key hierarchy to protect DynamoDB streams, global tables, and backups when they are written to durable media.

We recommend that you plan your encryption strategy before implementing your table in DynamoDB. If you store sensitive or confidential data in DynamoDB, consider including client-side encryption in your plan. This way you can encrypt data as close as possible to its origin, and ensure its protection throughout its lifecycle. For more information see the DynamoDB encryption client documentation.

AWS KMS key

Encryption at rest protects your DynamoDB tables under an AWS KMS key. By default, DynamoDB uses an AWS owned key, a multi-tenant encryption key that is created and managed in a DynamoDB service account. But you can encrypt your DynamoDB tables under a customer managed key for DynamoDB (`aws/dynamodb`) in your AWS account. You can select a different KMS key for each table. The KMS key you select for a table is also used to encrypt its local and global secondary indexes, streams, and backups.

You select the KMS key for a table when you create or update the table. You can change the KMS key for a table at any time, either in the DynamoDB console or by using the UpdateTable operation. The process of switching keys is seamless and does not require downtime or degrade service.

> **Important**
>
> DynamoDB supports only symmetric KMS keys. You cannot use an asymmetric KMS key to encrypt your DynamoDB tables.

Use a customer managed key to get the following features:

- You create and manage the KMS key, including setting the key policies, IAM policies and grants to control access to the KMS key. You can enable and disable the KMS key, enable and disable automatic key rotation, and delete the KMS key when it is no longer in use.
- You can use a customer managed key with imported key material or a customer managed key in a custom key store that you own and manage.
- You can audit the encryption and decryption of your DynamoDB table by examining the DynamoDB API calls to AWS KMS in AWS CloudTrail logs.

Use the AWS managed key if you need any of the following features:

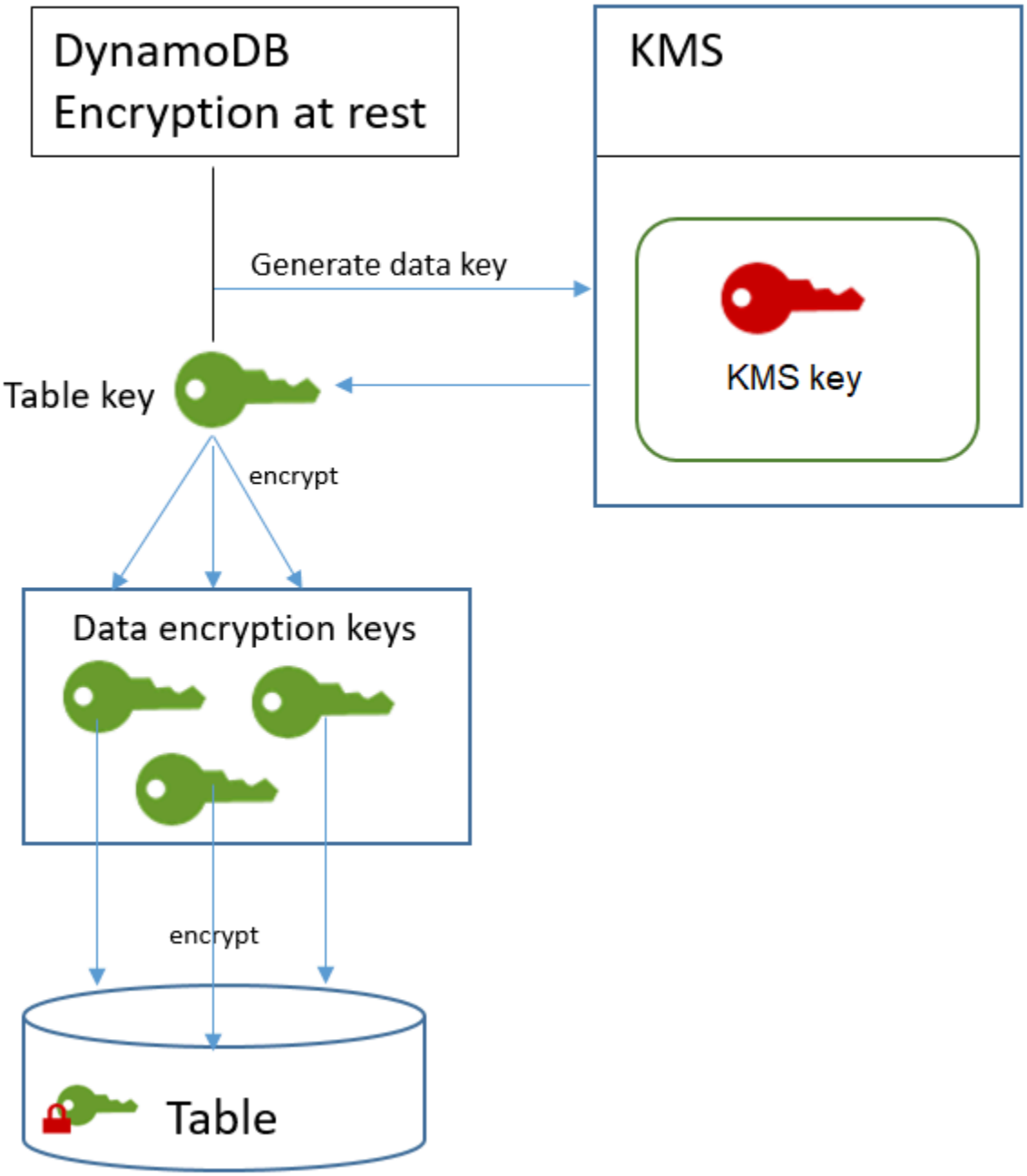
- You can view the KMS key and view its key policy. (You cannot change the key policy.)
- You can audit the encryption and decryption of your DynamoDB table by examining the DynamoDB API calls to AWS KMS in AWS CloudTrail logs.

However, the AWS owned key is free of charge and its use does not count against AWS KMS resource or request quotas. Customer managed keys and AWS managed keys incur a charge for each API call and AWS KMS quotas apply to these KMS keys.

Table keys

DynamoDB uses the KMS key for the table to generate and encrypt a unique data key for the table, known as the *table key*. The table key persists for the lifetime of the encrypted table.

The table key is used as a key encryption key. DynamoDB uses this table key to protect data encryption keys that are used to encrypt the table data. DynamoDB generates a unique data encryption key for each underlying structure in a table, but multiple table items might be protected by the same data encryption key.



When you first access an encrypted table, DynamoDB sends a request to AWS KMS to use the KMS key to decrypt the table key. Then, it uses the plaintext table key to decrypt the data encryption keys, and uses the plaintext data encryption keys to decrypt table data.

DynamoDB stores and uses the table key and data encryption keys outside of AWS KMS. It protects all keys with Advanced Encryption Standard (AES) encryption and 256-bit encryption

keys. Then, it stores the encrypted keys with the encrypted data so they are available to decrypt the table data on demand.

If you change the KMS key for your table, DynamoDB generates a new table key. Then, it uses the new table key to re-encrypt the data encryption keys.

Table key caching

To avoid calling AWS KMS for every DynamoDB operation, DynamoDB caches the plaintext table keys for each caller in memory. If DynamoDB gets a request for the cached table key after five minutes of inactivity, it sends a new request to AWS KMS to decrypt the table key. This call will capture any changes made to the access policies of the KMS key in AWS KMS or AWS Identity and Access Management (IAM) since the last request to decrypt the table key.

Authorizing use of your KMS key

If you use a customer managed key or the AWS managed key in your account to protect your DynamoDB table, the policies on that KMS key must give DynamoDB permission to use it on your behalf. The authorization context on the AWS managed key for DynamoDB includes its key policy and grants that delegate the permissions to use it.

You have full control over the policies and grants on a customer managed key Because the AWS managed key is in your account, you can view its policies and grants. But, because it is managed by AWS, you cannot change the policies.

DynamoDB does not need additional authorization to use the default AWS owned key to protect the DynamoDB tables in your AWS account.

Topics

- Key policy for an AWS managed key
- Key policy for a customer managed key
- Using grants to authorize DynamoDB

Key policy for an AWS managed key

When DynamoDB uses the AWS managed key for DynamoDB (`aws/dynamodb`) in cryptographic operations, it does so on behalf of the user who is accessing the DynamoDB resource. The key policy on the AWS managed key gives all users in the account permission to use the AWS managed key for specified operations. But permission is granted only when DynamoDB makes the request on

the user's behalf. The ViaService condition in the key policy does not allow any user to use the AWS managed key unless the request originates with the DynamoDB service.

This key policy, like the policies of all AWS managed keys, is established by AWS. You cannot change it, but you can view it at any time. For details, see Viewing a key policy.

The policy statements in the key policy have the following effect:

- Allow users in the account to use the AWS managed key for DynamoDB in cryptographic operations when the request comes from DynamoDB on their behalf. The policy also allows users to create grants for the KMS key.
- Allows authorized IAM identities in the account to view the properties of the AWS managed key for DynamoDB and to revoke the grant that allows DynamoDB to use the KMS key. DynamoDB uses grants for ongoing maintenance operations.
- Allows DynamoDB to perform read-only operations to find the AWS managed key for DynamoDB in your account.

JSON

```
{
  "Version":"2012-10-17",
  "Id" : "auto-dynamodb-1",
  "Statement" : [ {
    "Sid" : "Allow access through Amazon DynamoDB for all principals in the
 account that are authorized to use Amazon DynamoDB",
    "Effect" : "Allow",
    "Principal" : {
      "AWS" : "*"
    },
    "Action" : [ "kms:Encrypt", "kms:Decrypt", "kms:ReEncrypt*",
 "kms:GenerateDataKey*", "kms:CreateGrant", "kms:DescribeKey" ],
    "Resource" : "*",
    "Condition" : {
      "StringEquals" : {
        "kms:CallerAccount" : "111122223333",
        "kms:ViaService" : "dynamodb.us-west-2.amazonaws.com"
      }
    }
  }, {
    "Sid" : "Allow direct access to key metadata to the account",
```

```
    "Effect" : "Allow",
    "Principal" : {
      "AWS" : "arn:aws:iam::111122223333:root"
    },
    "Action" : [ "kms:Describe*", "kms:Get*", "kms:List*", "kms:RevokeGrant" ],
    "Resource" : "*"
  }, {
    "Sid" : "Allow DynamoDB Service with service principal name
 dynamodb.amazonaws.com to describe the key directly",
    "Effect" : "Allow",
    "Principal" : {
      "Service" : "dynamodb.amazonaws.com"
    },
    "Action" : [ "kms:Describe*", "kms:Get*", "kms:List*" ],
    "Resource" : "*"
  } ]
}
```

Key policy for a customer managed key

When you select a customer managed key to protect a DynamoDB table, DynamoDB gets permission to use the KMS key on behalf of the principal who makes the selection. That principal, a user or role, must have the permissions on the KMS key that DynamoDB requires. You can provide these permissions in a key policy, an IAM policy, or a grant.

At a minimum, DynamoDB requires the following permissions on a customer managed key:

- kms:Encrypt
- kms:Decrypt
- kms:ReEncrypt* (for kms:ReEncryptFrom and kms:ReEncryptTo)
- kms:GenerateDataKey* (for kms:GenerateDataKey and kms:GenerateDataKeyWithoutPlaintext)
- kms:DescribeKey
- kms:CreateGrant

For example, the following example key policy provides only the required permissions. The policy has the following effects:

- Allows DynamoDB to use the KMS key in cryptographic operations and create grants, but only when it is acting on behalf of principals in the account who have permission to use DynamoDB. If

the principals specified in the policy statement don't have permission to use DynamoDB, the call fails, even when it comes from the DynamoDB service.

- The kms:ViaService condition key allows the permissions only when the request comes from DynamoDB on behalf of the principals listed in the policy statement. These principals can't call these operations directly. Note that the `kms:ViaService` value, `dynamodb.*.amazonaws.com`, has an asterisk (*) in the Region position. DynamoDB requires the permission to be independent of any particular AWS Region so it can make cross-Region calls to support DynamoDB global tables.
- Gives the KMS key administrators (users who can assume the `db-team` role) read-only access to the KMS key and permission to revoke grants, including the grants that DynamoDB requires to protect the table.

Before using an example key policy, replace the example principals with actual principals from your AWS account.

JSON

```
{
  "Id": "key-policy-dynamodb",
  "Version":"2012-10-17",
  "Statement": [
    {
      "Sid" : "Allow access through Amazon DynamoDB for all principals in the
 account that are authorized to use Amazon DynamoDB",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::111122223333:user/db-lead"},
      "Action": [
        "kms:Encrypt",
        "kms:Decrypt",
        "kms:ReEncrypt*",
        "kms:GenerateDataKey*",
        "kms:DescribeKey",
        "kms:CreateGrant"
      ],
      "Resource": "*",
      "Condition": {
         "StringLike": {
           "kms:ViaService" : "dynamodb.*.amazonaws.com"
         }
```

```
      }
    },
    {
      "Sid":  "Allow administrators to view the KMS key and revoke grants",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::111122223333:role/db-team"
       },
      "Action": [
        "kms:Describe*",
        "kms:Get*",
        "kms:List*",
        "kms:RevokeGrant"
      ],
      "Resource": "*"
    }
  ]
}
```

Using grants to authorize DynamoDB

In addition to key policies, DynamoDB uses grants to set permissions on a customer managed key or the AWS managed key for DynamoDB (`aws/dynamodb`). To view the grants on a KMS key in your account, use the ListGrants operation. DynamoDB does not need grants, or any additional permissions, to use the AWS owned key to protect your table.

DynamoDB uses the grant permissions when it performs background system maintenance and continuous data protection tasks. It also uses grants to generate table keys.

Each grant is specific to a table. If the account includes multiple tables encrypted under the same KMS key, there is a grant of each type for each table. The grant is constrained by the DynamoDB encryption context, which includes the table name and the AWS account ID, and it includes permission to the retire the grant if it is no longer needed.

To create the grants, DynamoDB must have permission to call `CreateGrant` on behalf of the user who created the encrypted table. For AWS managed keys, DynamoDB gets `kms:CreateGrant` permission from the key policy, which allows account users to call CreateGrant on the KMS key only when DynamoDB makes the request on an authorized user's behalf.

The key policy can also allow the account to revoke the grant on the KMS key. However, if you revoke the grant on an active encrypted table, DynamoDB will not be able to protect and maintain the table.

DynamoDB encryption context

An encryption context is a set of key–value pairs that contain arbitrary nonsecret data. When you include an encryption context in a request to encrypt data, AWS KMS cryptographically binds the encryption context to the encrypted data. To decrypt the data, you must pass in the same encryption context.

DynamoDB uses the same encryption context in all AWS KMS cryptographic operations. If you use a customer managed key or an AWS managed key to protect your DynamoDB table, you can use the encryption context to identify use of the KMS key in audit records and logs. It also appears in plaintext in logs, such as AWS CloudTrail and Amazon CloudWatch Logs.

The encryption context can also be used as a condition for authorization in policies and grants. DynamoDB uses the encryption context to constrain the grants that allow access to the customer managed key or AWS managed key in your account and region.

In its requests to AWS KMS, DynamoDB uses an encryption context with two key–value pairs.

```
"encryptionContextSubset": {
    "aws:dynamodb:tableName": "Books"
    "aws:dynamodb:subscriberId": "111122223333"
}
```

- **Table** – The first key–value pair identifies the table that DynamoDB is encrypting. The key is `aws:dynamodb:tableName`. The value is the name of the table.

  ```
  "aws:dynamodb:tableName": "<table-name>"
  ```

  For example:

  ```
  "aws:dynamodb:tableName": "Books"
  ```

- **Account** – The second key–value pair identifies the AWS account. The key is `aws:dynamodb:subscriberId`. The value is the account ID.

  ```
  "aws:dynamodb:subscriberId": "<account-id>"
  ```

For example:

```
"aws:dynamodb:subscriberId": "111122223333"
```

Monitoring DynamoDB interaction with AWS KMS

If you use a customer managed key or an AWS managed key to protect your DynamoDB tables, you can use AWS CloudTrail logs to track the requests that DynamoDB sends to AWS KMS on your behalf.

The `GenerateDataKey`, `Decrypt`, and `CreateGrant` requests are discussed in this section. In addition, DynamoDB uses a DescribeKey operation to determine whether the KMS key you selected exists in the account and region. It also uses a RetireGrant operation to remove a grant when you delete a table.

GenerateDataKey

When you enable encryption at rest on a table, DynamoDB creates a unique table key. It sends a GenerateDataKey request to AWS KMS that specifies the KMS key for the table.

The event that records the `GenerateDataKey` operation is similar to the following example event. The user is the DynamoDB service account. The parameters include the Amazon Resource Name (ARN) of the KMS key, a key specifier that requires a 256-bit key, and the encryption context that identifies the table and the AWS account.

```
{
    "eventVersion": "1.05",
    "userIdentity": {
        "type": "AWSService",
        "invokedBy": "dynamodb.amazonaws.com"
    },
    "eventTime": "2018-02-14T00:15:17Z",
    "eventSource": "kms.amazonaws.com",
    "eventName": "GenerateDataKey",
    "awsRegion": "us-west-2",
    "sourceIPAddress": "dynamodb.amazonaws.com",
    "userAgent": "dynamodb.amazonaws.com",
    "requestParameters": {
        "encryptionContext": {
            "aws:dynamodb:tableName": "Services",
```

```
            "aws:dynamodb:subscriberId": "111122223333"
        },
        "keySpec": "AES_256",
        "keyId": "1234abcd-12ab-34cd-56ef-1234567890ab"
    },
    "responseElements": null,
    "requestID": "229386c1-111c-11e8-9e21-c11ed5a52190",
    "eventID": "e3c436e9-ebca-494e-9457-8123a1f5e979",
    "readOnly": true,
    "resources": [
        {
            "ARN": "arn:aws:kms:us-
west-2:111122223333:key/1234abcd-12ab-34cd-56ef-1234567890ab",
            "accountId": "111122223333",
            "type": "AWS::KMS::Key"
        }
    ],
    "eventType": "AwsApiCall",
    "recipientAccountId": "111122223333",
    "sharedEventID": "bf915fa6-6ceb-4659-8912-e36b69846aad"
}
```

Decrypt

When you access an encrypted DynamoDB table, DynamoDB needs to decrypt the table key so that it can decrypt the keys below it in the hierarchy. It then decrypts the data in the table. To decrypt the table key. DynamoDB sends a Decrypt request to AWS KMS that specifies the KMS key for the table.

The event that records the `Decrypt` operation is similar to the following example event. The user is the principal in your AWS account who is accessing the table. The parameters include the encrypted table key (as a ciphertext blob) and the encryption context that identifies the table and the AWS account. AWS KMS derives the ID of the KMS key from the ciphertext.

```
{
    "eventVersion": "1.05",
    "userIdentity": {
        "type": "AssumedRole",
        "principalId": "AROAIGDTESTANDEXAMPLE:user01",
        "arn": "arn:aws:sts::111122223333:assumed-role/Admin/user01",
        "accountId": "111122223333",
        "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
        "sessionContext": {
```

```
            "attributes": {
                "mfaAuthenticated": "false",
                "creationDate": "2018-02-14T16:42:15Z"
            },
            "sessionIssuer": {
                "type": "Role",
                "principalId": "AROAIGDT3HGFQZX4RY6RU",
                "arn": "arn:aws:iam::111122223333:role/Admin",
                "accountId": "111122223333",
                "userName": "Admin"
            }
        },
        "invokedBy": "dynamodb.amazonaws.com"
    },
    "eventTime": "2018-02-14T16:42:39Z",
    "eventSource": "kms.amazonaws.com",
    "eventName": "Decrypt",
    "awsRegion": "us-west-2",
    "sourceIPAddress": "dynamodb.amazonaws.com",
    "userAgent": "dynamodb.amazonaws.com",
    "requestParameters":
    {
        "encryptionContext":
        {
            "aws:dynamodb:tableName": "Books",
            "aws:dynamodb:subscriberId": "111122223333"
        }
    },
    "responseElements": null,
    "requestID": "11cab293-11a6-11e8-8386-13160d3e5db5",
    "eventID": "b7d16574-e887-4b5b-a064-bf92f8ec9ad3",
    "readOnly": true,
    "resources": [
        {
            "ARN": "arn:aws:kms:us-
west-2:111122223333:key/1234abcd-12ab-34cd-56ef-1234567890ab",
            "accountId": "111122223333",
            "type": "AWS::KMS::Key"
        }
    ],
    "eventType": "AwsApiCall",
    "recipientAccountId": "111122223333"
}
```

CreateGrant

When you use a customer managed key or an AWS managed key to protect your DynamoDB table, DynamoDB uses grants to allow the service to perform continuous data protection and maintenance and durability tasks. These grants are not required on AWS owned key.

The grants that DynamoDB creates are specific to a table. The principal in the CreateGrant request is the user who created the table.

The event that records the `CreateGrant` operation is similar to the following example event. The parameters include the Amazon Resource Name (ARN) of the KMS key for the table, the grantee principal and retiring principal (the DynamoDB service), and the operations that the grant covers. It also includes a constraint that requires all encryption operation use the specified encryption context.

```
{
    "eventVersion": "1.05",
    "userIdentity":
    {
        "type": "AssumedRole",
        "principalId": "AROAIGDTESTANDEXAMPLE:user01",
        "arn": "arn:aws:sts::111122223333:assumed-role/Admin/user01",
        "accountId": "111122223333",
        "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
        "sessionContext": {
            "attributes": {
                "mfaAuthenticated": "false",
                "creationDate": "2018-02-14T00:12:02Z"
            },
            "sessionIssuer": {
                "type": "Role",
                "principalId": "AROAIGDTESTANDEXAMPLE",
                "arn": "arn:aws:iam::111122223333:role/Admin",
                "accountId": "111122223333",
                "userName": "Admin"
            }
        },
        "invokedBy": "dynamodb.amazonaws.com"
    },
    "eventTime": "2018-02-14T00:15:15Z",
    "eventSource": "kms.amazonaws.com",
    "eventName": "CreateGrant",
    "awsRegion": "us-west-2",
```

```
    "sourceIPAddress": "dynamodb.amazonaws.com",
    "userAgent": "dynamodb.amazonaws.com",
    "requestParameters": {
        "keyId": "1234abcd-12ab-34cd-56ef-1234567890ab",
        "retiringPrincipal": "dynamodb.us-west-2.amazonaws.com",
        "constraints": {
            "encryptionContextSubset": {
                "aws:dynamodb:tableName": "Books",
                "aws:dynamodb:subscriberId": "111122223333"
            }
        },
        "granteePrincipal": "dynamodb.us-west-2.amazonaws.com",
        "operations": [
            "DescribeKey",
            "GenerateDataKey",
            "Decrypt",
            "Encrypt",
            "ReEncryptFrom",
            "ReEncryptTo",
            "RetireGrant"
        ]
    },
    "responseElements": {
        "grantId":
 "5c5cd4a3d68e65e77795f5ccc2516dff057308172b0cd107c85b5215c6e48bde"
    },
    "requestID": "2192b82a-111c-11e8-a528-f398979205d8",
    "eventID": "a03d65c3-9fee-4111-9816-8bf96b73df01",
    "readOnly": false,
    "resources": [
        {
            "ARN": "arn:aws:kms:us-
west-2:111122223333:key/1234abcd-12ab-34cd-56ef-1234567890ab",
            "accountId": "111122223333",
            "type": "AWS::KMS::Key"
        }
    ],
    "eventType": "AwsApiCall",
    "recipientAccountId": "111122223333"
}
```

Managing encrypted tables in DynamoDB

You can use the AWS Management Console or the AWS Command Line Interface (AWS CLI) to specify the encryption key on new tables and update the encryption keys on existing tables in Amazon DynamoDB.

Topics

- Specifying the encryption key for a new table
- Updating an encryption key

Specifying the encryption key for a new table

Follow these steps to specify the encryption key on a new table using the Amazon DynamoDB console or the AWS CLI.

Creating an encrypted table (console)

1. Sign in to the AWS Management Console and open the DynamoDB console at https://console.aws.amazon.com/dynamodb/.
2. In the navigation pane on the left side of the console, choose **Tables**.
3. Choose **Create Table**. For the **Table name**, enter **`Music`**. For the primary key, enter **`Artist`**, and for the sort key, enter **`SongTitle`**, both as strings.
4. In **Settings**, make sure that **Customize settings** is selected.

   > **Note**
   >
   > If **Use default settings** is selected, tables are encrypted at rest with the AWS owned key at no additional cost.

5. Under **Encryption at rest**, choose an encryption type - AWS owned key, AWS managed key, or customer managed key.

   - **Owned by Amazon DynamoDB.** AWS owned key, specifically owned and managed by DynamoDB. You are not charged an additional fee for using this key.
   - **AWS managed key**. Key alias: `aws/dynamodb`. The key is stored in your account and is managed by AWS Key Management Service (AWS KMS). AWS KMS charges apply.

- **Stored in your account, and owned and managed by you.** Customer managed key. The key is stored in your account and is managed by AWS Key Management Service (AWS KMS). AWS KMS charges apply.

> **Note**
>
> If you select to own and manage your own key, make sure the KMS Key Policy is appropriately set. For more information including examples, see Key policy for a customer managed key.

6. Choose **Create table** to create the encrypted table. To confirm the encryption type, select the table details on the **Overview** tab and review the **Additional details** section.

Creating an encrypted table (AWS CLI)

Use the AWS CLI to create a table with the default AWS owned key, the AWS managed key, or a customer managed key for Amazon DynamoDB.

To create an encrypted table with the default AWS owned key

- Create the encrypted `Music` table as follows.

```
aws dynamodb create-table \
  --table-name Music \
  --attribute-definitions \
      AttributeName=Artist,AttributeType=S \
      AttributeName=SongTitle,AttributeType=S \
  --key-schema \
      AttributeName=Artist,KeyType=HASH \
      AttributeName=SongTitle,KeyType=RANGE \
  --provisioned-throughput \
      ReadCapacityUnits=10,WriteCapacityUnits=5
```

> **Note**
>
> This table is now encrypted using the default AWS owned key in the DynamoDB service account.

To create an encrypted table with the AWS managed key for DynamoDB

- Create the encrypted `Music` table as follows.

```
aws dynamodb create-table \
  --table-name Music \
  --attribute-definitions \
      AttributeName=Artist,AttributeType=S \
      AttributeName=SongTitle,AttributeType=S \
  --key-schema \
      AttributeName=Artist,KeyType=HASH \
      AttributeName=SongTitle,KeyType=RANGE \
  --provisioned-throughput \
      ReadCapacityUnits=10,WriteCapacityUnits=5 \
  --sse-specification Enabled=true,SSEType=KMS
```

The `SSEDescription` status of the table description is set to `ENABLED` and the `SSEType` is `KMS`.

```
"SSEDescription": {
  "SSEType": "KMS",
  "Status": "ENABLED",
  "KMSMasterKeyArn": "arn:aws:kms:us-east-1:123456789012:key/abcd1234-abcd-1234-
a123-ab1234a1b234",
}
```

To create an encrypted table with a customer managed key for DynamoDB

- Create the encrypted `Music` table as follows.

```
aws dynamodb create-table \
  --table-name Music \
  --attribute-definitions \
      AttributeName=Artist,AttributeType=S \
      AttributeName=SongTitle,AttributeType=S \
  --key-schema \
      AttributeName=Artist,KeyType=HASH \
      AttributeName=SongTitle,KeyType=RANGE \
  --provisioned-throughput \
      ReadCapacityUnits=10,WriteCapacityUnits=5 \
```

```
  --sse-specification Enabled=true,SSEType=KMS,KMSMasterKeyId=abcd1234-abcd-1234-
a123-ab1234a1b234
```

> **Note**
>
> For the `KMSMasterKeyId`, you can use a key ID, key ARN, or key alias. If you use a key alias (for example, `alias/my-key`), DynamoDB resolves the alias and associates the underlying AWS KMS key with the table. In the table description, the `KMSMasterKeyArn` will always show the key ARN of the resolved key, not the alias. For more information about key identifiers, see Key identifiers (KeyId) in the *AWS Key Management Service Developer Guide*.

The `SSEDescription` status of the table description is set to `ENABLED` and the `SSEType` is `KMS`.

```
"SSEDescription": {
  "SSEType": "KMS",
  "Status": "ENABLED",
  "KMSMasterKeyArn": "arn:aws:kms:us-east-1:123456789012:key/abcd1234-abcd-1234-
a123-ab1234a1b234",
}
```

Updating an encryption key

You can also use the DynamoDB console or the AWS CLI to update the encryption keys of an existing table between an AWS owned key, AWS managed key, and customer managed key at any time.

Updating an encryption key (console)

1. Sign in to the AWS Management Console and open the DynamoDB console at https://console.aws.amazon.com/dynamodb/.
2. In the navigation pane on the left side of the console, choose **Tables**.
3. Choose the table that you want to update.
4. Select the **Actions** dropdown, and then select the **Update settings** option.
5. Go to the **Additional settings** tab.

6. Under **Encryption**, choose **Manage encryption**.
7. Choose an encryption type:

   - **Owned by Amazon DynamoDB.** The AWS KMS key is owned and managed by DynamoDB. You are not charged an additional fee for using this key.
   - **AWS managed key** Key alias: `aws/dynamodb`. The key is stored in your account and is managed by AWS Key Management Service. (AWS KMS). AWS KMS charges apply.
   - **Stored in your account, and owned and managed by you.** The key is stored in your account and is managed by AWS Key Management Service. (AWS KMS). AWS KMS charges apply.

   > **Note**
   >
   > If you select to own and manage your own key, make sure the KMS Key Policy is appropriately set. For more information see Key policy for a customer managed key.

   Then choose **Save** to update the encrypted table. To confirm the encryption type, check the table details under the **Overview** tab.

Updating an encryption key (AWS CLI)

The following examples show how to update an encrypted table using the AWS CLI.

To update an encrypted table with the default AWS owned key

- Update the encrypted `Music` table, as in the following example.

```
aws dynamodb update-table \
  --table-name Music \
  --sse-specification Enabled=false
```

> **Note**
>
> This table is now encrypted using the default AWS owned key in the DynamoDB service account.

To update an encrypted table with the AWS managed key for DynamoDB

- Update the encrypted `Music` table, as in the following example.

```
aws dynamodb update-table \
  --table-name Music \
  --sse-specification Enabled=true
```

The `SSEDescription` status of the table description is set to `ENABLED` and the `SSEType` is `KMS`.

```
"SSEDescription": {
  "SSEType": "KMS",
  "Status": "ENABLED",
  "KMSMasterKeyArn": "arn:aws:kms:us-east-1:123456789012:key/abcd1234-abcd-1234-
a123-ab1234a1b234",
}
```

To update an encrypted table with a customer managed key for DynamoDB

- Update the encrypted `Music` table, as in the following example.

```
aws dynamodb update-table \
  --table-name Music \
  --sse-specification Enabled=true,SSEType=KMS,KMSMasterKeyId=abcd1234-abcd-1234-
a123-ab1234a1b234
```

> **Note**
>
> For the `KMSMasterKeyId`, you can use a key ID, key ARN, or key alias. If you use a key alias (for example, `alias/my-key`), DynamoDB resolves the alias and associates the underlying AWS KMS key with the table. In the table description, the `KMSMasterKeyArn` will always show the key ARN of the resolved key, not the alias.

The `SSEDescription` status of the table description is set to `ENABLED` and the `SSEType` is `KMS`.

```
"SSEDescription": {
  "SSEType": "KMS",
  "Status": "ENABLED",
  "KMSMasterKeyArn": "arn:aws:kms:us-east-1:123456789012:key/abcd1234-abcd-1234-
a123-ab1234a1b234",
}
```

Securing DynamoDB connections using VPC endpoints and IAM policies"

Connections are protected both between Amazon DynamoDB and on-premises applications and between DynamoDB and other AWS resources within the same AWS Region.

Required policy for endpoints

Amazon DynamoDB provides a DescribeEndpoints API that enables you to enumerate regional endpoint information. For requests to the public DynamoDB endpoints, the API responds regardless of the configured DynamoDB IAM policy, even if there is an explicit or implicit deny in the IAM or VPC endpoint policy. This is because DynamoDB intentionally skips authorization for the `DescribeEndpoints` API.

For requests from a VPC endpoint, both the IAM and Virtual Private Cloud (VPC) endpoint policies must authorize the `DescribeEndpoints` API call for the requesting Identity and Access Management (IAM) principal(s) using the IAM `dynamodb:DescribeEndpoints` action. Otherwise, access to the `DescribeEndpoints` API will be denied.

The following is an example of an endpoints policy.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::111122223333:root"
```

```
            },
            "Action": "dynamodb:DescribeEndpoints",
            "Resource": "*"
        }
    ]
}
```

Traffic between service and on-premises clients and applications

You have two connectivity options between your private network and AWS:

- An AWS Site-to-Site VPN connection. For more information, see What is AWS Site-to-Site VPN? in the *AWS Site-to-Site VPN User Guide*.
- An Direct Connect connection. For more information, see What is Direct Connect? in the *Direct Connect User Guide*.

Access to DynamoDB via the network is through AWS published APIs. Clients must support Transport Layer Security (TLS) 1.2. We recommend TLS 1.3. Clients must also support cipher suites with Perfect Forward Secrecy (PFS), such as Ephemeral Diffie-Hellman (DHE) or Elliptic Curve Diffie-Hellman Ephemeral (ECDHE). Most modern systems such as Java 7 and later support these modes. Additionally, you must sign requests using an access key ID and a secret access key that are associated with an IAM principal, or you can use the AWS Security Token Service (STS) to generate temporary security credentials to sign requests.

Traffic between AWS resources in the same Region

An Amazon Virtual Private Cloud (Amazon VPC) endpoint for DynamoDB is a logical entity within a VPC that allows connectivity only to DynamoDB. The Amazon VPC routes requests to DynamoDB and routes responses back to the VPC. For more information, see VPC endpoints in the *Amazon VPC User Guide*. For example policies that you can use to control access from VPC endpoints, see Using IAM policies to control access to DynamoDB.

> **Note**
>
> Amazon VPC endpoints are not accessible via AWS Site-to-Site VPN or Direct Connect.

AWS Identity and Access Management (IAM) and DynamoDB

AWS Identity and Access Management is an AWS service that helps an administrator securely control access to AWS resources. Administrators control who can be authenticated (signed in) and authorized (have permissions) to use Amazon DynamoDB and DynamoDB Accelerator resources. You can use IAM to manage access permissions and implement security policies for both Amazon DynamoDB and DynamoDB Accelerator. IAM is an AWS service that you can use with no additional charge.

Topics

- Identity and Access Management for Amazon DynamoDB
- Using IAM policy conditions for fine-grained access control

Identity and Access Management for Amazon DynamoDB

AWS Identity and Access Management (IAM) is an AWS service that helps an administrator securely control access to AWS resources. IAM administrators control who can be *authenticated* (signed in) and *authorized* (have permissions) to use DynamoDB resources. IAM is an AWS service that you can use with no additional charge.

Topics

- Audience
- Authenticating with identities
- Managing access using policies
- How Amazon DynamoDB works with IAM
- Identity-based policy examples for Amazon DynamoDB
- Troubleshooting Amazon DynamoDB identity and access
- IAM policy to prevent the purchase of DynamoDB reserved capacity

Audience

How you use AWS Identity and Access Management (IAM) differs based on your role:

- **Service user** - request permissions from your administrator if you cannot access features (see Troubleshooting Amazon DynamoDB identity and access)
- **Service administrator** - determine user access and submit permission requests (see How Amazon DynamoDB works with IAM)
- **IAM administrator** - write policies to manage access (see Identity-based policy examples for Amazon DynamoDB)

Authenticating with identities

Authentication is how you sign in to AWS using your identity credentials. You must be authenticated as the AWS account root user, an IAM user, or by assuming an IAM role.

You can sign in as a federated identity using credentials from an identity source like AWS IAM Identity Center (IAM Identity Center), single sign-on authentication, or Google/Facebook credentials. For more information about signing in, see How to sign in to your AWS account in the *AWS Sign-In User Guide*.

For programmatic access, AWS provides an SDK and CLI to cryptographically sign requests. For more information, see AWS Signature Version 4 for API requests in the *IAM User Guide*.

AWS account root user

When you create an AWS account, you begin with one sign-in identity called the AWS account *root user* that has complete access to all AWS services and resources. We strongly recommend that you don't use the root user for everyday tasks. For tasks that require root user credentials, see Tasks that require root user credentials in the *IAM User Guide*.

Federated identity

As a best practice, require human users to use federation with an identity provider to access AWS services using temporary credentials.

A *federated identity* is a user from your enterprise directory, web identity provider, or Directory Service that accesses AWS services using credentials from an identity source. Federated identities assume roles that provide temporary credentials.

For centralized access management, we recommend AWS IAM Identity Center. For more information, see What is IAM Identity Center? in the *AWS IAM Identity Center User Guide*.

IAM users and groups

An *IAM user* is an identity with specific permissions for a single person or application. We recommend using temporary credentials instead of IAM users with long-term credentials. For more information, see Require human users to use federation with an identity provider to access AWS using temporary credentials in the *IAM User Guide*.

An *IAM group* specifies a collection of IAM users and makes permissions easier to manage for large sets of users. For more information, see Use cases for IAM users in the *IAM User Guide*.

IAM roles

An *IAM role* is an identity with specific permissions that provides temporary credentials. You can assume a role by switching from a user to an IAM role (console) or by calling an AWS CLI or AWS API operation. For more information, see Methods to assume a role in the *IAM User Guide*.

IAM roles are useful for federated user access, temporary IAM user permissions, cross-account access, cross-service access, and applications running on Amazon EC2. For more information, see Cross account resource access in IAM in the *IAM User Guide*.

Managing access using policies

You control access in AWS by creating policies and attaching them to AWS identities or resources. A policy defines permissions when associated with an identity or resource. AWS evaluates these policies when a principal makes a request. Most policies are stored in AWS as JSON documents. For more information about JSON policy documents, see Overview of JSON policies in the *IAM User Guide*.

Using policies, administrators specify who has access to what by defining which **principal** can perform **actions** on what **resources**, and under what **conditions**.

By default, users and roles have no permissions. An IAM administrator creates IAM policies and adds them to roles, which users can then assume. IAM policies define permissions regardless of the method used to perform the operation.

Identity-based policies

Identity-based policies are JSON permissions policy documents that you attach to an identity (user, group, or role). These policies control what actions identities can perform, on which resources, and under what conditions. To learn how to create an identity-based policy, see Define custom IAM permissions with customer managed policies in the *IAM User Guide*.

Identity-based policies can be *inline policies* (embedded directly into a single identity) or *managed policies* (standalone policies attached to multiple identities). To learn how to choose between managed and inline policies, see Choose between managed policies and inline policies in the *IAM User Guide*.

Resource-based policies

Resource-based policies are JSON policy documents that you attach to a resource. Examples include IAM *role trust policies* and Amazon S3 *bucket policies*. In services that support resource-based policies, service administrators can use them to control access to a specific resource. You must specify a principal in a resource-based policy.

Resource-based policies are inline policies that are located in that service. You can't use AWS managed policies from IAM in a resource-based policy.

Other policy types

AWS supports additional policy types that can set the maximum permissions granted by more common policy types:

- **Permissions boundaries** – Set the maximum permissions that an identity-based policy can grant to an IAM entity. For more information, see Permissions boundaries for IAM entities in the *IAM User Guide*.
- **Service control policies (SCPs)** – Specify the maximum permissions for an organization or organizational unit in AWS Organizations. For more information, see Service control policies in the *AWS Organizations User Guide*.
- **Resource control policies (RCPs)** – Set the maximum available permissions for resources in your accounts. For more information, see Resource control policies (RCPs) in the *AWS Organizations User Guide*.
- **Session policies** – Advanced policies passed as a parameter when creating a temporary session for a role or federated user. For more information, see Session policies in the *IAM User Guide*.

Multiple policy types

When multiple types of policies apply to a request, the resulting permissions are more complicated to understand. To learn how AWS determines whether to allow a request when multiple policy types are involved, see Policy evaluation logic in the *IAM User Guide*.

How Amazon DynamoDB works with IAM

Before you use IAM to manage access to DynamoDB, learn what IAM features are available to use with DynamoDB.

IAM feature	DynamoDB support
Identity-based policies	Yes
Resource-based policies	Yes
Policy actions	Yes
Policy resources	Yes
Policy condition keys	Yes
ACLs	No
ABAC (tags in policies)	Yes
Temporary credentials	Yes
Principal permissions	Yes
Service roles	Yes
Service-linked roles	Yes

To get a high-level view of how DynamoDB and other AWS services work with most IAM features, see AWS services that work with IAM in the *IAM User Guide*.

Identity-based policies for DynamoDB

Supports identity-based policies: Yes

Identity-based policies are JSON permissions policy documents that you can attach to an identity, such as an IAM user, group of users, or role. These policies control what actions users and roles can perform, on which resources, and under what conditions. To learn how to create an identity-based policy, see Define custom IAM permissions with customer managed policies in the *IAM User Guide*.

With IAM identity-based policies, you can specify allowed or denied actions and resources as well as the conditions under which actions are allowed or denied. To learn about all of the elements that you can use in a JSON policy, see IAM JSON policy elements reference in the *IAM User Guide*.

Identity-based policy examples for DynamoDB

To view examples of DynamoDB identity-based policies, see Identity-based policy examples for Amazon DynamoDB.

Resource-based policies within DynamoDB

Supports resource-based policies: Yes

Resource-based policies are JSON policy documents that you attach to a resource. Examples of resource-based policies are IAM *role trust policies* and Amazon S3 *bucket policies*. In services that support resource-based policies, service administrators can use them to control access to a specific resource. For the resource where the policy is attached, the policy defines what actions a specified principal can perform on that resource and under what conditions. You must specify a principal in a resource-based policy. Principals can include accounts, users, roles, federated users, or AWS services.

To enable cross-account access, you can specify an entire account or IAM entities in another account as the principal in a resource-based policy. For more information, see Cross account resource access in IAM in the *IAM User Guide*.

Policy actions for DynamoDB

Supports policy actions: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The `Action` element of a JSON policy describes the actions that you can use to allow or deny access in a policy. Include actions in a policy to grant permissions to perform the associated operation.

To see a list of DynamoDB actions, see Actions defined by Amazon DynamoDB in the *Service Authorization Reference*.

Policy actions in DynamoDB use the following prefix before the action:

```
aws
```

To specify multiple actions in a single statement, separate them with commas.

```
"Action": [
      "aws:action1",
      "aws:action2"
         ]
```

To view examples of DynamoDB identity-based policies, see Identity-based policy examples for Amazon DynamoDB.

Policy resources for DynamoDB

Supports policy resources: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The `Resource` JSON policy element specifies the object or objects to which the action applies. As a best practice, specify a resource using its Amazon Resource Name (ARN). For actions that don't support resource-level permissions, use a wildcard (*) to indicate that the statement applies to all resources.

```
"Resource": "*"
```

To see a list of DynamoDB resource types and their ARNs, see Resources defined by Amazon DynamoDB in the *Service Authorization Reference*. To learn with which actions you can specify the ARN of each resource, see Actions defined by Amazon DynamoDB.

To view examples of DynamoDB identity-based policies, see Identity-based policy examples for Amazon DynamoDB.

Policy condition keys for DynamoDB

Supports service-specific policy condition keys: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The `Condition` element specifies when statements execute based on defined criteria. You can create conditional expressions that use condition operators, such as equals or less than, to match the condition in the policy with values in the request. To see all AWS global condition keys, see AWS global condition context keys in the *IAM User Guide*.

To see a list of DynamoDB condition keys, see Condition keys for Amazon DynamoDB in the *Service Authorization Reference*. To learn with which actions and resources you can use a condition key, see Actions defined by Amazon DynamoDB.

To view examples of DynamoDB identity-based policies, see Identity-based policy examples for Amazon DynamoDB.

Access control lists (ACLs) in DynamoDB

Supports ACLs: No

Access control lists (ACLs) control which principals (account members, users, or roles) have permissions to access a resource. ACLs are similar to resource-based policies, although they do not use the JSON policy document format.

Attribute-based access control (ABAC) with DynamoDB

Supports ABAC (tags in policies): Yes

Attribute-based access control (ABAC) is an authorization strategy that defines permissions based on attributes called tags. You can attach tags to IAM entities and AWS resources, then design ABAC policies to allow operations when the principal's tag matches the tag on the resource.

To control access based on tags, you provide tag information in the condition element of a policy using the `aws:ResourceTag/`*`key-name`*, `aws:RequestTag/`*`key-name`*, or `aws:TagKeys` condition keys.

If a service supports all three condition keys for every resource type, then the value is **Yes** for the service. If a service supports all three condition keys for only some resource types, then the value is **Partial**.

For more information about ABAC, see Define permissions with ABAC authorization in the *IAM User Guide*. To view a tutorial with steps for setting up ABAC, see Use attribute-based access control (ABAC) in the *IAM User Guide*.

Using Temporary credentials with DynamoDB

Supports temporary credentials: Yes

Temporary credentials provide short-term access to AWS resources and are automatically created when you use federation or switch roles. AWS recommends that you dynamically generate temporary credentials instead of using long-term access keys. For more information, see Temporary security credentials in IAM and AWS services that work with IAM in the *IAM User Guide*.

Cross-service principal permissions for DynamoDB

Supports forward access sessions (FAS): Yes

Forward access sessions (FAS) use the permissions of the principal calling an AWS service, combined with the requesting AWS service to make requests to downstream services. For policy details when making FAS requests, see Forward access sessions.

Service roles for DynamoDB

Supports service roles: Yes

A service role is an IAM role that a service assumes to perform actions on your behalf. An IAM administrator can create, modify, and delete a service role from within IAM. For more information, see Create a role to delegate permissions to an AWS service in the *IAM User Guide*.

> **Warning**
>
> Changing the permissions for a service role might break DynamoDB functionality. Edit service roles only when DynamoDB provides guidance to do so.

Service-linked roles for DynamoDB

Supports service-linked roles: Yes

A service-linked role is a type of service role that is linked to an AWS service. The service can assume the role to perform an action on your behalf. Service-linked roles appear in your AWS account and are owned by the service. An IAM administrator can view, but not edit the permissions for service-linked roles.

For details about creating or managing service-linked roles, see AWS services that work with IAM. Find a service in the table that includes a `Yes` in the **Service-linked role** column. Choose the **Yes** link to view the service-linked role documentation for that service.

Supported service-linked roles in DynamoDB

The following service-linked roles are supported in DynamoDB.

- DynamoDB uses the service-linked role **AWSServiceRoleForDynamoDBReplication** for global tables replication across AWS Regions. See DynamoDB global tables security for more information about the **AWSServiceRoleForDynamoDBReplication** service-linked role.
- DynamoDB Accelerator (DAX) uses the service-linked role **AWSServiceRoleForDAX** for configuring and maintaining a DAX cluster. See the section called "Using service-linked roles for DAX" for more information about the **AWSServiceRoleForDAX** service-linked role.

In addition to these DynamoDB service-linked roles, DynamoDB uses the Application Auto Scaling service for automatically managing throughput settings on provisioned capacity mode tables. The Application Auto Scaling service uses the service-linked role **AWSServiceRoleForApplicationAutoScaling_DynamoDBTable** to manage throughput settings on DynamoDB tables that have auto scaling enabled. See Service-linked roles for Application Auto Scaling for more information.

Identity-based policy examples for Amazon DynamoDB

By default, users and roles don't have permission to create or modify DynamoDB resources. To grant users permission to perform actions on the resources that they need, an IAM administrator can create IAM policies.

To learn how to create an IAM identity-based policy by using these example JSON policy documents, see Create IAM policies (console) in the *IAM User Guide*.

For details about actions and resource types defined by DynamoDB, including the format of the ARNs for each of the resource types, see Actions, resources, and condition keys for Amazon DynamoDB in the *Service Authorization Reference*.

Topics

- Policy best practices
- Using the DynamoDB console
- Allow users to view their own permissions
- Using identity-based policies with Amazon DynamoDB

Policy best practices

Identity-based policies determine whether someone can create, access, or delete DynamoDB resources in your account. These actions can incur costs for your AWS account. When you create or edit identity-based policies, follow these guidelines and recommendations:

- **Get started with AWS managed policies and move toward least-privilege permissions** – To get started granting permissions to your users and workloads, use the *AWS managed policies* that grant permissions for many common use cases. They are available in your AWS account. We recommend that you reduce permissions further by defining AWS customer managed policies that are specific to your use cases. For more information, see AWS managed policies or AWS managed policies for job functions in the *IAM User Guide*.
- **Apply least-privilege permissions** – When you set permissions with IAM policies, grant only the permissions required to perform a task. You do this by defining the actions that can be taken on specific resources under specific conditions, also known as *least-privilege permissions*. For more information about using IAM to apply permissions, see Policies and permissions in IAM in the *IAM User Guide*.
- **Use conditions in IAM policies to further restrict access** – You can add a condition to your policies to limit access to actions and resources. For example, you can write a policy condition to specify that all requests must be sent using SSL. You can also use conditions to grant access to service actions if they are used through a specific AWS service, such as CloudFormation. For more information, see IAM JSON policy elements: Condition in the *IAM User Guide*.
- **Use IAM Access Analyzer to validate your IAM policies to ensure secure and functional permissions** – IAM Access Analyzer validates new and existing policies so that the policies adhere to the IAM policy language (JSON) and IAM best practices. IAM Access Analyzer provides more than 100 policy checks and actionable recommendations to help you author secure and functional policies. For more information, see Validate policies with IAM Access Analyzer in the *IAM User Guide*.
- **Require multi-factor authentication (MFA)** – If you have a scenario that requires IAM users or a root user in your AWS account, turn on MFA for additional security. To require MFA when API operations are called, add MFA conditions to your policies. For more information, see Secure API access with MFA in the *IAM User Guide*.

For more information about best practices in IAM, see Security best practices in IAM in the *IAM User Guide*.

Using the DynamoDB console

To access the Amazon DynamoDB console, you must have a minimum set of permissions. These permissions must allow you to list and view details about the DynamoDB resources in your AWS account. If you create an identity-based policy that is more restrictive than the minimum required permissions, the console won't function as intended for entities (users or roles) with that policy.

You don't need to allow minimum console permissions for users that are making calls only to the AWS CLI or the AWS API. Instead, allow access to only the actions that match the API operation that they're trying to perform.

To ensure that users and roles can still use the DynamoDB console, also attach the DynamoDB `ConsoleAccess` or `ReadOnly` AWS managed policy to the entities. For more information, see Adding permissions to a user in the *IAM User Guide*.

Allow users to view their own permissions

This example shows how you might create a policy that allows IAM users to view the inline and managed policies that are attached to their user identity. This policy includes permissions to complete this action on the console or programmatically using the AWS CLI or AWS API.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "ViewOwnUserInfo",
            "Effect": "Allow",
            "Action": [
                "iam:GetUserPolicy",
                "iam:ListGroupsForUser",
                "iam:ListAttachedUserPolicies",
                "iam:ListUserPolicies",
                "iam:GetUser"
            ],
            "Resource": ["arn:aws:iam::*:user/${aws:username}"]
        },
        {
            "Sid": "NavigateInConsole",
            "Effect": "Allow",
            "Action": [
                "iam:GetGroupPolicy",
                "iam:GetPolicyVersion",
                "iam:GetPolicy",
```

```
                "iam:ListAttachedGroupPolicies",
                "iam:ListGroupPolicies",
                "iam:ListPolicyVersions",
                "iam:ListPolicies",
                "iam:ListUsers"
            ],
            "Resource": "*"
        }
    ]
}
```

Using identity-based policies with Amazon DynamoDB

This topic covers using identity-based AWS Identity and Access Management (IAM) policies with Amazon DynamoDB and provides examples. The examples show how an account administrator can attach permissions policies to IAM identities (users, groups, and roles) and thereby grant permissions to perform operations on Amazon DynamoDB resources.

The sections in this topic cover the following:

- IAM permissions required to use the Amazon DynamoDB console
- AWS managed (predefined) IAM policies for Amazon DynamoDB
- Customer managed policy examples

The following is an example of a permissions policy.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "DescribeQueryScanBooksTable",
            "Effect": "Allow",
            "Action": [
                "dynamodb:DescribeTable",
                "dynamodb:Query",
                "dynamodb:Scan"
            ],
            "Resource": "arn:aws:dynamodb:us-west-2:111122223333:table/Books"
```

```
        }
    ]
}
```

The preceding policy has one statement that grants permissions for three DynamoDB actions (`dynamodb:DescribeTable`, `dynamodb:Query`, and `dynamodb:Scan`) on a table in the `us-west-2` AWS Region, which is owned by the AWS account specified by *`account-id`*. The *Amazon Resource Name (ARN)* in the `Resource` value specifies the table that the permissions apply to.

IAM permissions required to use the Amazon DynamoDB console

To work with the DynamoDB console, a user must have a minimum set of permissions that allow the user to work with their AWS account's DynamoDB resources. In addition to these DynamoDB permissions, the console requires permissions:

- Amazon CloudWatch permissions to display metrics and graphs.
- AWS Data Pipeline permissions to export and import DynamoDB data.
- AWS Identity and Access Management permissions to access roles necessary for exports and imports.
- Amazon Simple Notification Service permissions to notify you whenever a CloudWatch alarm is triggered.
- AWS Lambda permissions to process DynamoDB Streams records.

If you create an IAM policy that is more restrictive than the minimum required permissions, the console won't function as intended for users with that IAM policy. To ensure that those users can still use the DynamoDB console, also attach the `AmazonDynamoDBReadOnlyAccess` AWS managed policy to the user, as described in AWS managed (predefined) IAM policies for Amazon DynamoDB.

You don't need to allow minimum console permissions for users who are making calls only to the AWS CLI or the Amazon DynamoDB API.

> **Note**
>
> If you refer to a VPC endpoint, you will also need to authorize the DescribeEndpoints API call for the requesting IAM principal(s) with the IAM action (dynamodb:DescribeEndpoints). For more information see Required policy for endpoints.

AWS managed (predefined) IAM policies for Amazon DynamoDB

AWS addresses some common use cases by providing standalone IAM policies that are created and administered by AWS. These AWS managed policies grant necessary permissions for common use cases so that you can avoid having to investigate which permissions are needed. For more information, see AWS Managed Policies in the *IAM User Guide*.

The following AWS managed policies, which you can attach to users in your account, are specific to DynamoDB and are grouped by use-case scenario:

- **AmazonDynamoDBReadOnlyAccess** – Grants read-only access to DynamoDB resources through the AWS Management Console.
- **AmazonDynamoDBFullAccess** – Grants full access to DynamoDB resources through the AWS Management Console.

You can review these AWS managed permissions policies by signing in to the IAM console and searching for specific policies there.

> **Important**
>
> The best practice is to create custom IAM policies that grant least-privilege to the users, roles, or groups that require them.

Customer managed policy examples

In this section, you can find policy examples that grant permissions for various DynamoDB actions. These policies work when you use AWS SDKs or the AWS CLI. When you use the console, you need to grant additional permissions that are specific to the console. For more information, see IAM permissions required to use the Amazon DynamoDB console.

> **Note**
>
> All of the following policy examples use one of the AWS Regions and contain fictitious account IDs and table names.

Examples:

- IAM policy to grant permissions to all DynamoDB actions on a table
- IAM policy to grant read-only permissions on items in a DynamoDB table
- IAM policy to grant access to a specific DynamoDB table and its indexes
- IAM policy to read, write, update, and delete access on a DynamoDB table
- IAM policy to separate DynamoDB environments in the same AWS account
- IAM policy to prevent the purchase of DynamoDB reserved capacity
- IAM policy to grant read access for a DynamoDB stream only (not for the table)
- IAM policy to allow an AWS Lambda function to access DynamoDB stream records
- IAM policy for read and write access to a DynamoDB Accelerator (DAX) cluster

The *IAM User Guide*, includes three additional DynamoDB examples:

- Amazon DynamoDB: Allows Access to a Specific Table
- Amazon DynamoDB: Allows Access to Specific Columns
- Amazon DynamoDB: Allows Row-Level Access to DynamoDB Based on an Amazon Cognito ID

IAM policy to grant permissions to all DynamoDB actions on a table

The following policy grants permissions for *all* DynamoDB actions on a table called `Books`. The resource ARN specified in the `Resource` identifies a table in a specific AWS Region. If you replace the table name `Books` in the `Resource` ARN with a wildcard character (*), *all* DynamoDB actions are allowed on *all* tables in the account. Carefully consider the possible security implications before using a wildcard character on this or any IAM policy.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "AllAPIActionsOnBooks",
            "Effect": "Allow",
            "Action": "dynamodb:*",
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:table/Books"
        }
    ]
```

```
}
```

> **Note**
>
> This is an example of using a wildcard character (*) to allow *all* actions, including administration, data operations, monitoring, and purchase of DynamoDB reserved capacity. Instead, it is a best practice to explicitly specify each action to be granted and only what that user, role, or group needs.

IAM policy to grant read-only permissions on items in a DynamoDB table

The following permissions policy grants permissions for the `GetItem`, `BatchGetItem`, `Scan`, `Query`, and `ConditionCheckItem` DynamoDB actions only, and as a result, sets read-only access on the `Books` table.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "ReadOnlyAPIActionsOnBooks",
            "Effect": "Allow",
            "Action": [
                "dynamodb:GetItem",
                "dynamodb:BatchGetItem",
                "dynamodb:Scan",
                "dynamodb:Query",
                "dynamodb:ConditionCheckItem"
            ],
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:table/Books"
        }
    ]
}
```

IAM policy to grant access to a specific DynamoDB table and its indexes

The following policy grants permissions for data modification actions on a DynamoDB table called Books and all of that table's indexes. For more information about how indexes work, see Improving data access with secondary indexes in DynamoDB.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "AccessTableAllIndexesOnBooks",
            "Effect": "Allow",
            "Action": [
              "dynamodb:PutItem",
              "dynamodb:UpdateItem",
              "dynamodb:DeleteItem",
              "dynamodb:BatchWriteItem",
              "dynamodb:GetItem",
              "dynamodb:BatchGetItem",
              "dynamodb:Scan",
              "dynamodb:Query",
              "dynamodb:ConditionCheckItem"
            ],
            "Resource": [
                "arn:aws:dynamodb:us-west-2:123456789012:table/Books",
                "arn:aws:dynamodb:us-west-2:123456789012:table/Books/index/*"
            ]
        }
    ]
}
```

IAM policy to read, write, update, and delete access on a DynamoDB table

Use this policy if you need to allow your application to create, read, update, and delete data in Amazon DynamoDB tables, indexes, and streams. Substitute the AWS Region name, your account ID, and the table name or wildcard character (*) where appropriate.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "DynamoDBIndexAndStreamAccess",
            "Effect": "Allow",
            "Action": [
                "dynamodb:GetShardIterator",
                "dynamodb:Scan",
                "dynamodb:Query",
                "dynamodb:DescribeStream",
                "dynamodb:GetRecords",
                "dynamodb:ListStreams"
            ],
            "Resource": [
                "arn:aws:dynamodb:us-west-2:123456789012:table/Books/index/*",
                "arn:aws:dynamodb:us-west-2:123456789012:table/Books/stream/*"
            ]
        },
        {
            "Sid": "DynamoDBTableAccess",
            "Effect": "Allow",
            "Action": [
                "dynamodb:BatchGetItem",
                "dynamodb:BatchWriteItem",
                "dynamodb:ConditionCheckItem",
                "dynamodb:PutItem",
                "dynamodb:DescribeTable",
                "dynamodb:DeleteItem",
                "dynamodb:GetItem",
                "dynamodb:Scan",
                "dynamodb:Query",
                "dynamodb:UpdateItem"
            ],
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:table/Books"
        },
        {
            "Sid": "DynamoDBDescribeLimitsAccess",
            "Effect": "Allow",
            "Action": "dynamodb:DescribeLimits",
            "Resource": [
```

```
                "arn:aws:dynamodb:us-west-2:123456789012:table/Books",
                "arn:aws:dynamodb:us-west-2:123456789012:table/Books/index/*"
            ]
        }
    ]
}
```

To expand this policy to cover all DynamoDB tables in all AWS Regions for this account, use a wildcard (*) for the Region and table name. For example:

```
"Resource":[
                "arn:aws:dynamodb:*:123456789012:table/*",
                "arn:aws:dynamodb:*:123456789012:table/*/index/*"
                ]
```

IAM policy to separate DynamoDB environments in the same AWS account

Suppose that you have separate environments where each environment maintains its own version of a table named `ProductCatalog`. If you create two `ProductCatalog` tables in the same AWS account, work in one environment might affect the other environment because of the way that permissions are set up. For example, quotas on the number of concurrent control plane operations (such as `CreateTable`) are set at the AWS account level.

As a result, each action in one environment reduces the number of operations available in the other environment. There is also a risk that the code in one environment might accidentally access tables in the other environment.

> **Note**
>
> If you want to separate production and test workloads to help control an event's potential "blast radius," the best practice is to create separate AWS accounts for test and production workloads. For more information, see AWS Account Management and Separation.

Suppose further that you have two developers, Amit and Alice, who are testing the `ProductCatalog` table. Instead of each developer requiring a separate AWS account, your developers can share the same test AWS account. In this test account, you can create a copy of the same table for each developer to work on, such as `Alice_ProductCatalog` and

`Amit_ProductCatalog`. In this case, you can create users Alice and Amit in the AWS account that you created for the test environment. You then can grant permissions to these users to perform DynamoDB actions on the tables that they own.

To grant these IAM user permissions, you can do either of the following:

- Create a separate policy for each user, and then attach each policy to its user separately. For example, you can attach the following policy to user Alice to allow her access to DynamoDB actions on the `Alice_ProductCatalog` table:

  JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "AllAPIActionsOnAliceTable",
            "Effect": "Allow",
            "Action": [
              "dynamodb:DeleteItem",
              "dynamodb:DescribeContributorInsights",
              "dynamodb:RestoreTableToPointInTime",
              "dynamodb:ListTagsOfResource",
              "dynamodb:CreateTableReplica",
              "dynamodb:UpdateContributorInsights",
              "dynamodb:CreateBackup",
              "dynamodb:DeleteTable",
              "dynamodb:UpdateTableReplicaAutoScaling",
              "dynamodb:UpdateContinuousBackups",
              "dynamodb:TagResource",
              "dynamodb:DescribeTable",
              "dynamodb:GetItem",
              "dynamodb:DescribeContinuousBackups",
              "dynamodb:BatchGetItem",
              "dynamodb:UpdateTimeToLive",
              "dynamodb:BatchWriteItem",
              "dynamodb:ConditionCheckItem",
              "dynamodb:UntagResource",
              "dynamodb:PutItem",
              "dynamodb:Scan",
              "dynamodb:Query",
              "dynamodb:UpdateItem",
              "dynamodb:DeleteTableReplica",
```

```
                "dynamodb:DescribeTimeToLive",
                "dynamodb:RestoreTableFromBackup",
                "dynamodb:UpdateTable",
                "dynamodb:DescribeTableReplicaAutoScaling",
                "dynamodb:GetShardIterator",
                "dynamodb:DescribeStream",
                "dynamodb:GetRecords",
                "dynamodb:DescribeLimits",
                "dynamodb:ListStreams"
            ],
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:table/
Alice_ProductCatalog/*"
        }
    ]
}
```

Then, you can create a similar policy with a different resource (the `Amit_ProductCatalog` table) for user Amit.

- Instead of attaching policies to individual users, you can use IAM policy variables to write a single policy and attach it to a group. You need to create a group and, for this example, add both users Alice and Amit to the group. The following example grants permissions to perform all DynamoDB actions on the `${aws:username}_ProductCatalog` table. The policy variable `${aws:username}` is replaced by the requester's user name when the policy is evaluated. For example, if Alice sends a request to add an item, the action is allowed only if Alice is adding items to the `Alice_ProductCatalog` table.

  JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "ActionsOnUserSpecificTable",
            "Effect": "Allow",
            "Action": [
                "dynamodb:PutItem",
                "dynamodb:UpdateItem",
                "dynamodb:DeleteItem",
                "dynamodb:BatchWriteItem",
                "dynamodb:GetItem",
                "dynamodb:BatchGetItem",
                "dynamodb:Scan",
```

```
                "dynamodb:Query",
                "dynamodb:ConditionCheckItem"
            ],
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:table/
${aws:username}_ProductCatalog"
        },
        {
            "Sid": "AdditionalPrivileges",
            "Effect": "Allow",
            "Action": [
                "dynamodb:ListTables",
                "dynamodb:DescribeTable",
                "dynamodb:DescribeContributorInsights"
            ],
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:table/*"
        }
    ]
}
```

> **Note**
>
> When using IAM policy variables, you must explicitly specify the `2012-10-17` version of the IAM policy language in the policy. The default version of the IAM policy language (`2008-10-17`) does not support policy variables.

Instead of identifying a specific table as a resource as you normally would, you could use a wildcard character (*) to grant permissions on all tables where the table name is prefixed with the user that is making the request, as shown in the following example.

```
"Resource":"arn:aws:dynamodb:us-west-2:123456789012:table/${aws:username}_*"
```

IAM policy to prevent the purchase of DynamoDB reserved capacity

With Amazon DynamoDB reserved capacity, you pay a one-time, upfront fee and commit to paying for a minimum usage level at significant savings over a period of time. You can use the AWS Management Console to view and purchase reserved capacity. However, you might not want all of the users in your organization to be able to purchase reserved capacity. For more information about reserved capacity, see Amazon DynamoDB pricing.

DynamoDB provides the following API operations for controlling access to reserved capacity management:

- `dynamodb:DescribeReservedCapacity` – Returns the reserved capacity purchases that are currently in effect.
- `dynamodb:DescribeReservedCapacityOfferings` – Returns details about the reserved capacity plans that are currently offered by AWS.
- `dynamodb:PurchaseReservedCapacityOfferings` – Performs an actual purchase of reserved capacity.

The AWS Management Console uses these API actions to display reserved capacity information and make purchases. You cannot call these operations from an application program because they can be accessed only from the console. However, you can allow or deny access to these operations in an IAM permissions policy.

The following policy allows users to view reserved capacity purchases and offerings by using the AWS Management Console — but new purchases are denied.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "AllowReservedCapacityDescriptions",
            "Effect": "Allow",
            "Action": [
                "dynamodb:DescribeReservedCapacity",
                "dynamodb:DescribeReservedCapacityOfferings"
            ],
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:*"
        },
        {
            "Sid": "DenyReservedCapacityPurchases",
            "Effect": "Deny",
            "Action": "dynamodb:PurchaseReservedCapacityOfferings",
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:*"
        }
    ]
```

```
}
```

Note that this policy uses the wildcard character (*) to allow describe permissions for all, and to deny the purchase of DynamoDB reserved capacity for all.

IAM policy to grant read access for a DynamoDB stream only (not for the table)

When you enable DynamoDB Streams on a table, information is captured about every modification to items in the table. For more information, see Change data capture for DynamoDB Streams.

In some cases, you might want to prevent an application from reading data from a DynamoDB table, but still allow access to that table's streams. For example, you can configure AWS Lambda to poll a stream and invoke a Lambda function when item updates are detected, and then perform additional processing.

The following actions are available for controlling access to DynamoDB streams:

- `dynamodb:DescribeStream`
- `dynamodb:GetRecords`
- `dynamodb:GetShardIterator`
- `dynamodb:ListStreams`

The following example policy grants users permissions to access the streams of a table named `GameScores`. The wildcard character (*) in the ARN matches any stream associated with that table.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "AccessGameScoresStreamOnly",
            "Effect": "Allow",
            "Action": [
                "dynamodb:DescribeStream",
                "dynamodb:GetRecords",
                "dynamodb:GetShardIterator",
                "dynamodb:ListStreams"
```

```
            ],
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:table/
GameScores/stream/*"
        }
    ]
}
```

Note that this policy grants access to the `GameScores` table's streams, but not to the table itself.

IAM policy to allow an AWS Lambda function to access DynamoDB stream records

If you want certain actions to be performed based on events in a DynamoDB stream, you can write an AWS Lambda function that is triggered by these events. A Lambda function such as this needs permissions to read data from a DynamoDB stream. For more information about using Lambda with DynamoDB Streams, see DynamoDB Streams and AWS Lambda triggers.

To grant permissions to Lambda, use the permissions policy that is associated with the Lambda function's IAM role (also known as an execution role). Specify this policy when you create the Lambda function.

For example, you can associate the following permissions policy with an execution role to grant Lambda permissions to perform the DynamoDB Streams actions listed.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "APIAccessForDynamoDBStreams",
            "Effect": "Allow",
            "Action": [
                "dynamodb:GetRecords",
                "dynamodb:GetShardIterator",
                "dynamodb:DescribeStream",
                "dynamodb:ListStreams"
            ],
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:table/
GameScores/stream/*"
        }
    ]
```

```
}
```

For more information, see AWS Lambda permissions in the *AWS Lambda Developer Guide*.

IAM policy for read and write access to a DynamoDB Accelerator (DAX) cluster

The following policy allows read, write, update, and delete access to a DynamoDB Accelerator (DAX) cluster, but not to the associated DynamoDB table. To use this policy, substitute the AWS Region name, your account ID, and the name of your DAX cluster.

> **Note**
>
> This policy gives access to DAX cluster, but not to the associated DynamoDB table. Make sure that your DAX cluster has the correct policy to perform these same operations on the DynamoDB table on your behalf.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "AmazonDynamoDBDAXDataOperations",
            "Effect": "Allow",
            "Action": [
                "dax:GetItem",
                "dax:PutItem",
                "dax:ConditionCheckItem",
                "dax:BatchGetItem",
                "dax:BatchWriteItem",
                "dax:DeleteItem",
                "dax:Query",
                "dax:UpdateItem",
                "dax:Scan"
            ],
            "Resource": "arn:aws:dax:eu-west-1:123456789012:cache/MyDAXCluster"
        }
    ]
}
```

To expand this policy to cover DAX access for all AWS Regions for an account, use a wildcard character (*) for the Region name.

```
"Resource": "arn:aws:dax:*:123456789012:cache/MyDAXCluster"
```

Troubleshooting Amazon DynamoDB identity and access

Use the following information to help you diagnose and fix common issues that you might encounter when working with DynamoDB and IAM.

Topics

- I am not authorized to perform an action in DynamoDB
- I am not authorized to perform iam:PassRole
- I want to allow people outside of my AWS account to access my DynamoDB resources

I am not authorized to perform an action in DynamoDB

If the AWS Management Console tells you that you're not authorized to perform an action, then you must contact your administrator for assistance. Your administrator is the person that provided you with your user name and password.

The following example error occurs when the `mateojackson` user tries to use the console to view details about a fictional *`my-example-widget`* resource but does not have the fictional `aws:`*`GetWidget`* permissions.

```
User: arn:aws:iam::123456789012:user/mateojackson is not authorized to perform:
 aws:GetWidget on resource: my-example-widget
```

In this case, Mateo asks his administrator to update his policies to allow him to access the *`my-example-widget`* resource using the `aws:`*`GetWidget`* action.

I am not authorized to perform iam:PassRole

If you receive an error that you're not authorized to perform the `iam:PassRole` action, your policies must be updated to allow you to pass a role to DynamoDB.

Some AWS services allow you to pass an existing role to that service instead of creating a new service role or service-linked role. To do this, you must have permissions to pass the role to the service.

The following example error occurs when an IAM user named `marymajor` tries to use the console to perform an action in DynamoDB. However, the action requires the service to have permissions that are granted by a service role. Mary does not have permissions to pass the role to the service.

```
User: arn:aws:iam::123456789012:user/marymajor is not authorized to perform:
 iam:PassRole
```

In this case, Mary's policies must be updated to allow her to perform the `iam:PassRole` action.

If you need help, contact your AWS administrator. Your administrator is the person who provided you with your sign-in credentials.

I want to allow people outside of my AWS account to access my DynamoDB resources

You can create a role that users in other accounts or people outside of your organization can use to access your resources. You can specify who is trusted to assume the role. For services that support resource-based policies or access control lists (ACLs), you can use those policies to grant people access to your resources.

To learn more, consult the following:

- To learn whether DynamoDB supports these features, see How Amazon DynamoDB works with IAM.
- To learn how to provide access to your resources across AWS accounts that you own, see Providing access to an IAM user in another AWS account that you own in the *IAM User Guide*.
- To learn how to provide access to your resources to third-party AWS accounts, see Providing access to AWS accounts owned by third parties in the *IAM User Guide*.
- To learn how to provide access through identity federation, see Providing access to externally authenticated users (identity federation) in the *IAM User Guide*.
- To learn the difference between using roles and resource-based policies for cross-account access, see Cross account resource access in IAM in the *IAM User Guide*.

IAM policy to prevent the purchase of DynamoDB reserved capacity

With Amazon DynamoDB reserved capacity, you pay a one-time, upfront fee and commit to paying for a minimum usage level at significant savings over a period of time. You can use the AWS Management Console to view and purchase reserved capacity. However, you might not want all of the users in your organization to be able to purchase reserved capacity. For more information about reserved capacity, see Amazon DynamoDB pricing.

DynamoDB provides the following API operations for controlling access to reserved capacity management:

- `dynamodb:DescribeReservedCapacity` – Returns the reserved capacity purchases that are currently in effect.
- `dynamodb:DescribeReservedCapacityOfferings` – Returns details about the reserved capacity plans that are currently offered by AWS.
- `dynamodb:PurchaseReservedCapacityOfferings` – Performs an actual purchase of reserved capacity.

The AWS Management Console uses these API actions to display reserved capacity information and make purchases. You cannot call these operations from an application program because they can be accessed only from the console. However, you can allow or deny access to these operations in an IAM permissions policy.

The following policy allows users to view reserved capacity purchases and offerings by using the AWS Management Console — but new purchases are denied.

JSON

```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Sid": "AllowReservedCapacityDescriptions",
            "Effect": "Allow",
            "Action": [
                "dynamodb:DescribeReservedCapacity",
                "dynamodb:DescribeReservedCapacityOfferings"
            ],
```

```
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:*"
        },
        {
            "Sid": "DenyReservedCapacityPurchases",
            "Effect": "Deny",
            "Action": "dynamodb:PurchaseReservedCapacityOfferings",
            "Resource": "arn:aws:dynamodb:us-west-2:123456789012:*"
        }
    ]
}
```

Note that this policy uses the wildcard character (*) to allow describe permissions for all, and to deny the purchase of DynamoDB reserved capacity for all.

Using IAM policy conditions for fine-grained access control

When you grant permissions in DynamoDB, you can specify conditions that determine how a permissions policy takes effect.

Overview

In DynamoDB, you have the option to specify conditions when granting permissions using an IAM policy (see Identity and Access Management for Amazon DynamoDB). For example, you can:

- Grant permissions to allow users read-only access to certain items and attributes in a table or a secondary index.
- Grant permissions to allow users write-only access to certain attributes in a table, based upon the identity of that user.

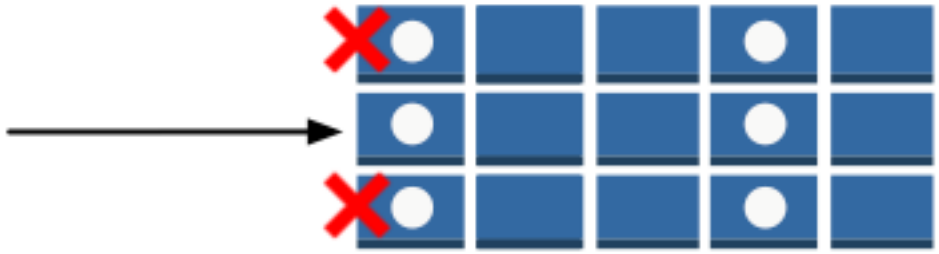
In DynamoDB, you can specify conditions in an IAM policy using condition keys, as illustrated in the use case in the following section.

Permissions use case

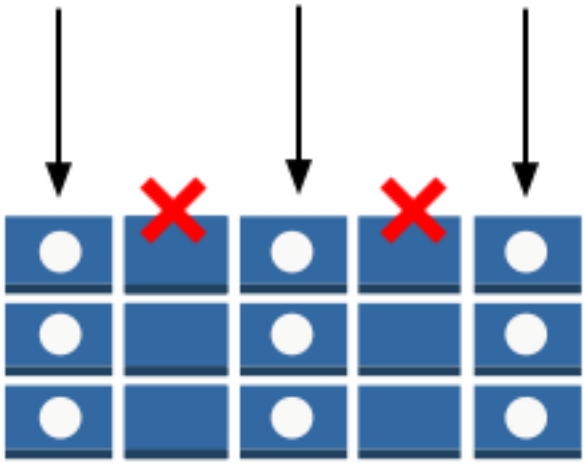
In addition to controlling access to DynamoDB API actions, you can also control access to individual data items and attributes. For example, you can do the following:

- Grant permissions on a table, but restrict access to specific items in that table based on certain primary key values. An example might be a social networking app for games, where all users'

saved game data is stored in a single table, but no users can access data items that they do not own, as shown in the following illustration:



- Hide information so that only a subset of attributes is visible to the user. An example might be an app that displays flight data for nearby airports, based on the user's location. Airline names, arrival and departure times, and flight numbers are all displayed. However, attributes such as pilot names or the number of passengers are hidden, as shown in the following illustration:



To implement this kind of fine-grained access control, you write an IAM permissions policy that specifies conditions for accessing security credentials and the associated permissions. You then apply the policy to users, groups, or roles that you create using the IAM console. Your IAM policy can restrict access to individual items in a table, access to the attributes in those items, or both at the same time.

You can optionally use web identity federation to control access by users who are authenticated by Login with Amazon, Facebook, or Google. For more information, see Using web identity federation.

You use the IAM `Condition` element to implement a fine-grained access control policy. By adding a `Condition` element to a permissions policy, you can allow or deny access to items and attributes in DynamoDB tables and indexes, based upon your particular business requirements.

The video below explains fine-grained access control in DynamoDB using IAM policy conditions.

Understanding Fine-Grained Access Control in DynamoDB

Fine-grained access control in DynamoDB allows you to create precise permissions boundaries at multiple levels:

1. **Item-level access control:** Restrict users to only access items that contain specific key values, typically matching their identity or permission scope.
2. **Attribute-level access control:** Limit which attributes (columns) users can view or modify, enabling you to protect sensitive information while allowing access to non-sensitive data within the same items.
3. **Operation-specific controls:** Apply different permission rules based on the type of operation being performed.

These controls are implemented through IAM policies using DynamoDB-specific condition keys.

Specifying conditions: Using condition keys

AWS provides a set of predefined condition keys (AWS-wide condition keys) for all AWS services that support IAM for access control. For example, you can use the `aws:SourceIp` condition key to check the requester's IP address before allowing an action to be performed. For more information and a list of the AWS-wide keys, see Available Keys for Conditions in the IAM User Guide.

The following are the DynamoDB service-specific condition keys that apply to DynamoDB.

`dynamodb:LeadingKeys`

Represents the first key attribute of a table—in other words, the partition key. The key name `LeadingKeys` is plural, even if the key is used with single-item actions. In addition, you must use the `ForAllValues` modifier when using `LeadingKeys` in a condition.

`dynamodb:Select`

Represents the `Select` parameter of a request. `Select` can be any of the following values:

- `ALL_ATTRIBUTES`

- `ALL_PROJECTED_ATTRIBUTES`
- `SPECIFIC_ATTRIBUTES`
- `COUNT`

> **Note**
>
> While often associated with Query and Scan operations, this condition key applies to all DynamoDB operations that return item attributes and is essential for controlling attribute access across all API actions. Using StringEqualsIfExists or similar constraints on this condition key will apply constraints on operations where this condition key applies, while ignoring it on operations where it does not apply.

`dynamodb:Attributes`

Represents a list of the *top-level* attributes accessed by a request. A top-level attribute is accessed by a request if it, or any nested attribute that it contains, is specified in the request parameters. For example, a `GetItem` request that specifies a `ProjectionExpression` of `"Name, Address.City"`, the `dynamodb:Attributes` list would include "Name" and "Address". If the `Attributes` parameter is enumerated in a fine grained access control policy then consider also restricting `ReturnValues` and `Select` parameters to ensure restricted access to specified attributes across multiple API actions like `GetItem`, `Query`, and `Scan`.

> **Note**
>
> This condition is evaluated only on the attributes specified in the request (such as in a ProjectionExpression), not on attributes in the response. If no ProjectionExpression is provided in the request, all attributes will be returned regardless of any attribute restrictions in the policy. See the section "Ensuring attribute-based restrictions are enforced" below for details on how to properly secure attribute access.

`dynamodb:ReturnValues`

Represents the `ReturnValues` parameter of a request. Depending on the API action, `ReturnValues` could be any of the following values:

- `ALL_OLD`

- `UPDATED_OLD`
- `ALL_NEW`
- `UPDATED_NEW`
- `NONE`

`dynamodb:ReturnConsumedCapacity`

Represents the `ReturnConsumedCapacity` parameter of a request. `ReturnConsumedCapacity` can be one of the following values:

- `TOTAL`
- `NONE`

`dynamodb:FirstPartitionKeyValues`

Represents the first key attribute of a table—in other words, the first partition key. The key name `FirstPartitionKeyValues` is plural, even if the key is used with single-item actions. In addition, you must use the `ForAllValues` modifier when using `FirstPartitionKeyValues` in a condition. `FirstPartitionKeyValues` and `LeadingKeys` can used exchangeable.

`dynamodb:SecondPartitionKeyValues`

Similar to `dynamodb:FirstPartitionKeyValues`. Represents resources' second partition key. The key name `SecondPartitionKeyValues` is plural, even if the key is used with single-item actions.

`dynamodb:ThirdPartitionKeyValues`

Similar to `dynamodb:FirstPartitionKeyValues`. Represents resources' third partition key. The key name `ThirdPartitionKeyValues` is plural, even if the key is used with single-item actions.

`dynamodb:FourthPartitionKeyValues`

Similar to `dynamodb:FirstPartitionKeyValues`. Represents resources' fourth partition key. The key name `FourthPartitionKeyValues` is plural, even if the key is used with single-item actions.

Ensuring attribute-based restrictions are enforced

When using attribute-based conditions to restrict access to specific attributes, it's important to understand how these conditions are evaluated:

- **Attribute conditions are evaluated only on attributes specified in the request**, not on attributes in the response.
- **For read operations without a ProjectionExpression** (GetItem, Query, Scan, etc.), all attributes will be returned regardless of attribute restrictions in your policy. To prevent this potential exposure of sensitive data, implement both attribute conditions (`dynamodb:Attributes`) and a condition requiring specific attributes must be requested (`dynamodb:Select`).
- **For write operations** (PutItem, UpdateItem, DeleteItem), the ReturnValues parameter can return complete items, potentially exposing restricted attributes even when the write operation itself complies with your policy. To prevent this exposure, implement both attribute conditions (`dynamodb:Attributes`) and restrictions on ReturnValues (`dynamodb:ReturnValues`) in your policy.

Limiting user access

Many IAM permissions policies allow users to access only those items in a table where the partition key value matches the user identifier. For example, the game app preceding limits access in this way so that users can only access game data that is associated with their user ID. The IAM substitution variables `${www.amazon.com:user_id}`, `${graph.facebook.com:id}`, and `${accounts.google.com:sub}` contain user identifiers for Login with Amazon, Facebook, and Google. To learn how an application logs in to one of these identity providers and obtains these identifiers, see Using web identity federation.

> **Important**
>
> Fine-grained access control isn't supported for restricting global tables replication. Applying policy conditions for fine-grained access control to DynamoDB service principals or service-linked roles used for global tables replication may interrupt replication within a global table.

> **Note**
>
> Each of the examples in the following section sets the `Effect` clause to `Allow` and specifies only the actions, resources, and parameters that are allowed. Access is permitted only to what is explicitly listed in the IAM policy.
> In some cases, it is possible to rewrite these policies so that they are deny-based (that is, setting the `Effect` clause to `Deny` and inverting all of the logic in the policy). However,

we recommend that you avoid using deny-based policies with DynamoDB because they're difficult to write correctly, compared to allow-based policies. In addition, future changes to the DynamoDB API (or changes to existing API inputs) can render a deny-based policy ineffective.

Example policies: Using conditions for fine-grained access control

This section shows several policies for implementing fine-grained access control on DynamoDB tables and indexes.

> **Note**
>
> All examples use the us-west-2 Region and contain fictitious account IDs.

Example 1. Basic partition key-based access control with attribute restrictions

As an example, consider a mobile gaming app that lets players select from and play a variety of different games. The app uses a DynamoDB table named `GameScores` to keep track of high scores and other user data. Each item in the table is uniquely identified by a user ID and the name of the game that the user played. The `GameScores` table has a primary key consisting of a partition key (`UserId`) and sort key (`GameTitle`). Users only have access to game data associated with their user ID. A user who wants to play a game must belong to an IAM role named `GameRole`, which has a security policy attached to it.

To manage user permissions in this app, you could write a permissions policy such as the following:

JSON

```
{
   "Version":"2012-10-17",
   "Statement":[
      {
         "Sid":"AllowAccessToOnlyItemsMatchingUserID",
         "Effect":"Allow",
         "Action":[
            "dynamodb:GetItem",
            "dynamodb:BatchGetItem",
```

```
            "dynamodb:Query",
            "dynamodb:PutItem",
            "dynamodb:UpdateItem",
            "dynamodb:DeleteItem",
            "dynamodb:BatchWriteItem"
         ],
         "Resource":[
            "arn:aws:dynamodb:us-west-2:123456789012:table/GameScores"
         ],
         "Condition":{
            "ForAllValues:StringEquals":{
               "dynamodb:LeadingKeys":[
                  "${www.amazon.com:user_id}"
               ],
               "dynamodb:Attributes":[
                  "UserId",
                  "GameTitle",
                  "Wins",
                  "Losses",
                  "TopScore",
                  "TopScoreDateTime"
               ]
            },
            "StringEqualsIfExists":{
               "dynamodb:Select":"SPECIFIC_ATTRIBUTES"
            }
         }
      }
   ]
}
```

In addition to granting permissions for specific DynamoDB actions (`Action` element) on the `GameScores` table (`Resource` element), the `Condition` element uses the following condition keys specific to DynamoDB that limit the permissions as follows:

- `dynamodb:LeadingKeys` – This condition key allows users to access only the items where the partition key value matches their user ID. This ID, `${www.amazon.com:user_id}`, is a substitution variable. For more information about substitution variables, see Using web identity federation.
- `dynamodb:Attributes` – This condition key limits access to the specified attributes so that only the actions listed in the permissions policy can return values for these attributes. In

addition, the `StringEqualsIfExists` clause ensures that the app must always provide a list of specific attributes to act upon and that the app can't request all attributes.

When an IAM policy is evaluated, the result is always either true (access is allowed) or false (access is denied). If any part of the `Condition` element is false, the entire policy evaluates to false and access is denied.

> **Important**
>
> If you use `dynamodb:Attributes`, you must specify the names of all of the primary key and index key attributes for the table and any secondary indexes that are listed in the policy. Otherwise, DynamoDB can't use these key attributes to perform the requested action.

IAM policy documents can contain only the following Unicode characters: horizontal tab (U+0009), linefeed (U+000A), carriage return (U+000D), and characters in the range U+0020 to U+00FF.

Example 2: Grant permissions that limit access to items with a specific partition key value

The following permissions policy grants permissions that allow a set of DynamoDB actions on the `GamesScore` table. It uses the `dynamodb:LeadingKeys` condition key to limit user actions only on the items whose `UserID` partition key value matches the Login with Amazon unique user ID for this app.

> **Important**
>
> The list of actions does not include permissions for `Scan` because `Scan` returns all items regardless of the leading keys.

JSON

```
{
   "Version":"2012-10-17",
   "Statement":[
      {
         "Sid":"FullAccessToUserItems",
```

```
            "Effect":"Allow",
            "Action":[
               "dynamodb:GetItem",
               "dynamodb:BatchGetItem",
               "dynamodb:Query",
               "dynamodb:PutItem",
               "dynamodb:UpdateItem",
               "dynamodb:DeleteItem",
               "dynamodb:BatchWriteItem"
            ],
            "Resource":[
               "arn:aws:dynamodb:us-west-2:123456789012:table/GameScores"
            ],
            "Condition":{
               "ForAllValues:StringEquals":{
                  "dynamodb:LeadingKeys":[
                     "${www.amazon.com:user_id}"
                  ]
               }
            }
         }
      ]
   }
```

Note

When using policy variables, you must explicitly specify version 2012-10-17 in the policy. The default version of the access policy language, 2008-10-17, does not support policy variables.

To implement read-only access, you can remove any actions that can modify the data. In the following policy, only those actions that provide read-only access are included in the condition.

JSON

```
{
   "Version":"2012-10-17",
   "Statement":[
      {
```

```
            "Sid":"ReadOnlyAccessToUserItems",
            "Effect":"Allow",
            "Action":[
               "dynamodb:GetItem",
               "dynamodb:BatchGetItem",
               "dynamodb:Query"
            ],
            "Resource":[
               "arn:aws:dynamodb:us-west-2:123456789012:table/GameScores"
            ],
            "Condition":{
               "ForAllValues:StringEquals":{
                  "dynamodb:LeadingKeys":[
                     "${www.amazon.com:user_id}"
                  ]
               }
            }
         }
      ]
   }
```

Important

If you use `dynamodb:Attributes`, you must specify the names of all of the primary key and index key attributes, for the table and any secondary indexes that are listed in the policy. Otherwise, DynamoDB can't use these key attributes to perform the requested action.

Example 3: Grant permissions that limit access to specific attributes in a table

The following permissions policy allows access to only two specific attributes in a table by adding the `dynamodb:Attributes` condition key. These attributes can be read, written, or evaluated in a conditional write or scan filter.

JSON

```
{
   "Version":"2012-10-17",
   "Statement":[
```

```
        {
            "Sid":"LimitAccessToSpecificAttributes",
            "Effect":"Allow",
            "Action":[
                "dynamodb:UpdateItem",
                "dynamodb:GetItem",
                "dynamodb:Query",
                "dynamodb:BatchGetItem",
                "dynamodb:Scan"
            ],
            "Resource":[
                "arn:aws:dynamodb:us-west-2:123456789012:table/GameScores"
            ],
            "Condition":{
                "ForAllValues:StringEquals":{
                    "dynamodb:Attributes":[
                        "UserId",
                        "TopScore"
                    ]
                },
                "StringEqualsIfExists":{
                    "dynamodb:Select":"SPECIFIC_ATTRIBUTES",
                    "dynamodb:ReturnValues":[
                        "NONE",
                        "UPDATED_OLD",
                        "UPDATED_NEW"
                    ]
                }
            }
        }
    ]
}
```

Note

The policy takes an allow list approach, which allows access to a named set of attributes. You can write an equivalent policy that denies access to other attributes instead. We don't recommend this deny list approach. Users can determine the names of these denied attributes by follow the principle of least privilege, as explained in Wikipedia at http://

en.wikipedia.org/wiki/Principle_of_least_privilege, and use an allow list approach to enumerate all of the allowed values, rather than specifying the denied attributes.

This policy doesn't permit `PutItem`, `DeleteItem`, or `BatchWriteItem`. These actions always replace the entire previous item, which would allow users to delete the previous values for attributes that they are not allowed to access.

The `StringEqualsIfExists` clause in the permissions policy ensures the following:

- If the user specifies the `Select` parameter, then its value must be `SPECIFIC_ATTRIBUTES`. This requirement prevents the API action from returning any attributes that aren't allowed, such as from an index projection.
- If the user specifies the `ReturnValues` parameter, then its value must be `NONE`, `UPDATED_OLD`, or `UPDATED_NEW`. This is required because the `UpdateItem` action also performs implicit read operations to check whether an item exists before replacing it, and so that previous attribute values can be returned if requested. Restricting `ReturnValues` in this way ensures that users can only read or write the allowed attributes.
- The `StringEqualsIfExists` clause assures that only one of these parameters — `Select` or `ReturnValues` — can be used per request, in the context of the allowed actions.

The following are some variations on this policy:

- To allow only read actions, you can remove `UpdateItem` from the list of allowed actions. Because none of the remaining actions accept `ReturnValues`, you can remove `ReturnValues` from the condition. You can also change `StringEqualsIfExists` to `StringEquals` because the `Select` parameter always has a value (`ALL_ATTRIBUTES`, unless otherwise specified).
- To allow only write actions, you can remove everything except `UpdateItem` from the list of allowed actions. Because `UpdateItem` does not use the `Select` parameter, you can remove `Select` from the condition. You must also change `StringEqualsIfExists` to `StringEquals` because the `ReturnValues` parameter always has a value (`NONE` unless otherwise specified).
- To allow all attributes whose name matches a pattern, use `StringLike` instead of `StringEquals`, and use a multi-character pattern match wildcard character (*).

Example 4: Grant permissions to prevent updates on certain attributes

The following permissions policy limits user access to updating only the specific attributes identified by the `dynamodb:Attributes` condition key. The `StringNotLike` condition prevents an application from updating the attributes specified using the `dynamodb:Attributes` condition key.

JSON

```
{
   "Version":"2012-10-17",
   "Statement":[
      {
         "Sid":"PreventUpdatesOnCertainAttributes",
         "Effect":"Allow",
         "Action":[
            "dynamodb:UpdateItem"
         ],
         "Resource":"arn:aws:dynamodb:us-west-2:123456789012:table/GameScores",
         "Condition":{
            "ForAllValues:StringNotLike":{
               "dynamodb:Attributes":[
                  "FreeGamesAvailable",
                  "BossLevelUnlocked"
               ]
            },
            "StringEqualsIfExists":{
               "dynamodb:Select":"SPECIFIC_ATTRIBUTES",
               "dynamodb:ReturnValues":[
                  "NONE",
                  "UPDATED_OLD",
                  "UPDATED_NEW"
               ]
            }
         }
      }
   ]
}
```

Note the following:

- The `UpdateItem` action, like other write actions, requires read access to the items so that it can return values before and after the update. In the policy, you limit the action to accessing only the attributes that are allowed to be updated by specifying the `dynamodb:ReturnValues` condition key. The condition key restricts `ReturnValues` in the request to specify only `NONE`, `UPDATED_OLD`, or `UPDATED_NEW` and doesn't include `ALL_OLD` or `ALL_NEW`.
- The `StringEqualsIfExists` operator ensures that if `dynamodb:Select` or `dynamodb:ReturnValues` is present in the request, it must match the specified values. This prevents operations from returning complete items.
- When restricting attribute updates, you should also control what data can be returned to prevent information disclosure of protected attributes.
- The `PutItem` and `DeleteItem` actions replace an entire item, and thus allows applications to modify any attributes. So when limiting an application to updating only specific attributes, you should not grant permission for these APIs.

Example 5: Grant permissions to query only projected attributes in an index

The following permissions policy allows queries on a secondary index (`TopScoreDateTimeIndex`) by using the `dynamodb:Attributes` condition key. The policy also limits queries to requesting only specific attributes that have been projected into the index.

To require the application to specify a list of attributes in the query, the policy also specifies the `dynamodb:Select` condition key to require that the `Select` parameter of the DynamoDB `Query` action is `SPECIFIC_ATTRIBUTES`. The list of attributes is limited to a specific list that is provided using the `dynamodb:Attributes` condition key.

JSON

```
{
   "Version":"2012-10-17",
   "Statement":[
      {
         "Sid":"QueryOnlyProjectedIndexAttributes",
         "Effect":"Allow",
         "Action":[
            "dynamodb:Query"
         ],
         "Resource":[
```

```
            "arn:aws:dynamodb:us-west-2:123456789012:table/GameScores/index/
TopScoreDateTimeIndex"
         ],
         "Condition":{
            "ForAllValues:StringEquals":{
               "dynamodb:Attributes":[
                  "TopScoreDateTime",
                  "GameTitle",
                  "Wins",
                  "Losses",
                  "Attempts"
               ]
            },
            "StringEquals":{
               "dynamodb:Select":"SPECIFIC_ATTRIBUTES"
            }
         }
      }
   ]
}
```

The following permissions policy is similar, but the query must request all of the attributes that have been projected into the index.

JSON

```
{
   "Version":"2012-10-17",
   "Statement":[
      {
         "Sid":"QueryAllIndexAttributes",
         "Effect":"Allow",
         "Action":[
            "dynamodb:Query"
         ],
         "Resource":[
            "arn:aws:dynamodb:us-west-2:123456789012:table/GameScores/index/
TopScoreDateTimeIndex"
         ],
         "Condition":{
            "StringEquals":{
```

```
                "dynamodb:Select":"ALL_PROJECTED_ATTRIBUTES"
            }
         }
      }
   ]
}
```

Example 6: Grant permissions to limit access to certain attributes and partition key values

The following permissions policy allows specific DynamoDB actions (specified in the `Action` element) on a table and a table index (specified in the `Resource` element). The policy uses the `dynamodb:LeadingKeys` condition key to restrict permissions to only the items whose partition key value matches the user's Facebook ID.

JSON

```
{
   "Version":"2012-10-17",
   "Statement":[
      {
         "Sid":"LimitAccessToCertainAttributesAndKeyValues",
         "Effect":"Allow",
         "Action":[
            "dynamodb:UpdateItem",
            "dynamodb:GetItem",
            "dynamodb:Query",
            "dynamodb:BatchGetItem"
         ],
         "Resource":[
            "arn:aws:dynamodb:us-west-2:123456789012:table/GameScores",
            "arn:aws:dynamodb:us-west-2:123456789012:table/GameScores/index/
TopScoreDateTimeIndex"
         ],
         "Condition":{
            "ForAllValues:StringEquals":{
               "dynamodb:LeadingKeys":[
                  "${graph.facebook.com:id}"
               ],
               "dynamodb:Attributes":[
                  "attribute-A",
                  "attribute-B"
```

```
                ]
            },
            "StringEqualsIfExists":{
                "dynamodb:Select":"SPECIFIC_ATTRIBUTES",
                "dynamodb:ReturnValues":[
                    "NONE",
                    "UPDATED_OLD",
                    "UPDATED_NEW"
                ]
            }
        }
    }
]
}
```

Note the following:

- Write actions allowed by the policy (`UpdateItem`) can only modify attribute-A or attribute-B.
- Because the policy allows `UpdateItem`, an application can insert new items, and the hidden attributes will be null in the new items. If these attributes are projected into `TopScoreDateTimeIndex`, the policy has the added benefit of preventing queries that cause fetches from the table.
- Applications cannot read any attributes other than those listed in `dynamodb:Attributes`. With this policy in place, an application must set the `Select` parameter to `SPECIFIC_ATTRIBUTES` in read requests, and only attributes in the allow list can be requested. For write requests, the application cannot set `ReturnValues` to `ALL_OLD` or `ALL_NEW` and it cannot perform conditional write operations based on any other attributes.

Example 7: Deny permissions to limit access to specific attributes in a table

The following policy denies access to sensitive attributes and ensures this restriction cannot be bypassed by omitting a projection expression. It allows general access to the `CustomerData` table while explicitly denying access to `SSN` and `CreditCardNumber` attributes.

JSON

```
{
    "Version":"2012-10-17",
```

```
    "Statement":[
       {
          "Effect":"Allow",
          "Action":[
             "dynamodb:GetItem",
             "dynamodb:Query",
             "dynamodb:Scan"
          ],
          "Resource":"arn:aws:dynamodb:us-west-2:123456789012:table/CustomerData"
       },
       {
          "Effect":"Deny",
          "Action":[
             "dynamodb:GetItem",
             "dynamodb:Query",
             "dynamodb:Scan"
          ],
          "Resource":"arn:aws:dynamodb:us-west-2:123456789012:table/CustomerData",
          "Condition":{
             "ForAnyValue:StringEquals":{
                "dynamodb:Attributes":[
                   "SSN",
                   "CreditCardNumber"
                ]
             }
          }
       },
       {
          "Effect":"Deny",
          "Action":[
             "dynamodb:GetItem",
             "dynamodb:Query",
             "dynamodb:Scan"
          ],
          "Resource":"arn:aws:dynamodb:us-west-2:123456789012:table/CustomerData",
          "Condition":{
             "StringNotEqualsIfExists":{
                "dynamodb:Select":"SPECIFIC_ATTRIBUTES"
             }
          }
       }
    ]
}
```

Related topics

- Identity and Access Management for Amazon DynamoDB
- DynamoDB API permissions: Actions, resources, and conditions reference

Using web identity federation

If you are writing an application targeted at large numbers of users, you can optionally use *web identity federation* for authentication and authorization. Web identity federation removes the need for creating individual users. Instead, users can sign in to an identity provider and then obtain temporary security credentials from AWS Security Token Service (AWS STS). The app can then use these credentials to access AWS services.

Web identity federation supports the following identity providers:

- Login with Amazon
- Facebook
- Google

Additional resources for web identity federation

The following resources can help you learn more about web identity federation:

- The post Web Identity Federation using the AWS SDK for .NET on the AWS Developer blog walks through how to use web identity federation with Facebook. It includes code snippets in C# that show how to assume an IAM role with web identity and how to use temporary security credentials to access an AWS resource.
- The AWS Mobile SDK for iOS and the AWS Mobile SDK for Android contain sample apps. They include code that shows how to invoke the identity providers, and then how to use the information from these providers to get and use temporary security credentials.
- The article Web Identity Federation with Mobile Applications discusses web identity federation and shows an example of how to use web identity federation to access an AWS resource.

Example policy for web identity federation

To show how you can use web identity federation with DynamoDB, revisit the *GameScores* table that was introduced in Using IAM policy conditions for fine-grained access control. Here is the primary key for *GameScores*.

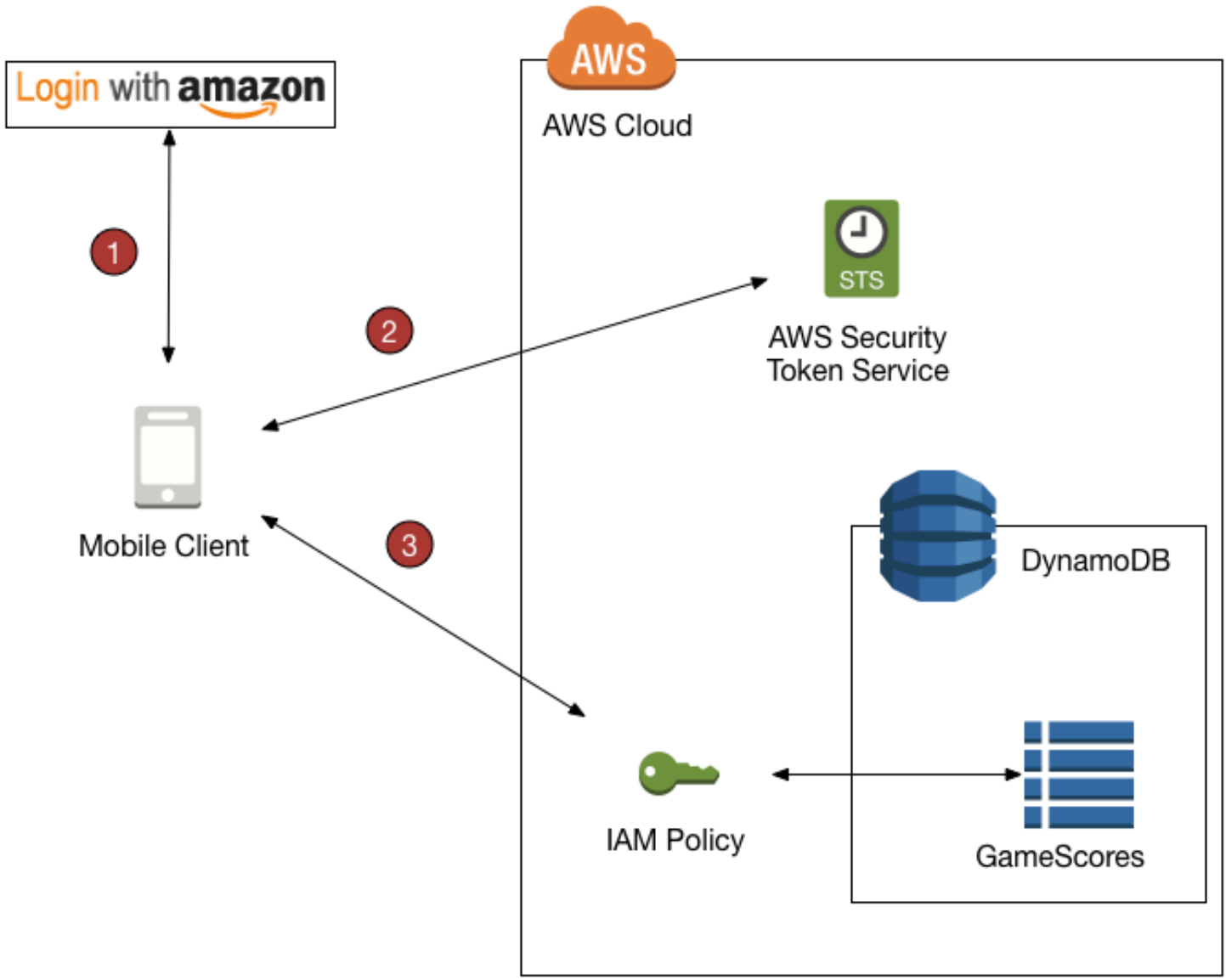
Table Name	Primary Key Type	Partition Key Name and Type	Sort Key Name and Type
GameScores (UserId, GameTitle, ...)	Composite	Attribute Name: UserId Type: String	Attribute Name: GameTitle Type: String

Now suppose that a mobile gaming app uses this table, and that app needs to support thousands, or even millions, of users. At this scale, it becomes very difficult to manage individual app users, and to guarantee that each user can only access their own data in the *GameScores* table. Fortunately, many users already have accounts with a third-party identity provider, such as Facebook, Google, or Login with Amazon. So it makes sense to use one of these providers for authentication tasks.

To do this using web identity federation, the app developer must register the app with an identity provider (such as Login with Amazon) and obtain a unique app ID. Next, the developer needs to create an IAM role. (For this example, this role is named *GameRole*.) The role must have an IAM policy document attached to it, specifying the conditions under which the app can access *GameScores* table.

When a user wants to play a game, they sign in to their Login with Amazon account from within the gaming app. The app then calls AWS Security Token Service (AWS STS), providing the Login with Amazon app ID and requesting membership in *GameRole*. AWS STS returns temporary AWS credentials to the app and allows it to access the *GameScores* table, subject to the *GameRole* policy document.

The following diagram shows how these pieces fit together.



Web identity federation overview

1. The app calls a third-party identity provider to authenticate the user and the app. The identity provider returns a web identity token to the app.
2. The app calls AWS STS and passes the web identity token as input. AWS STS authorizes the app and gives it temporary AWS access credentials. The app is allowed to assume an IAM role (*GameRole*) and access AWS resources in accordance with the role's security policy.
3. The app calls DynamoDB to access the *GameScores* table. Because it has assumed the *GameRole*, the app is subject to the security policy associated with that role. The policy document prevents the app from accessing data that does not belong to the user.

Once again, here is the security policy for *GameRole* that was shown in Using IAM policy conditions for fine-grained access control:

JSON

```
{
   "Version":"2012-10-17",
   "Statement":[
      {
         "Sid":"AllowAccessToOnlyItemsMatchingUserID",
         "Effect":"Allow",
         "Action":[
            "dynamodb:GetItem",
            "dynamodb:BatchGetItem",
            "dynamodb:Query",
            "dynamodb:PutItem",
            "dynamodb:UpdateItem",
            "dynamodb:DeleteItem",
            "dynamodb:BatchWriteItem"
         ],
         "Resource":[
            "arn:aws:dynamodb:us-west-2:123456789012:table/GameScores"
         ],
         "Condition":{
            "ForAllValues:StringEquals":{
               "dynamodb:LeadingKeys":[
                  "${www.amazon.com:user_id}"
               ],
               "dynamodb:Attributes":[
                  "UserId",
                  "GameTitle",
                  "Wins",
                  "Losses",
                  "TopScore",
                  "TopScoreDateTime"
               ]
            },
            "StringEqualsIfExists":{
               "dynamodb:Select":"SPECIFIC_ATTRIBUTES"
            }
         }
      }
   ]
```

```
}
```

The `Condition` clause determines which items in *GameScores* are visible to the app. It does this by comparing the Login with Amazon ID to the `UserId` partition key values in `GameScores`. Only the items belonging to the current user can be processed using one of DynamoDB actions that are listed in this policy. Other items in the table cannot be accessed. Furthermore, only the specific attributes listed in the policy can be accessed.

Preparing to use web identity federation

If you are an application developer and want to use web identity federation for your app, follow these steps:

1. **Sign up as a developer with a third-party identity provider.** The following external links provide information about signing up with supported identity providers:
   - Login with Amazon Developer Center
   - Registration on the Facebook site
   - Using OAuth 2.0 to Access Google APIs on the Google site
2. **Register your app with the identity provider.** When you do this, the provider gives you an ID that's unique to your app. If you want your app to work with multiple identity providers, you need to obtain an app ID from each provider.
3. **Create one or more IAM roles.** You need one role for each identity provider for each app. For example, you might create a role that can be assumed by an app where the user signed in using Login with Amazon, a second role for the same app where the user has signed in using Facebook, and a third role for the app where users sign in using Google.

   As part of the role creation process, you need to attach an IAM policy to the role. Your policy document should define the DynamoDB resources required by your app, and the permissions for accessing those resources.

For more information, see About Web Identity Federation in *IAM User Guide*.

> **Note**
>
> As an alternative to AWS Security Token Service, you can use Amazon Cognito. Amazon Cognito is the preferred service for managing temporary credentials for mobile apps. For more information, see Getting credentials in the *Amazon Cognito Developer Guide.*

Generating an IAM policy using the DynamoDB console

The DynamoDB console can help you create an IAM policy for use with web identity federation. To do this, you choose a DynamoDB table and specify the identity provider, actions, and attributes to be included in the policy. The DynamoDB console then generates a policy that you can attach to an IAM role.

1. Sign in to the AWS Management Console and open the DynamoDB console at https://console.aws.amazon.com/dynamodb/.
2. In the navigation pane, choose **Tables**.
3. In the list of tables, choose the table for which you want to create the IAM policy.
4. Select the **Actions** button, and choose **Create Access Control Policy**.
5. Choose the identity provider, actions, and attributes for the policy.

   When the settings are as you want them, choose **Generate Policy**. The generated policy appears.
6. Choose **See Documentation**, and follow the steps required to attach the generated policy to an IAM role.

Writing your app to use web identity federation

To use web identity federation, your app must assume the IAM role that you created. From that point on, the app honors the access policy that you attached to the role.

At runtime, if your app uses web identity federation, it must follow these steps:

1. **Authenticate with a third-party identity provider.** Your app must call the identity provider using an interface that they provide. The exact way in which you authenticate the user depends on the provider and on what platform your app is running. Typically, if the user is not already signed in, the identity provider takes care of displaying a sign-in page for that provider.

After the identity provider authenticates the user, the provider returns a web identity token to your app. The format of this token depends on the provider, but is typically a very long string of characters.

2. **Obtain temporary AWS security credentials.** To do this, your app sends a `AssumeRoleWithWebIdentity` request to AWS Security Token Service (AWS STS). This request contains the following:

   - The web identity token from the previous step
   - The app ID from the identity provider
   - The Amazon Resource Name (ARN) of the IAM role that you created for this identity provider for this app

   AWS STS returns a set of AWS security credentials that expire after a certain amount of time (3,600 seconds, by default).

   The following is a sample request and response from a `AssumeRoleWithWebIdentity` action in AWS STS. The web identity token was obtained from the Login with Amazon identity provider.

```
GET / HTTP/1.1
Host: sts.amazonaws.com
Content-Type: application/json; charset=utf-8
URL: https://sts.amazonaws.com/?ProviderId=www.amazon.com
&DurationSeconds=900&Action=AssumeRoleWithWebIdentity
&Version=2011-06-15&RoleSessionName=web-identity-federation
&RoleArn=arn:aws:iam::123456789012:role/GameRole
&WebIdentityToken=Atza|IQEBLjAsAhQluyKqyBiYZ8-kclvGTYM81e...(remaining characters
 omitted)
```

```
<AssumeRoleWithWebIdentityResponse
  xmlns="https://sts.amazonaws.com/doc/2011-06-15/">
  <AssumeRoleWithWebIdentityResult>
    <SubjectFromWebIdentityToken>amzn1.account.AGJZDKHJKAUUSW6C44CHPEXAMPLE</
SubjectFromWebIdentityToken>
    <Credentials>
      <SessionToken>AQoDYXdzEMf//////////wEa8AP6nNDwcSLnf+cHupC...(remaining
 characters omitted)</SessionToken>
      <SecretAccessKey>8Jhi60+EWUUbbUShTEsjTxqQtM8UKvsM6XAjdA==</SecretAccessKey>
      <Expiration>2013-10-01T22:14:35Z</Expiration>
      <AccessKeyId>06198791C436IEXAMPLE</AccessKeyId>
```

```
    </Credentials>
    <AssumedRoleUser>
      <Arn>arn:aws:sts::123456789012:assumed-role/GameRole/web-identity-federation</
Arn>
      <AssumedRoleId>AROAJU4SA2VW5SZRF2YMG:web-identity-federation</AssumedRoleId>
    </AssumedRoleUser>
  </AssumeRoleWithWebIdentityResult>
  <ResponseMetadata>
    <RequestId>c265ac8e-2ae4-11e3-8775-6969323a932d</RequestId>
  </ResponseMetadata>
</AssumeRoleWithWebIdentityResponse>
```

3. **Access AWS resources.** The response from AWS STS contains information that your app requires in order to access DynamoDB resources:
   - The `AccessKeyID`, `SecretAccessKey`, and `SessionToken` fields contain security credentials that are valid for this user and this app only.
   - The `Expiration` field signifies the time limit for these credentials, after which they are no longer valid.
   - The `AssumedRoleId` field contains the name of a session-specific IAM role that has been assumed by the app. The app honors the access controls in the IAM policy document for the duration of this session.
   - The `SubjectFromWebIdentityToken` field contains the unique ID that appears in an IAM policy variable for this particular identity provider. The following are the IAM policy variables for supported providers, and some example values for them:

Policy Variable	Example Value
`${www.amazon.com:user_id}`	`amzn1.account.AGJZDKHJKAUUSW6C44CHPEXAMPLE`
`${graph.facebook.com:id}`	`123456789`
`${accounts.google.com:sub}`	`123456789012345678901`

For example IAM policies where these policy variables are used, see [Example policies: Using conditions for fine-grained access control](#).

For more information about how AWS STS generates temporary access credentials, see Requesting Temporary Security Credentials in *IAM User Guide*.

DynamoDB API permissions: Actions, resources, and conditions reference

When you are setting up Identity and Access Management for Amazon DynamoDB and writing a permissions policy that you can attach to an IAM identity (identity-based policies), you can use the list of Actions, resources, and condition keys for Amazon DynamoDB in the *IAM User Guide* as a reference. The page lists each DynamoDB API operation, the corresponding actions for which you can grant permissions to perform the action, and the AWS resource for which you can grant the permissions. You specify the actions in the policy's `Action` field, and you specify the resource value in the policy's `Resource` field.

You can use AWS-wide condition keys in your DynamoDB policies to express conditions. For a complete list of AWS-wide keys, see the IAM JSON policy elements reference in the *IAM User Guide*.

In addition to the AWS-wide condition keys, DynamoDB has its own specific keys that you can use in conditions. For more information, see Using IAM policy conditions for fine-grained access control.

Related topics

- Identity and Access Management for Amazon DynamoDB
- Using IAM policy conditions for fine-grained access control

Compliance validation by industry for DynamoDB

To learn whether an AWS service is within the scope of specific compliance programs, see AWS services in Scope by Compliance Program and choose the compliance program that you are interested in. For general information, see AWS Compliance Programs.

You can download third-party audit reports using AWS Artifact. For more information, see Downloading Reports in AWS Artifact.

Your compliance responsibility when using AWS services is determined by the sensitivity of your data, your company's compliance objectives, and applicable laws and regulations. For more information about your compliance responsibility when using AWS services, see AWS Security Documentation.

Resilience and disaster recovery in Amazon DynamoDB

The AWS global infrastructure is built around AWS Regions and Availability Zones. AWS Regions provide multiple physically separated and isolated Availability Zones, which are connected with low-latency, high-throughput, and highly redundant networking. With Availability Zones, you can design and operate applications and databases that automatically fail over between Availability Zones without interruption. Availability Zones are more highly available, fault tolerant, and scalable than traditional single or multiple data center infrastructures.

If you need to replicate your data or applications over greater geographic distances, use AWS Local Regions. An AWS Local Region is a single data center designed to complement an existing AWS Region. Like all AWS Regions, AWS Local Regions are completely isolated from other AWS Regions.

For more information about AWS Regions and Availability Zones, see AWS global infrastructure.

Amazon DynamoDB automatically replicates your data across three Availability Zones in a Region, providing built-in high durability and a 99.99% availability SLA. In addition, DynamoDB offers several features to help support your data resiliency and backup needs.

On-demand backup and restore

DynamoDB provides on-demand backup capability. It allows you to create full backups of your tables for long-term retention and archival. For more information, see On-Demand backup and restore for DynamoDB.

Point-in-time recovery

Point-in-time recovery helps protect your DynamoDB tables from accidental write or delete operations. With point in time recovery, you don't have to worry about creating, maintaining, or scheduling on-demand backups. For more information, see Point-in-time recovery for DynamoDB.

Global tables that sync across AWS regions

DynamoDB automatically spreads the data and traffic for your tables over a sufficient number of servers to handle your throughput and storage requirements, while maintaining consistent and fast performance. All of your data is stored on solid-state disks (SSDs) and is automatically replicated across multiple Availability Zones in an AWS Region, providing built-in high availability and data durability. You can use global tables to keep DynamoDB tables in sync across AWS Regions.

Infrastructure security in Amazon DynamoDB

As a managed service, Amazon DynamoDB is protected by the AWS global network security procedures that are described in Infrastructure protection located in the AWS Well-Architected Framework.

You use AWS published API calls to access DynamoDB through the network. Clients can use TLS (Transport Layer Security) version 1.2 or 1.3. Clients must also support cipher suites with perfect forward secrecy (PFS) such as Ephemeral Diffie-Hellman (DHE) or Elliptic Curve Diffie-Hellman Ephemeral (ECDHE). Most modern systems such as Java 7 and later support these modes. Additionally, requests must be signed using an access key ID and a secret access key that is associated with an IAM principal. Or you can use the AWS Security Token Service (AWS STS) to generate temporary security credentials to sign requests.

You can also use a virtual private cloud (VPC) endpoint for DynamoDB to enable Amazon EC2 instances in your VPC to use their private IP addresses to access DynamoDB with no exposure to the public internet. For more information, see Using Amazon VPC endpoints to access DynamoDB.

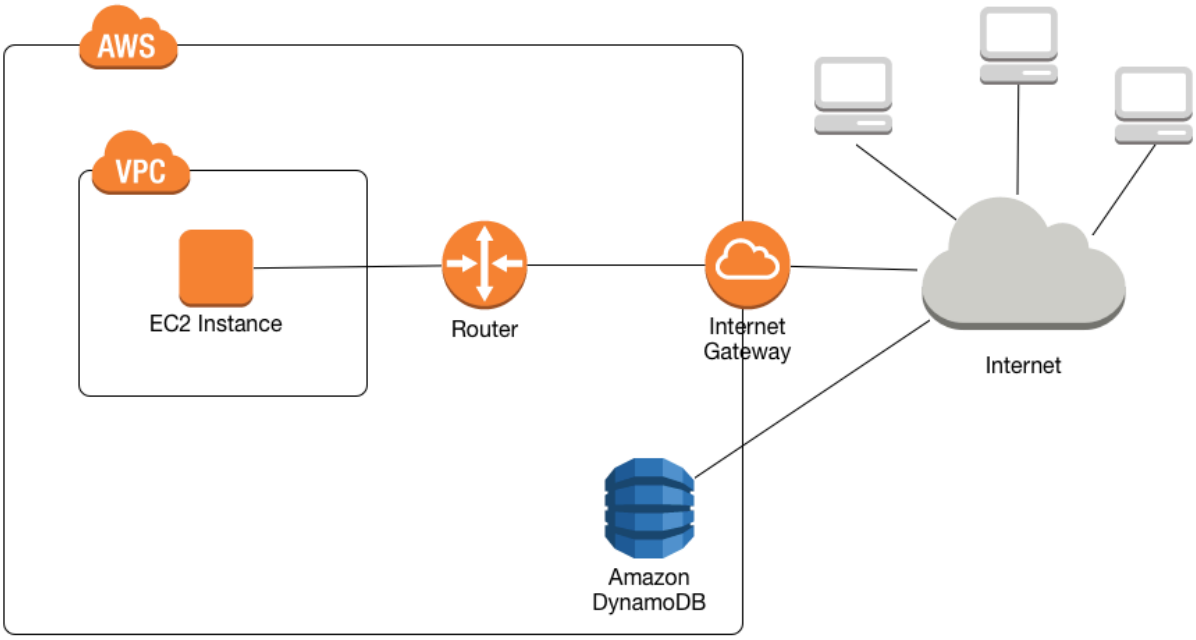
Using Amazon VPC endpoints to access DynamoDB

For security reasons, many AWS customers run their applications within an Amazon Virtual Private Cloud environment (Amazon VPC). With Amazon VPC, you can launch Amazon EC2 instances into a virtual private cloud, which is logically isolated from other networks—including the public internet. With an Amazon VPC, you have control over its IP address range, subnets, routing tables, network gateways, and security settings.

> **Note**
>
> If you created your AWS account after December 4, 2013, then you already have a default VPC in each AWS Region. A default VPC is ready for you to use—you can immediately start using it without having to perform any additional configuration steps.
> For more information, see Default VPC and Default Subnets in the *Amazon VPC User Guide*.

To access the public internet, your VPC must have an internet gateway—a virtual router that connects your VPC to the internet. This allows applications running on Amazon EC2 in your VPC to access internet resources, such as Amazon DynamoDB.

By default, communications to and from DynamoDB use the HTTPS protocol, which protects network traffic by using SSL/TLS encryption. The following diagram shows an Amazon EC2 instance in a VPC accessing DynamoDB, by having DynamoDB use an internet gateway rather than VPC endpoints.



Many customers have legitimate privacy and security concerns about sending and receiving data across the public internet. You can address these concerns by using a virtual private network (VPN) to route all DynamoDB network traffic through your own corporate network infrastructure. However, this approach can introduce bandwidth and availability challenges.

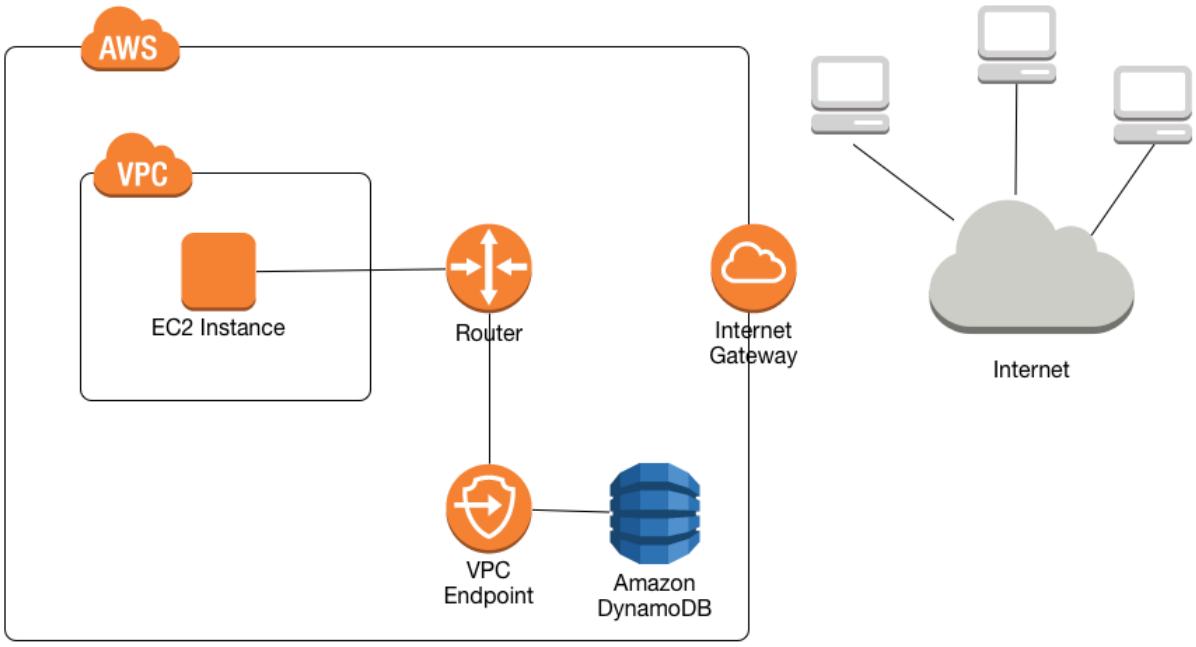
VPC endpoints for DynamoDB can alleviate these challenges. A *VPC endpoint* for DynamoDB enables Amazon EC2 instances in your VPC to use their private IP addresses to access DynamoDB with no exposure to the public internet. Your EC2 instances do not require public IP addresses, and you don't need an internet gateway, a NAT device, or a virtual private gateway in your VPC. You use endpoint policies to control access to DynamoDB. Traffic between your VPC and the AWS service does not leave the Amazon network.

> **Note**
>
> Even when you use public IP addresses, all VPC communication between instances and services hosted in AWS is kept private within the AWS network. Packets that originate from

the AWS network with a destination on the AWS network stay on the AWS global network, except traffic to or from AWS China Regions.

When you create a VPC endpoint for DynamoDB, any requests to a DynamoDB endpoint within the Region (for example, *dynamodb.us-west-2.amazonaws.com*) are routed to a private DynamoDB endpoint within the Amazon network. You don't need to modify your applications running on EC2 instances in your VPC. The endpoint name remains the same, but the route to DynamoDB stays entirely within the Amazon network, and does not access the public internet.

The following diagram shows how an EC2 instance in a VPC can use a VPC endpoint to access DynamoDB.



For more information, see the section called “Tutorial: Using a VPC endpoint for DynamoDB”.

Sharing Amazon VPC endpoints and DynamoDB

In order to enable access to the DynamoDB service through a VPC subnet's gateway endpoint, you must have owner account permissions for that VPC subnet.

Once the VPC subnet’s gateway endpoint has been granted access to DynamoDB, any AWS account with access to that subnet can use DynamoDB. This means all account users within the VPC

subnet can use any DynamoDB tables which they have access to. This includes DynamoDB tables associated with a different account than the VPC subnet. The VPC subnet owner can still restrict any particular user within the subnet from using the DynamoDB service through the gateway endpoint, at their discretion.

Tutorial: Using a VPC endpoint for DynamoDB

This section walks you through setting up and using a VPC endpoint for DynamoDB.

Topics

- Step 1: Launch an Amazon EC2 instance
- Step 2: Configure your Amazon EC2 instance
- Step 3: Create a VPC endpoint for DynamoDB
- Step 4: (Optional) Clean up

Step 1: Launch an Amazon EC2 instance

In this step, you launch an Amazon EC2 instance in your default Amazon VPC. You can then create and use a VPC endpoint for DynamoDB.

1. Open the Amazon EC2 console at https://console.aws.amazon.com/ec2/.
2. Choose **Launch Instance** and do the following:

   Step 1: Choose an Amazon Machine Image (AMI)

   - At the top of the list of AMIs, go to **Amazon Linux AMI** and choose **Select**.

   Step 2: Choose an Instance Type

   - At the top of the list of instance types, choose **t2.micro**.
   - Choose **Next: Configure Instance Details**.

   Step 3: Configure Instance Details

   - Go to **Network** and choose your default VPC.

     Choose **Next: Add Storage**.

Step 4: Add Storage

- Skip this step by choosing **Next: Tag Instance**.

Step 5: Tag Instance

- Skip this step by choosing **Next: Configure Security Group**.

Step 6: Configure Security Group

- Choose **Select an existing security group**.
- In the list of security groups, choose **default**. This is the default security group for your VPC.
- Choose **Next: Review and Launch**.

Step 7: Review Instance Launch

- Choose **Launch**.

3. In the **Select an existing key pair or create a new key pair** window, do one of the following:
   - If you do not have an Amazon EC2 key pair, choose **Create a new key pair** and follow the instructions. You will be asked to download a private key file (*.pem* file); you will need this file later when you log in to your Amazon EC2 instance.
   - If you already have an existing Amazon EC2 key pair, go to **Select a key pair** and choose your key pair from the list. You must already have the private key file (*.pem* file) available in order to log in to your Amazon EC2 instance.
4. When you have configured your key pair, choose **Launch Instances**.
5. Return to the Amazon EC2 console home page and choose the instance that you launched. In the lower pane, on the **Description** tab, find the **Public DNS** for your instance. For example: `ec2-00-00-00-00.us-east-1.compute.amazonaws.com`.

   Make a note of this public DNS name, because you will need it in the next step in this tutorial (Step 2: Configure your Amazon EC2 instance).

> **Note**
>
> It will take a few minutes for your Amazon EC2 instance to become available. Before you go on to the next step, ensure that the **Instance State** is `running` and that all of its **Status Checks** have passed.

Step 2: Configure your Amazon EC2 instance

When your Amazon EC2 instance is available, you will be able to log into it and prepare it for first use.

> **Note**
>
> The following steps assume that you are connecting to your Amazon EC2 instance from a computer running Linux. For other ways to connect, see Connect to Your Linux Instance in the Amazon EC2 User Guide.

1. You will need to authorize inbound SSH traffic to your Amazon EC2 instance. To do this, you will create a new EC2 security group, and then assign the security group to your EC2 instance.

   a. In the navigation pane, choose **Security Groups**.

   b. Choose **Create Security Group**. In the **Create Security Group** window, do the following:

      - **Security group name**—type a name for your security group. For example: `my-ssh-access`
      - **Description**—type a short description for the security group.
      - **VPC**—choose your default VPC.
      - In the **Security group rules** section, choose **Add Rule** and do the following:
        - **Type**—choose SSH.
        - **Source**—choose My IP.

      When the settings are as you want them, choose **Create**.

   c. In the navigation pane, choose **Instances**.

d. Choose the Amazon EC2 instance that you launched in Step 1: Launch an Amazon EC2 instance.

e. Choose **Actions** --> **Networking** --> **Change Security Groups**.

f. In the **Change Security Groups**, select the security group that you created earlier in this procedure (for example: `my-ssh-access`). The existing `default` security group should also be selected. When the settings are as you want them, choose **Assign Security Groups**.

2. Use the `ssh` command to log in to your Amazon EC2 instance, as in the following example.

```
ssh -i my-keypair.pem ec2-user@public-dns-name
```

You will need to specify your private key file (*.pem* file) and the public DNS name of your instance. (See Step 1: Launch an Amazon EC2 instance).

The login ID is `ec2-user`. No password is required.

3. Configure your AWS credentials as shown in the following example. Enter your AWS access key ID, secret key, and default Region name when prompted.

```
aws configure
```

```
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]:
```

You are now ready to create a VPC endpoint for DynamoDB.

Step 3: Create a VPC endpoint for DynamoDB

In this step, you will create a VPC endpoint for DynamoDB and test it to make sure that it works.

1. Before you begin, verify that you can communicate with DynamoDB using its public endpoint.

```
aws dynamodb list-tables
```

The output will show a list of DynamoDB tables that you currently own. (If you don't have any tables, the list will be empty.).

2. Verify that DynamoDB is an available service for creating VPC endpoints in the current AWS Region. (The command is shown in bold text, followed by example output.)

```
aws ec2 describe-vpc-endpoint-services
```

```
{
    "ServiceNames": [
        "com.amazonaws.us-east-1.s3",
        "com.amazonaws.us-east-1.dynamodb"
    ]
}
```

In the example output, DynamoDB is one of the services available, so you can proceed with creating a VPC endpoint for it.

3. Determine your VPC identifier.

```
aws ec2 describe-vpcs
```

```
{
    "Vpcs": [
        {
            "VpcId": "vpc-0bbc736e",
            "InstanceTenancy": "default",
            "State": "available",
            "DhcpOptionsId": "dopt-8454b7e1",
            "CidrBlock": "172.31.0.0/16",
            "IsDefault": true
        }
    ]
}
```

In the example output, the VPC ID is `vpc-0bbc736e`.

4. Create the VPC endpoint. For the `--vpc-id` parameter, specify the VPC ID from the previous step. Use the `--route-table-ids` parameter to associate the endpoint with your route tables.

```
aws ec2 create-vpc-endpoint --vpc-id vpc-0bbc736e --service-name com.amazonaws.us-
east-1.dynamodb --route-table-ids rtb-11aa22bb
```

```
{
    "VpcEndpoint": {
        "PolicyDocument": "{\"Version\":\"2008-10-17\",\"Statement\":[{\"Effect\":
\"Allow\",\"Principal\":\"*\",\"Action\":\"*\",\"Resource\":\"*\"}]}",
        "VpcId": "vpc-0bbc736e",
        "State": "available",
        "ServiceName": "com.amazonaws.us-east-1.dynamodb",
        "RouteTableIds": [
            "rtb-11aa22bb"
        ],
        "VpcEndpointId": "vpce-9b15e2f2",
        "CreationTimestamp": "2017-07-26T22:00:14Z"
    }
}
```

5. Verify that you can access DynamoDB through the VPC endpoint.

```
aws dynamodb list-tables
```

If you want, you can try some other AWS CLI commands for DynamoDB. For more information, see the AWS CLI Command Reference.

Step 4: (Optional) Clean up

If you want to delete the resources you have created in this tutorial, follow these procedures:

To remove your VPC endpoint for DynamoDB

1. Log in to your Amazon EC2 instance.
2. Determine the VPC endpoint ID.

```
aws ec2 describe-vpc-endpoints
```

```
{
    "VpcEndpoint": {
        "PolicyDocument": "{\"Version\":\"2008-10-17\",\"Statement\":[{\"Effect\":
\"Allow\",\"Principal\":\"*\",\"Action\":\"*\",\"Resource\":\"*\"}]}",
        "VpcId": "vpc-0bbc736e",
        "State": "available",
        "ServiceName": "com.amazonaws.us-east-1.dynamodb",
```

```
        "RouteTableIds": [],
        "VpcEndpointId": "vpce-9b15e2f2",
        "CreationTimestamp": "2017-07-26T22:00:14Z"
    }
}
```

In the example output, the VPC endpoint ID is `vpce-9b15e2f2`.

3. Delete the VPC endpoint.

```
aws ec2 delete-vpc-endpoints --vpc-endpoint-ids vpce-9b15e2f2
```

```
{
    "Unsuccessful": []
}
```

The empty array `[]` indicates success (there were no unsuccessful requests).

To terminate your Amazon EC2 instance

1. Open the Amazon EC2 console at https://console.aws.amazon.com/ec2/.
2. In the navigation pane, choose **Instances**.
3. Choose your Amazon EC2 instance.
4. Choose **Actions**, **Instance State**, **Terminate**.
5. In the confirmation window, choose **Yes, Terminate**.

AWS PrivateLink for DynamoDB

With AWS PrivateLink for DynamoDB, you can provision *interface Amazon VPC endpoints* (interface endpoints) in your virtual private cloud (Amazon VPC). These endpoints are directly accessible from applications that are on premises over VPN and Direct Connect, or in a different AWS Region over Amazon VPC peering. Using AWS PrivateLink and interface endpoints, you can simplify private network connectivity from your applications to DynamoDB.

Applications in your VPC do not need public IP addresses to communicate with DynamoDB using VPC interface endpoints for DynamoDB operations. Interface endpoints are represented by one or more elastic network interfaces (ENIs) that are assigned private IP addresses from subnets in your

Amazon VPC. Requests to DynamoDB over interface endpoints stay on the Amazon network. You can also access interface endpoints in your Amazon VPC from on-premises applications through AWS Direct Connect or AWS Virtual Private Network (Site-to-Site VPN). For more information about how to connect your Amazon VPC with your on-premises network, see the Direct Connect User Guide and the AWS Site-to-Site VPN User Guide.

For general information about interface endpoints, see Interface Amazon VPC endpoints (AWS PrivateLink) in the *AWS PrivateLink Guide*. AWS PrivateLink is also supported for Amazon DynamoDB Streams endpoints. For more information, see the section called "AWS PrivateLink for DynamoDB Streams".

Topics

- Types of Amazon VPC endpoints for Amazon DynamoDB
- Considerations when using AWS PrivateLink for Amazon DynamoDB
- Creating an Amazon VPC endpoint
- Accessing Amazon DynamoDB interface endpoints
- Accessing DynamoDB tables and control API operations from DynamoDB interface endpoints
- Updating an on-premises DNS configuration
- Creating an Amazon VPC endpoint policy for DynamoDB
- Using DynamoDB endpoints with AWS Management Console Private Access
- AWS PrivateLink for DynamoDB Streams
- Using AWS PrivateLink for DynamoDB Accelerator (DAX)

Types of Amazon VPC endpoints for Amazon DynamoDB

You can use two types of Amazon VPC endpoints to access Amazon DynamoDB: *gateway endpoints* and *interface endpoints* (by using AWS PrivateLink). A *gateway endpoint* is a gateway that you specify in your route table to access DynamoDB from your Amazon VPC over the AWS network. *Interface endpoints* extend the functionality of gateway endpoints by using private IP addresses to route requests to DynamoDB from within your Amazon VPC, on premises, or from an Amazon VPC in another AWS Region by using Amazon VPC peering or AWS Transit Gateway. For more information, see What is Amazon VPC peering? and Transit Gateway vs Amazon VPC peering.

Interface endpoints are compatible with gateway endpoints. If you have an existing gateway endpoint in the Amazon VPC, you can use both types of endpoints in the same Amazon VPC.

Gateway endpoints for DynamoDB	Interface endpoints for DynamoDB
In both cases, your network traffic remains on the AWS network.	
Use Amazon DynamoDB public IP addresses	Use private IP addresses from your Amazon VPC to access Amazon DynamoDB
Do not allow access from on premises	Allow access from on premises
Do not allow access from another AWS Region	Allow access from an Amazon VPC endpoint in another AWS Region by using Amazon VPC peering or AWS Transit Gateway
Not billed	Billed

For more information about gateway endpoints, see Gateway Amazon VPC endpoints in the *AWS PrivateLink Guide*.

Considerations when using AWS PrivateLink for Amazon DynamoDB

Amazon VPC considerations apply to AWS PrivateLink for Amazon DynamoDB. For more information, see Interface endpoint considerations and AWS PrivateLink quotas in the *AWS PrivateLink Guide*. In addition, the following restrictions apply.

AWS PrivateLink for Amazon DynamoDB does not support the following:

- Transport Layer Security (TLS) 1.1
- Private and Hybrid Domain Name System (DNS) services

> **Important**
>
> Do not create private hosted zones to override DynamoDB endpoint DNS names (such as `dynamodb.region.amazonaws.com` or `*.region.amazonaws.com`) to route traffic to your interface endpoints. DynamoDB DNS configurations may change over time. Custom DNS overrides are not compatible with these changes and can cause requests to unexpectedly route over public IP addresses instead of your interface endpoints.

To access DynamoDB through AWS PrivateLink, configure your clients to use the Amazon VPC endpoint URL directly (for example, `https://vpce-1a2b3c4d-5e6f.dynamodb.region.vpce.amazonaws.com`).

You can submit up to 50,000 requests per second for each AWS PrivateLink endpoint that you enable.

> **Note**
>
> Network connectivity timeouts to AWS PrivateLink endpoints are not within the scope of DynamoDB error responses and need to be appropriately handled by your applications connecting to the PrivateLink endpoints.

Creating an Amazon VPC endpoint

To create an Amazon VPC interface endpoint, see Create an Amazon VPC endpoint in the *AWS PrivateLink Guide*.

Accessing Amazon DynamoDB interface endpoints

When you create an interface endpoint, DynamoDB generates two types of endpoint-specific, DynamoDB DNS names: *Regional* and *Zonal*.

- A *Regional* DNS name includes a unique Amazon VPC endpoint ID, a service identifier, the AWS Region, and `vpce.amazonaws.com` in its name. For example, for Amazon VPC endpoint ID *`vpce-1a2b3c4d`*, the DNS name generated might be similar to *`vpce-1a2b3c4d-5e6f`*`.dynamodb.us-east-1.vpce.amazonaws.com`.
- A *Zonal* DNS name includes the Availability Zone—for example, *`vpce-1a2b3c4d-5e6f`*`-`**`us-east-1a.`**`dynamodb.us-east-1.vpce.amazonaws.com`. You might use this option if your architecture isolates Availability Zones. For example, you could use it for fault containment or to reduce Regional data transfer costs.

> **Note**
>
> To achieve optimal reliability, we recommend deploying your service across a minimum of three availability zones.

Accessing DynamoDB tables and control API operations from DynamoDB interface endpoints

You can use the AWS CLI or AWS SDKs to access DynamoDB tables and control API operations through DynamoDB interface endpoints.

AWS CLI examples

To access DynamoDB tables or DynamoDB control API operations through DynamoDB interface endpoints in AWS CLI commands, use the `--region` and `--endpoint-url` parameters.

Example: Create a VPC endpoint

```
aws ec2 create-vpc-endpoint \
--region us-east-1 \
--service-name com.amazonaws.us-east-1.dynamodb \
--vpc-id client-vpc-id \
--subnet-ids client-subnet-id \
--vpc-endpoint-type Interface \
--security-group-ids client-sg-id
```

Example: Modify a VPC endpoint

```
aws ec2 modify-vpc-endpoint \
--region us-east-1 \
--vpc-endpoint-id client-vpc-endpoint-id \
--policy-document policy-document \ #example optional parameter
--add-security-group-ids security-group-ids \ #example optional parameter
# any additional parameters needed, see Privatelink documentation for more details
```

Example: List tables using an endpoint URL

In the following example, replace the Region `us-east-1` and the DNS name of the VPC endpoint ID `vpce-1a2b3c4d-5e6f.dynamodb.us-east-1.vpce.amazonaws.com` with your own information.

```
aws dynamodb --region us-east-1 --endpoint https://vpce-1a2b3c4d-5e6f.dynamodb.us-
east-1.vpce.amazonaws.com list-tables
```

AWS SDK examples

To access DynamoDB tables or DynamoDB control API operations through DynamoDB interface endpoints when using the AWS SDKs, update your SDKs to the latest version. Then, configure your clients to use an endpoint URL for accessing a table or DynamoDB control API operation through DynamoDB interface endpoints.

SDK for Python (Boto3)

Example: Use an endpoint URL to access a DynamoDB table

In the following example, replace the Region `us-east-1` and VPC endpoint ID `https://vpce-1a2b3c4d-5e6f.dynamodb.us-east-1.vpce.amazonaws.com` with your own information.

```
ddb_client = session.client(
service_name='dynamodb',
region_name='us-east-1',
endpoint_url='https://vpce-1a2b3c4d-5e6f.dynamodb.us-east-1.vpce.amazonaws.com'
)
```

SDK for Java 1.x

Example: Use an endpoint URL to access a DynamoDB table

In the following example, replace the Region `us-east-1` and VPC endpoint ID `https://vpce-1a2b3c4d-5e6f.dynamodb.us-east-1.vpce.amazonaws.com` with your own information.

```
//client build with endpoint config
final AmazonDynamoDB dynamodb =
 AmazonDynamoDBClientBuilder.standard().withEndpointConfiguration(
```

```
        new AwsClientBuilder.EndpointConfiguration(
                "https://vpce-1a2b3c4d-5e6f.dynamodb.us-east-1.vpce.amazonaws.com",
                Regions.DEFAULT_REGION.getName()
        )
).build();
```

SDK for Java 2.x

Example: Use an endpoint URL to access DynamoDB table

In the following example, replace the Region us-east-1 and VPC endpoint ID https://vpce-1a2b3c4d-5e6f.dynamodb.us-east-1.vpce.amazonaws.com with your own information.

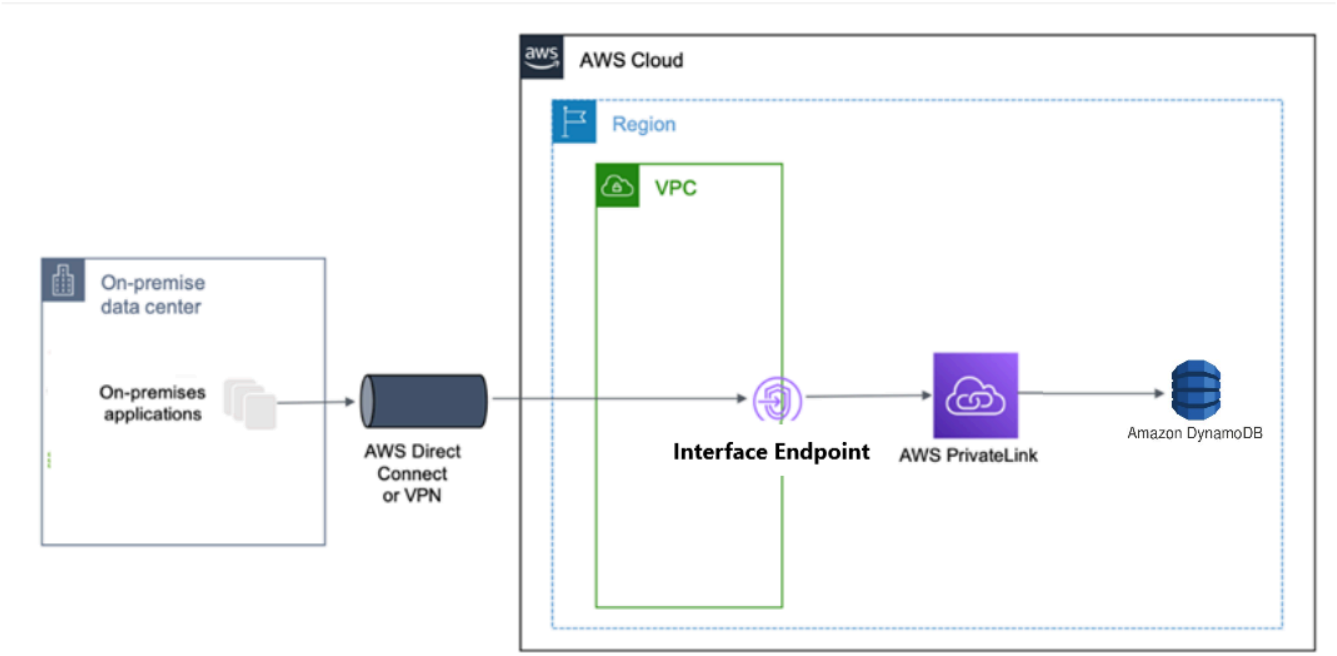
```
Region region = Region.US_EAST_1;
dynamoDbClient = DynamoDbClient.builder().region(region)
.endpointOverride(URI.create("https://vpce-1a2b3c4d-5e6f.dynamodb.us-
east-1.vpce.amazonaws.com"))
.build()
```

Updating an on-premises DNS configuration

When using endpoint-specific DNS names to access the interface endpoints for DynamoDB, you don't have to update your on-premises DNS resolver. You can resolve the endpoint-specific DNS name with the private IP address of the interface endpoint from the public DynamoDB DNS domain.

Using interface endpoints to access DynamoDB without a gateway endpoint or an internet gateway in the Amazon VPC

Interface endpoints in your Amazon VPC can route both in-Amazon VPC applications and on-premises applications to DynamoDB over the Amazon network, as illustrated in the following diagram.

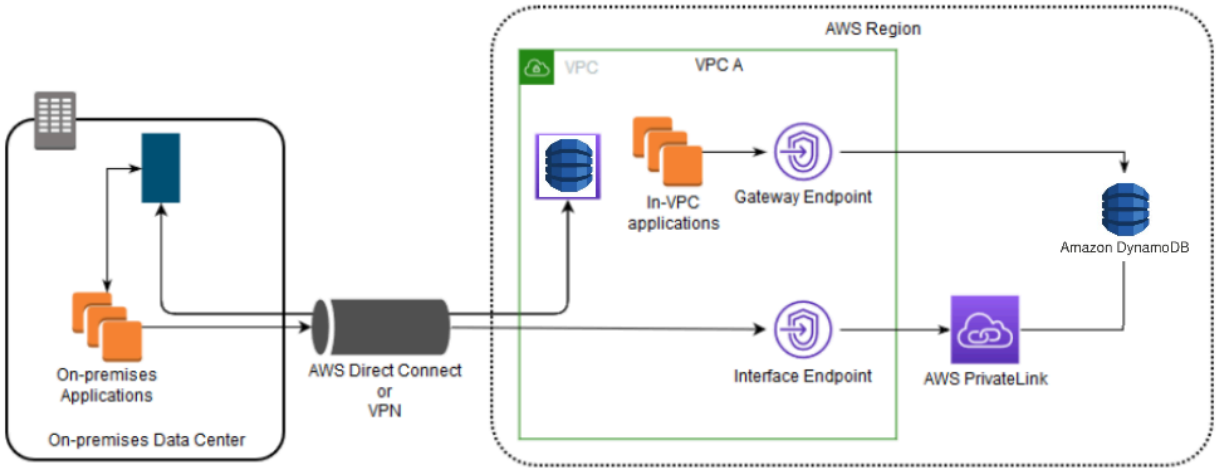


The diagram illustrates the following:

- Your on-premises network uses Direct Connect or Site-to-Site VPN to connect to Amazon VPC A.
- Your applications on-premises and in Amazon VPC A use endpoint-specific DNS names to access DynamoDB through the DynamoDB interface endpoint.
- On-premises applications send data to the interface endpoint in the Amazon VPC through Direct Connect (or Site-to-Site VPN). AWS PrivateLink moves the data from the interface endpoint to DynamoDB over the AWS network.
- In-Amazon VPC applications also send traffic to the interface endpoint. AWS PrivateLink moves the data from the interface endpoint to DynamoDB over the AWS network.

Using gateway endpoints and interface endpoints together in the same Amazon VPC to access DynamoDB

You can create interface endpoints and retain the existing gateway endpoint in the same Amazon VPC, as the following diagram shows. By taking this approach, you allow in-Amazon VPC applications to continue accessing DynamoDB through the gateway endpoint, which is not billed. Then, only your on-premises applications would use interface endpoints to access DynamoDB. To access DynamoDB this way, you must update your on-premises applications to use endpoint-specific DNS names for DynamoDB.



The diagram illustrates the following:

- On-premises applications use endpoint-specific DNS names to send data to the interface endpoint within the Amazon VPC through Direct Connect (or Site-to-Site VPN). AWS PrivateLink moves the data from the interface endpoint to DynamoDB over the AWS network.
- Using default Regional DynamoDB names, in-Amazon VPC applications send data to the gateway endpoint that connects to DynamoDB over the AWS network.

For more information about gateway endpoints, see Gateway Amazon VPC endpoints in the *Amazon VPC User Guide*.

Creating an Amazon VPC endpoint policy for DynamoDB

You can attach an endpoint policy to your Amazon VPC endpoint that controls access to DynamoDB. The policy specifies the following information:

- The AWS Identity and Access Management (IAM) principal that can perform actions
- The actions that can be performed
- The resources on which actions can be performed

Topics

- Example: Restricting access to a specific table from an Amazon VPC endpoint

Example: Restricting access to a specific table from an Amazon VPC endpoint

You can create an endpoint policy that restricts access to only specific DynamoDB tables. This type of policy is useful if you have other AWS services in your Amazon VPC that use tables. The following table policy restricts access to only the *DOC-EXAMPLE-TABLE*. To use this endpoint policy, replace *DOC-EXAMPLE-TABLE* with the name of your table.

JSON

```
{
"Version":"2012-10-17",
  "Id": "Policy1216114807515",
  "Statement": [
    { "Sid": "Access-to-specific-table-only",
      "Principal": "*",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem"
      ],
      "Effect": "Allow",
      "Resource": ["arn:aws:dynamodb:us-east-1:111122223333:table/DOC-EXAMPLE-
TABLE",
                   "arn:aws:dynamodb:us-east-1:111122223333:table/DOC-EXAMPLE-
TABLE/*"]
    }
  ]
}
```

Using DynamoDB endpoints with AWS Management Console Private Access

You must set up DNS configuration for DynamoDB and DynamoDB Streams when using VPC endpoints with the DynamoDB console in AWS Management Console Private Access.

To configure DynamoDB to be accessible in AWS Management Console Private Access, you must create the following two VPC endpoints:

- `com.amazonaws.<region>.dynamodb`
- `com.amazonaws.<region>.dynamodb-streams`

When you create the VPC endpoints, navitage to the Route53 console and create a private hosted zone for DynamoDB using the regional endpoint `dynamodb.us-east-1.amazonaws.com`.

Create the following two alias records in the private hosted zone:

- `dynamodb.<region>.amazonaws.com` that routes traffic to the VPC endpoint `com.amazonaws.<region>.dynamodb`.
- `streams.dynamodb.<region>.amazonaws.com` that routes traffic to the VPC endpoint `com.amazonaws.<region>.dynamodb-streams`.

AWS PrivateLink for DynamoDB Streams

With AWS PrivateLink for Amazon DynamoDB Streams, you can provision interface Amazon VPC endpoints (interface endpoints) in your virtual private cloud (Amazon VPC). These endpoints are directly accessible from applications that are on premises over VPN and Direct Connect, or in a different AWS Region over Amazon VPC peering. Using AWS PrivateLink and interface endpoints, you can simplify private network connectivity from your applications to DynamoDB Streams.

Applications in your Amazon VPC do not need public IP addresses to communicate with DynamoDB Streams using Amazon VPC interface endpoints for DynamoDB Streams operations. Interface endpoints are represented by one or more elastic network interfaces (ENIs) that are assigned private IP addresses from subnets in your Amazon VPC. Requests to DynamoDB Streams over interface endpoints stay on the Amazon network. You can also access interface endpoints in your Amazon VPC from on-premises applications through Direct Connect or AWS Virtual Private Network (AWS VPN). For more information about how to connect your AWS Virtual Private Network with your on-premises network, see the *Direct Connect User Guide* and the *AWS Site-to-Site VPN User Guide*.

For general information about interface endpoints, see Interface Amazon VPC endpoints (AWS PrivateLink).

> **Note**
>
> Only interface endpoints are supported for DynamoDB Streams. Gateway endpoints are not supported.

Topics

- Considerations when using AWS PrivateLink for Amazon DynamoDB Streams
- Creating an Amazon VPC endpoint
- Accessing Amazon DynamoDB Streams interface endpoints
- Accessing DynamoDB Streams API operations from DynamoDB Streams interface endpoints
- AWS SDK examples
- Creating an Amazon VPC endpoint policy for DynamoDB Streams
- Using DynamoDB endpoints with AWS Management Console Private Access

Considerations when using AWS PrivateLink for Amazon DynamoDB Streams

Amazon VPC considerations apply to AWS PrivateLink for Amazon DynamoDB Streams. For more information, see interface endpoint considerations and AWS PrivateLink quotas. The following restrictions apply.

AWS PrivateLink for Amazon DynamoDB Streams doesn't support the following:

- Transport Layer Security (TLS) 1.1
- Private and Hybrid Domain Name System (DNS) services

> **Important**
>
> Do not create private hosted zones to override DynamoDB Streams endpoint DNS names to route traffic to your interface endpoints. DynamoDB DNS configurations may change over time and custom DNS overrides can cause requests to unexpectedly route over public IP addresses instead of your interface endpoints.
> To access DynamoDB Streams through AWS PrivateLink, configure your clients to use the Amazon VPC endpoint URL directly (for example, `https://vpce-1a2b3c4d-5e6f.streams.dynamodb.`*`region`*`.vpce.amazonaws.com`).

> **Note**
>
> Network connectivity timeouts to AWS PrivateLink endpoints are not within the scope of DynamoDB Streams error responses and need to be appropriately handled by your applications connecting to the AWS PrivateLink endpoints.

Creating an Amazon VPC endpoint

To create an Amazon VPC interface endpoint, see Create an Amazon VPC endpoint in the *AWS PrivateLink Guide*.

Accessing Amazon DynamoDB Streams interface endpoints

When you create an interface endpoint, DynamoDB generates two types of endpoint-specific, DynamoDB Streams DNS names: *Regional* and *Zonal*.

- A *Regional* DNS name includes a unique Amazon VPC endpoint ID, a service identifier, the AWS Region, and `vpce.amazonaws.com` in its name. For example, for Amazon VPC endpoint ID *`vpce-1a2b3c4d`*, the DNS name generated might be similar to *`vpce-1a2b3c4d-5e6f`*`.streams.dynamodb.us-east-1.vpce.amazonaws.com`.
- A *Zonal* DNS name includes the Availability Zone—for example, *`vpce-1a2b3c4d-5e6f`*`-`**`us-east-1a`**`.streams.dynamodb.us-east-1.vpce.amazonaws.com`. You might use this option if your architecture isolates Availability Zones. For example, you could use it for fault containment or to reduce Regional data transfer costs.

Accessing DynamoDB Streams API operations from DynamoDB Streams interface endpoints

You can use the AWS CLI or AWS SDKs to access DynamoDB Streams API operations through DynamoDB Streams interface endpoints.

AWS CLI examples

To access DynamoDB Streams or API operations through DynamoDB Streams interface endpoints in AWS CLI commands, use the `--region` and `--endpoint-url` parameters.

Example: Create a VPC endpoint

```
aws ec2 create-vpc-endpoint \
--region us-east-1 \
--service-name com.amazonaws.us-east-1.dynamodb-streams \
--vpc-id client-vpc-id \
--subnet-ids client-subnet-id \
--vpc-endpoint-type Interface \
--security-group-ids client-sg-id
```

Example: Modify a VPC endpoint

```
aws ec2 modify-vpc-endpoint \
--region us-east-1 \
--vpc-endpoint-id client-vpc-endpoint-id \
--policy-document policy-document \ #example optional parameter
--add-security-group-ids security-group-ids \ #example optional parameter
# any additional parameters needed, see Privatelink documentation for more details
```

Example: List streams using an endpoint URL

In the following example, replace the Region `us-east-1` and the DNS name of the VPC endpoint ID `vpce-1a2b3c4d-5e6f.streams.dynamodb.us-east-1.vpce.amazonaws.com` with your own information.

```
aws dynamodbstreams --region us-east-1 –endpoint https://
vpce-1a2b3c4d-5e6f.streams.dynamodb.us-east-1.vpce.amazonaws.com list-streams
```

AWS SDK examples

To access Amazon DynamoDB Streams API operations through DynamoDB Streams interface endpoints when using the AWS SDKs, update your SDKs to the latest version. Then, configure your clients to use an endpoint URL for DynamoDB Streams API operation through DynamoDB Streams interface endpoints.

SDK for Python (Boto3)

Example: Use an endpoint URL to access a DynamoDB stream

In the following example, replace the Region `us-east-1` and VPC endpoint ID `https://vpce-1a2b3c4d-5e6f.streams.dynamodb.us-east-1.vpce.amazonaws.com` with your own information.

```
ddb_streams_client = session.client(
service_name='dynamodbstreams',
region_name='us-east-1',
endpoint_url='https://vpce-1a2b3c4d-5e6f.streams.dynamodb.us-
east-1.vpce.amazonaws.com'
)
```

SDK for Java 1.x

Example: Use an endpoint URL to access a DynamoDB stream

In the following example, replace the Region `us-east-1` and VPC endpoint ID `https://vpce-1a2b3c4d-5e6f.streams.dynamodb.us-east-1.vpce.amazonaws.com` with your own information.

```
//client build with endpoint config
final AmazonDynamoDBStreams dynamodbstreams =
 AmazonDynamoDBStreamsClientBuilder.standard().withEndpointConfiguration(
        new AwsClientBuilder.EndpointConfiguration(
                "https://vpce-1a2b3c4d-5e6f.streams.dynamodb.us-
east-1.vpce.amazonaws.com",
                Regions.DEFAULT_REGION.getName()
        )
).build();
```

SDK for Java 2.x

Example: Use an endpoint URL to access DynamoDB stream

In the following example, replace the Region `us-east-1` and VPC endpoint ID `https://vpce-1a2b3c4d-5e6f.streams.dynamodb.us-east-1.vpce.amazonaws.com` with your own information.

```
Region region = Region.US_EAST_1;
dynamoDbStreamsClient = DynamoDbStreamsClient.builder().region(region)
.endpointOverride(URI.create("https://vpce-1a2b3c4d-5e6f.streams.dynamodb.us-
east-1.vpce.amazonaws.com"))
.build()
```

Creating an Amazon VPC endpoint policy for DynamoDB Streams

You can attach an endpoint policy to your Amazon VPC endpoint that controls access to DynamoDB Streams. The policy specifies the following information:

- The AWS Identity and Access Management (IAM) principal that can perform actions
- The actions that can be performed
- The resources on which actions can be performed

Topics

- Example: Restricting access to a specific stream from an Amazon VPC endpoint

Example: Restricting access to a specific stream from an Amazon VPC endpoint

You can create an endpoint policy that restricts access to only specific DynamoDB Streams. This type of policy is useful if you have other AWS services in your Amazon VPC that use DynamoDB Streams. The following stream policy restricts access to only the stream *2025-02-20T11:22:33.444* attached to *DOC-EXAMPLE-TABLE*. To use this endpoint policy, replace *DOC-EXAMPLE-TABLE* with the name of your table and *2025-02-20T11:22:33.444* with the stream label.

JSON

```
{
"Version":"2012-10-17",
  "Id": "Policy1216114807515",
  "Statement": [
    { "Sid": "Access-to-specific-stream-only",
      "Principal": "*",
      "Action": [
        "dynamodb:DescribeStream",
        "dynamodb:GetRecords"
      ],
      "Effect": "Allow",
      "Resource": ["arn:aws:dynamodb:us-east-1:111122223333:table/table-name/
stream/2025-02-20T11:22:33.444"]
    }
  ]
}
```

> **Note**
>
> Gateway endpoints aren't supported in DynamoDB Streams.

Using DynamoDB endpoints with AWS Management Console Private Access

You must set up DNS configuration for DynamoDB and DynamoDB Streams when using VPC endpoints with the DynamoDB console in AWS Management Console Private Access.

To configure DynamoDB to be accessible in AWS Management Console Private Access, you must create the following two VPC endpoints:

- `com.amazonaws.<region>.dynamodb`
- `com.amazonaws.<region>.dynamodb-streams`

When you create the VPC endpoints, navitage to the Route53 console and create a private hosted zone for DynamoDB using the regional endpoint `dynamodb.us-east-1.amazonaws.com`.

Create the following two alias records in the private hosted zone:

- `dynamodb.<region>.amazonaws.com` that routes traffic to the VPC endpoint `com.amazonaws.<region>.dynamodb`.
- `streams.dynamodb.<region>.amazonaws.com` that routes traffic to the VPC endpoint `com.amazonaws.<region>.dynamodb-streams`.

Using AWS PrivateLink for DynamoDB Accelerator (DAX)

AWS PrivateLink for DynamoDB Accelerator (DAX) enables you to securely access DAX management APIs such as `CreateCluster`, `DescribeClusters`, and `DeleteCluster` over private IP addresses within your virtual private cloud (VPC). This feature enables you to access DAX services privately from your applications without exposing traffic to the public internet.

DAX PrivateLink supports dual-stack endpoints (`dax.{region}.api.aws`), enabling both IPv4 and IPv6 connectivity. With AWS PrivateLink for DAX, customers can access the service using private DNS names. The dual-stack endpoint support ensures transparent connectivity while maintaining network privacy. This allows you to access DAX through both public internet and VPC endpoints without making any changes to your SDK configuration.

Considerations when using AWS PrivateLink for DynamoDB Accelerator (DAX)

When implementing AWS PrivateLink for DynamoDB Accelerator (DAX), several important considerations must be taken into account.

Before you set up an interface endpoint for DAX consider the following:

- DAX interface endpoints only support access to the DAX management APIs within the same AWS Region. You can't use an interface endpoint to access DAX management APIs in other Regions.
- To access the AWS Management Console privately for DAX management, you may need to create additional VPC endpoints for services like `com.amazonaws.region.console` and related services.
- You are charged for creating and using an interface endpoint to DAX. For pricing information, see AWS PrivateLink pricing.

How AWS PrivateLink works with DAX

When you create an interface endpoint for DAX:

1. AWS creates an endpoint network interface in each subnet you enable for the interface endpoint.
2. These are requester-managed network interfaces that serve as entry points for traffic destined for DAX.
3. You can then access DAX through private IP addresses within your VPC.
4. This architecture allows you to use VPC security groups to manage access to the endpoints.
5. Applications can access both DynamoDB and DAX through their respective interface endpoints within a VPC, while also allowing on-premises applications to connect via Direct Connect or VPN.
6. This provides a consistent connectivity model across both services, simplifies architecture, and improves security by keeping traffic within the AWS network.

Creating Interface Endpoints for DAX

You can create an interface endpoint to connect to DAX using the AWS Management Console, AWS SDK, CloudFormation, or the AWS API.

To create an interface endpoint for DAX using the console

1. Navigate to the Amazon VPC console at https://console.aws.amazon.com/vpc/.
2. In the navigation pane, choose **Endpoints**.
3. Choose **Create Endpoint**.

4. For **Service category**, choose **AWS services** and for **Service Name**, search for and select `com.amazonaws.region.dax`.
5. For **VPC**, select the VPC from which you want to access DAX and for **Subnets**, select the subnets where AWS will create the endpoint network interfaces.
6. For **Security groups**, select or create security groups to associate with the endpoint network interfaces.
7. For **Policy**, keep the default **Full Access** or customize as needed.
8. Select **Enable DNS Name** to enable private DNS for the endpoint. Keep the private DNS name enabled to prevent changes in the SDK configuration. When enabled, your applications can continue using the standard service DNS name (example: `dax.region.amazonaws.com`). AWS creates a private hosted zone in your VPC that resolves this name to your endpoint's private IP address.

   > **Note**
   >
   > Use Regional DNS names if required. Using zonal DNS names isn't recommended. Also, select subnets from 3 or more AZs to ensure maximum availability through PrivateLink.

9. Choose **Create endpoint**.

To create an interface endpoint for DAX using the AWS CLI

Use the `create-vpc-endpoint` command with the `vpc-endpoint-type` parameter set to `Interface` and the `service-name` parameter set to `com.amazonaws.region.dax`.

```
aws ec2 create-vpc-endpoint \
    --vpc-id vpc-ec43eb89 \
    --vpc-endpoint-type Interface \
    --service-name com.amazonaws.us-east-1.dax \
    --subnet-ids subnet-abcd1234 subnet-1a2b3c4d \
    --security-group-ids sg-1a2b3c4d \
    --private-dns-enabled
```

Additional resources

For more information about AWS PrivateLink and VPC endpoints, see the following resources:

- AWS PrivateLink for DynamoDB

- AWS PrivateLink for DynamoDB Streams
- Connect your VPC to services using AWS PrivateLink
- Simplify private connectivity to DynamoDB with AWS PrivateLink
- AWS PrivateLink Whitepaper

Configuration and vulnerability analysis in Amazon DynamoDB

AWS handles basic security tasks like guest operating system (OS) and database patching, firewall configuration, and disaster recovery. These procedures have been reviewed and certified by the appropriate third parties. For more details, see the following resources:

- Compliance validation for Amazon DynamoDB
- Shared responsibility model
- Amazon Web Services: Overview of security processes(whitepaper)

The following security best practices also address configuration and vulnerability analysis in Amazon DynamoDB:

- Monitor DynamoDB compliance with AWS Config Rules
- Monitor DynamoDB configuration with AWS Config

Security best practices for Amazon DynamoDB

Amazon DynamoDB provides a number of security features to consider as you develop and implement your own security policies. The following best practices are general guidelines and don't represent a complete security solution. Because these best practices might not be appropriate or sufficient for your environment, treat them as helpful considerations rather than prescriptions.

Topics

- DynamoDB preventative security best practices
- DynamoDB detective security best practices

DynamoDB preventative security best practices

The following best practices can help you anticipate and prevent security incidents in Amazon DynamoDB.

Encryption at rest

DynamoDB encrypts at rest all user data stored in tables, indexes, streams, and backups using encryption keys stored in AWS Key Management Service (AWS KMS). This provides an additional layer of data protection by securing your data from unauthorized access to the underlying storage .

You can specify whether DynamoDB should use an AWS owned key (default encryption type), an AWS managed key, or a customer managed key to encrypt user data. For more information, see Amazon DynamoDB Encryption at Rest.

Use IAM roles to authenticate access to DynamoDB

For users, applications, and other AWS services to access DynamoDB, they must include valid AWS credentials in their AWS API requests. You should not store AWS credentials directly in the application or EC2 instance. These are long-term credentials that are not automatically rotated, and therefore could have significant business impact if they are compromised. An IAM role enables you to obtain temporary access keys that can be used to access AWS services and resources.

For more information, see Identity and Access Management for Amazon DynamoDB.

Use IAM policies for DynamoDB base authorization

When granting permissions, you decide who is getting them, which DynamoDB APIs they are getting permissions for, and the specific actions you want to allow on those resources. Implementing least privilege is key in reducing security risk and the impact that can result from errors or malicious intent.

Attach permissions policies to IAM identities (that is, users, groups, and roles) and thereby grant permissions to perform operations on DynamoDB resources.

You can do this by using the following:

- AWS Managed (predefined) policies
- Customer managed policies

Use IAM policy conditions for fine-grained access control

When you grant permissions in DynamoDB, you can specify conditions that determine how a permissions policy takes effect. Implementing least privilege is key in reducing security risk and the impact that can result from errors or malicious intent.

You can specify conditions when granting permissions using an IAM policy. For example, you can do the following:

- Grant permissions to allow users read-only access to certain items and attributes in a table or a secondary index.
- Grant permissions to allow users write-only access to certain attributes in a table, based upon the identity of that user.

For more information, see Using IAM Policy Conditions for Fine-Grained Access Control.

Use a VPC endpoint and policies to access DynamoDB

If you only require access to DynamoDB from within a virtual private cloud (VPC), you should use a VPC endpoint to limit access from only the required VPC. Doing this prevents that traffic from traversing the open internet and being subject to that environment.

Using a VPC endpoint for DynamoDB allows you to control and limit access using the following:

- VPC endpoint policies – These policies are applied on the DynamoDB VPC endpoint. They allow you to control and limit API access to the DynamoDB table.
- IAM policies – By using the `aws:sourceVpce` condition on policies attached to users, groups, or roles, you can enforce that all access to the DynamoDB table is via the specified VPC endpoint.

For more information, see Endpoints for Amazon DynamoDB.

Consider client-side encryption

We recommend that you plan your encryption strategy before implementing your table in DynamoDB. If you store sensitive or confidential data in DynamoDB, consider including client-side encryption in your plan. This way you can encrypt data as close as possible to its origin, and ensure its protection throughout its lifecycle. Encrypting your sensitive data in transit and at rest helps ensure that your plaintext data isn't available to any third party.

The AWS Database Encryption SDK for DynamoDB is a software library that helps you protect your table data before you send it to DynamoDB. It encrypts, signs, verifies, and decrypts your DynamoDB table items. You control which attributes are encrypted and signed.

Primary Key considerations

Do not use sensitive names or sensitive plaintext data in your Primary Key for your table and Global Secondary Indexes. Key names will show up in your table definition. For example, the Primary Key names are accessible to anyone with permissions to call DescribeTable. Key values can show up in your AWS CloudTrail and other logs. Additionally, DynamoDB uses the key values to distribute data and route requests and AWS administrators may observe the values to maintain the health of the service.

If you need to use sensitive data in your table or GSI key values, we recommend using end-to-end client encryption. This allows you to perform key-value references to your data while ensuring that it never appears unencrypted in your DynamoDB related logs. One way to accomplish this is to use the AWS Database Encryption SDK for DynamoDB, but that is not required. If you use your own solution, we should always use a sufficiently secure encryption algorithm. You should not use a non-cryptographic option like a hash, as they are not considered sufficiently secure in most situations.

If your Primary Key key names are sensitive, we recommend using `pk` and `sk` instead. This is a general best practice which leaves your Partition Key design flexible.

Always consult your security experts or AWS account team if you are concerned about what the right choice would be.

DynamoDB detective security best practices

The following best practices for Amazon DynamoDB can help you detect potential security weaknesses and incidents.

Use AWS CloudTrail to monitor AWS managed KMS key usage

If you are using an AWS managed key for encryption at rest, usage of this key is logged into AWS CloudTrail. CloudTrail provides visibility into user activity by recording actions taken on your account. CloudTrail records important information about each action, including who made the request, the services used, the actions performed, parameters for the actions, and the response elements returned by the AWS service. This information helps you track changes made to your AWS resources and troubleshoot operational issues. CloudTrail makes it easier to ensure compliance with internal policies and regulatory standards.

You can use CloudTrail to audit key usage. CloudTrail creates log files that contain a history of AWS API calls and related events for your account. These log files include all AWS KMS API requests made using the AWS Management Console, AWS SDKs, and command line tools, in addition to those made through integrated AWS services. You can use these log files to get information about when the KMS key was used, the operation that was requested, the identity of the requester, the IP address that the request came from, and so on. For more information, see Logging AWS KMS API Calls with AWS CloudTrail and the AWS CloudTrail User Guide.

Monitor DynamoDB operations using CloudTrail

CloudTrail can monitor both control plane events and data plane events. Control plane operations let you create and manage DynamoDB tables. They also let you work with indexes, streams, and other objects that are dependent on tables. Data plane operations let you perform create, read, update, and delete (also called *CRUD*) actions on data in a table. Some data plane operations also let you read data from a secondary index. To enable logging of data plane events in CloudTrail, you'll need to enable logging of data plane API activity in CloudTrail. See Logging data events for trails for more information.

When activity occurs in DynamoDB, that activity is recorded in a CloudTrail event along with other AWS service events in the event history. For more information, see Logging DynamoDB Operations by Using AWS CloudTrail. You can view, search, and download recent events in your AWS account. For more information, see Viewing Events with CloudTrail Event History in the *AWS CloudTrail User Guide*.

For an ongoing record of events in your AWS account, including events for DynamoDB, create a trail. A trail enables CloudTrail to deliver log files to an Amazon Simple Storage Service (Amazon S3) bucket. By default, when you create a trail on the console, the trail applies to all AWS Regions. The trail logs events from all Regions in the AWS partition and delivers the log files to the S3 bucket that you specify. Additionally, you can configure other AWS services to further analyze and act upon the event data collected in CloudTrail logs.

Use DynamoDB Streams to monitor data plane operations

DynamoDB is integrated with AWS Lambda so that you can create triggers—pieces of code that automatically respond to events in DynamoDB Streams. With triggers, you can build applications that react to data modifications in DynamoDB tables.

If you enable DynamoDB Streams on a table, you can associate the stream Amazon Resource Name (ARN) with a Lambda function that you write. Immediately after an item in the table is modified, a new record appears in the table's stream. AWS Lambda polls the stream and invokes

your Lambda function synchronously when it detects new stream records. The Lambda function can perform any actions that you specify, such as sending a notification or initiating a workflow.

For an example, see Tutorial: Using AWS Lambda with Amazon DynamoDB Streams. This example receives a DynamoDB event input, processes the messages that it contains, and writes some of the incoming event data to Amazon CloudWatch Logs.

Monitor DynamoDB configuration with AWS Config

Using AWS Config, you can continuously monitor and record configuration changes of your AWS resources. You can also use AWS Config to inventory your AWS resources. When a change from a previous state is detected, an Amazon Simple Notification Service (Amazon SNS) notification can be delivered for you to review and take action. Follow the guidance in Setting Up AWS Config with the Console, ensuring that DynamoDB resource types are included.

You can configure AWS Config to stream configuration changes and notifications to an Amazon SNS topic. For example, when a resource is updated, you can get a notification sent to your email, so that you can view the changes. You can also be notified when AWS Config evaluates your custom or managed rules against your resources.

For an example, see Notifications that AWS Config Sends to an Amazon SNS topic in the *AWS Config Developer Guide*.

Monitor DynamoDB compliance with AWS Config rules

AWS Config continuously tracks the configuration changes that occur among your resources. It checks whether these changes violate any of the conditions in your rules. If a resource violates a rule, AWS Config flags the resource and the rule as noncompliant.

By using AWS Config to evaluate your resource configurations, you can assess how well your resource configurations comply with internal practices, industry guidelines, and regulations. AWS Config provides AWS managed rules, which are predefined, customizable rules that AWS Config uses to evaluate whether your AWS resources comply with common best practices.

Tag your DynamoDB resources for identification and automation

You can assign metadata to your AWS resources in the form of tags. Each tag is a simple label consisting of a customer-defined key and an optional value that can make it easier to manage, search for, and filter resources.

Tagging allows for grouped controls to be implemented. Although there are no inherent types of tags, they enable you to categorize resources by purpose, owner, environment, or other criteria. The following are some examples:

- Security – Used to determine requirements such as encryption.
- Confidentiality – An identifier for the specific data-confidentiality level a resource supports.
- Environment – Used to distinguish between development, test, and production infrastructure.

For more information, see AWS Tagging Strategies and Tagging for DynamoDB.

Monitor your usage of Amazon DynamoDB as it relates to security best practices by using AWS Security Hub CSPM.

Security Hub CSPM uses security controls to evaluate resource configurations and security standards to help you comply with various compliance frameworks.

For more information about using Security Hub CSPM to evaluate DynamoDB resources, see Amazon DynamoDB controls in the *AWS Security Hub CSPM User Guide*.